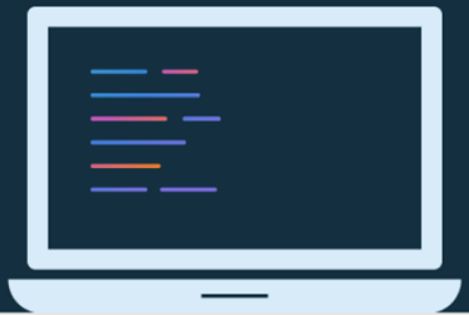


Lesson 1: Kotlin basics



This work is licensed under the Apache 2 license.

About this lesson

Lesson 1: Kotlin basics

- Get started
- Operators
- Data types
- Variables
- Conditionals
- Lists and arrays
- Null safety
- Summary

This work is licensed under the Apache 2 license.

Get started

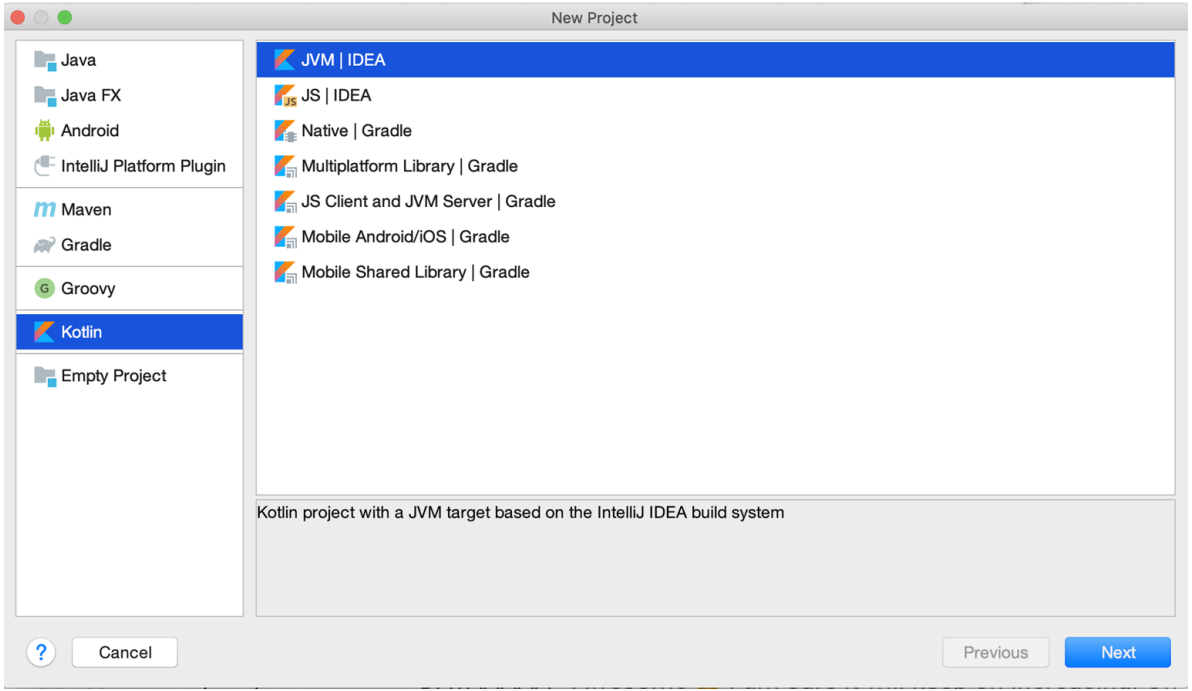
This work is licensed under the Apache 2 license.

Open IntelliJ IDEA



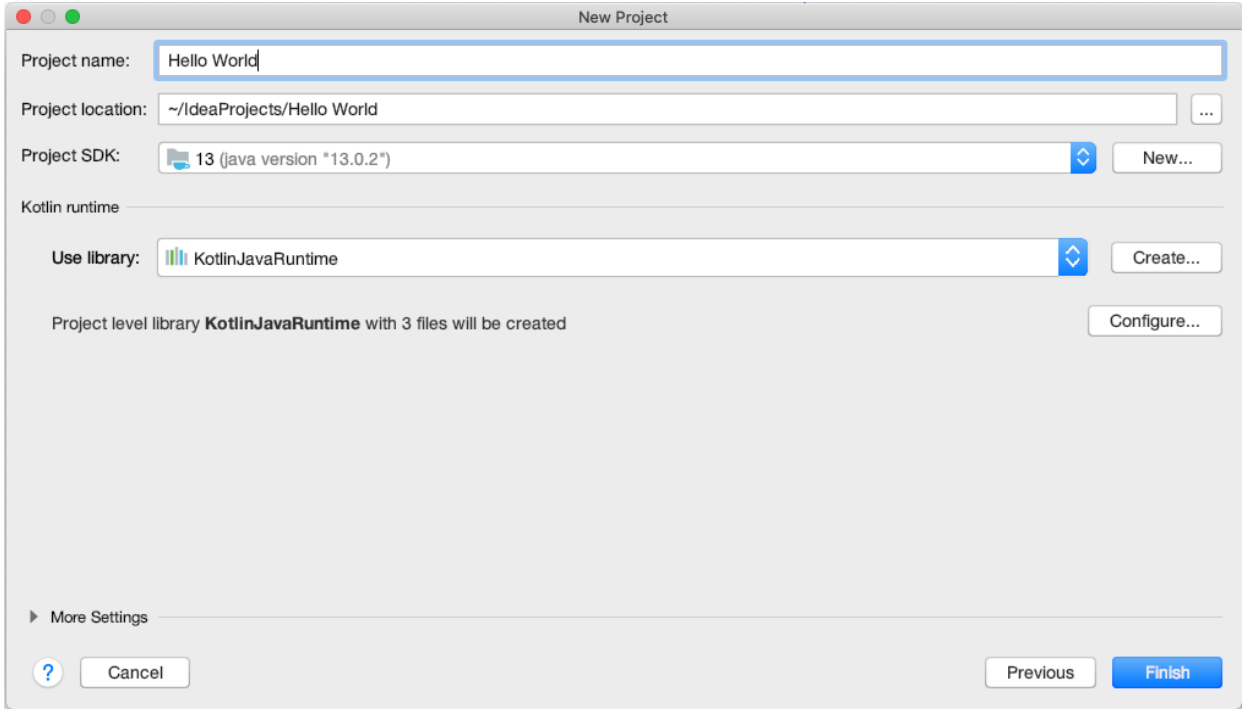
This work is licensed under the Apache 2 license.

Create a new project



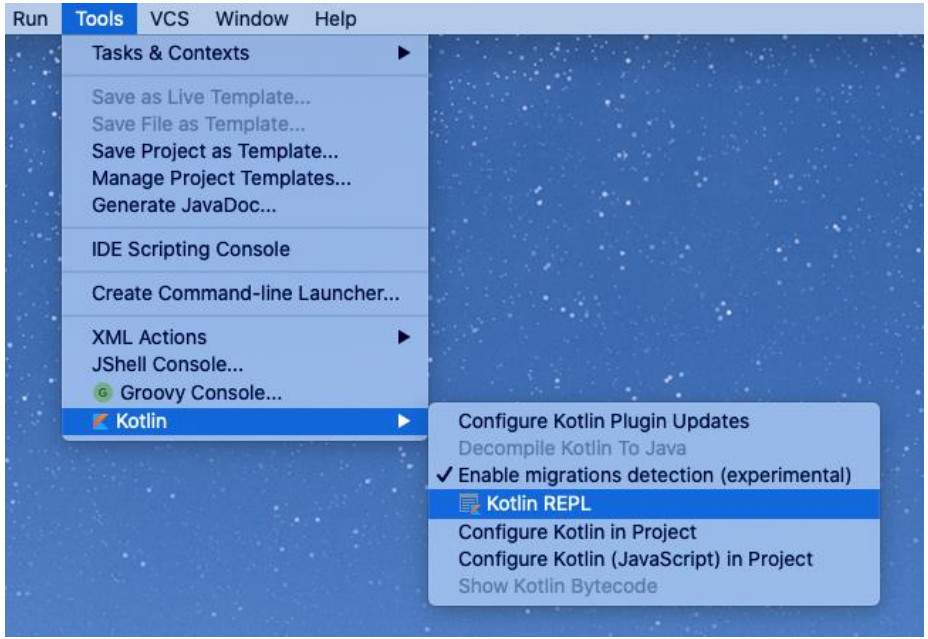
This work is licensed under the Apache 2 license.

Name the project



This work is licensed under the Apache 2 license.

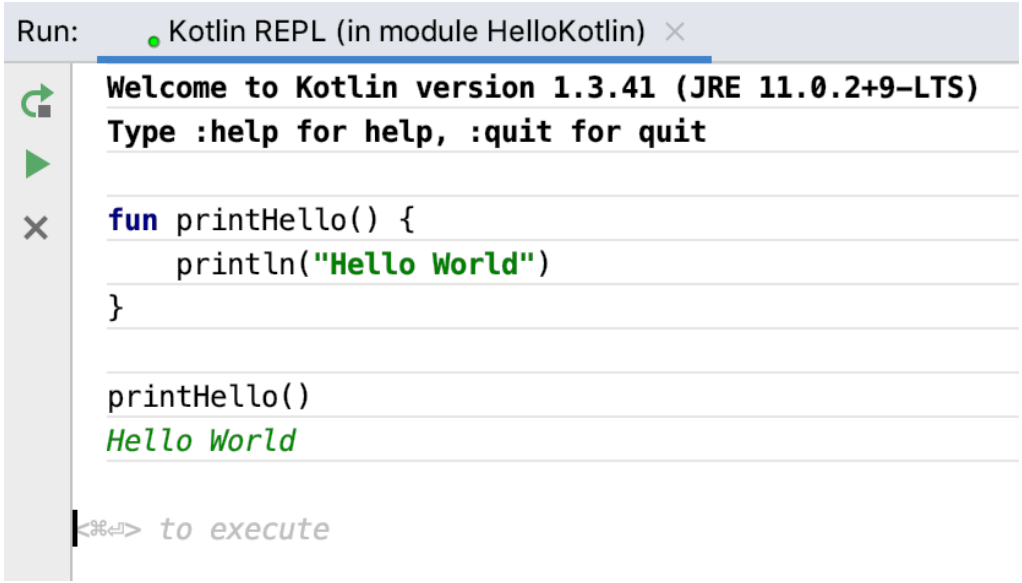
Open REPL (Read-Eval-Print-Loop)



It may take a few moments before the Kotlin menu appears under **Tools**.

This work is licensed under the Apache 2 license.

Create a printHello() function



Press **Control+Enter** (**Command+Enter** on a Mac) to execute.

This work is licensed under the Apache 2 license.

Operators

This work is licensed under the Apache 2 license.

Operators

- Mathematical operators `+ - * / %`
- Increment and decrement operators `++ --`
- Comparison operators `< <= > >=`
- Assignment operator `=`
- Equality operators `== !=`

This work is licensed under the Apache 2 license.

Math operators with integers

```
1 + 1      =>      2

53 - 3     =>      50

50 / 10    =>      5

9 % 3      =>      0
```


This work is licensed under the Apache 2 license.

Math operators with doubles

```
1.0 / 2.0    =>    0.5

2.0 * 3.5    =>    7.0
```


This work is licensed under the Apache 2 license.

Math operators

```
1+1
⇒ kotlin.Int = 2
```

```
1.0/2.0
⇒ kotlin.Double = 0.5
```

```
53-3
⇒ kotlin.Int = 50
```

```
2.0*3.5
⇒ kotlin.Double = 7.0
```

```
50/10
⇒ kotlin.Int = 5
```

⇒ indicates output from your code.

Result includes the type (`kotlin.Int`).

This work is licensed under the Apache 2 license.

Numeric operator methods

Kotlin keeps numbers as primitives, but lets you call methods on numbers as if they were objects.

```
2.times(3)
⇒ kotlin.Int = 6

3.5.plus(4)
⇒ kotlin.Double = 7.5

2.4.div(2)
⇒ kotlin.Double =
1.2
```


This work is licensed under the Apache 2 license.

Data types

This work is licensed under the Apache 2 license.

Integer types

Type	Bits	Notes
Long	64	From -2^{63} to $2^{63}-1$
Int	32	From -2^{31} to $2^{31}-1$
Short	16	From -32768 to 32767
Byte	8	From -128 to 127

This work is licensed under the Apache 2 license.

Floating-point and other numeric types

Type	Bits	Notes
Double	64	16 - 17 significant digits
Float	32	6 - 7 significant digits
Char	16	16-bit Unicode character
Boolean	8	True or false. Operations include: || - lazy disjunction, && - lazy conjunction, ! - negation

This work is licensed under the Apache 2 license.

Operand types

Results of operations keep the types of the operands

```
6*50
⇒ kotlin.Int = 300
```

```
1/2
⇒ kotlin.Int = 0
```

```
6.0*50.0
⇒ kotlin.Double = 300.0
```

```
1.0*2.0
⇒ kotlin.Double = 0.5
```

```
6.0*50
⇒ kotlin.Double = 300.0
```


This work is licensed under the Apache 2 license.

Type casting

Assign an `Int` to a `Byte`

```
val i: Int = 6
val b: Byte = i
println(b)
```

```
⇒ error: type mismatch: inferred type is Int but Byte was
expected
```

Convert `Int` to `Byte` with casting

```
val i: Int = 6
println(i.toByte())
```

```
⇒ 6
```


This work is licensed under the Apache 2 license.

Underscores for long numbers

Use underscores to make long numeric constants more readable.

```
val oneMillion = 1_000_000

val idNumber = 999_99_9999L

val hexBytes = 0xFF_EC_DE_5E

val bytes = 0b11010010_01101001_10010100_10010010
```


This work is licensed under the Apache 2 license.

Strings

Strings are any sequence of characters enclosed by double quotes.

```
val s1 = "Hello world!"
```

String literals can contain escape characters

```
val s2 = "Hello world!\n"
```

Or any arbitrary text delimited by a triple quote (""")

```
val text = """
    var bikes = 50
"""
```


This work is licensed under the Apache 2 license.

String concatenation

```
val numberOfDogs = 3
val numberOfCats = 2

"I have $numberOfDogs dogs" + " and $numberOfCats cats"

=> I have 3 dogs and 2 cats
```


This work is licensed under the Apache 2 license.

String templates

A template expression starts with a dollar sign ($) and can be a simple value:

```
val i = 10
println("i = $i")
=> i = 10
```

Or an expression inside curly braces:

```
val s = "abc"
println("$s.length is ${s.length}")
=> abc.length is 3
```


This work is licensed under the Apache 2 license.

String template expressions

```
val numberOfShirts = 10
val numberOfPants = 5

"I have ${numberOfShirts + numberOfPants} items of clothing"

=> I have 15 items of clothing
```


This work is licensed under the Apache 2 license.

Variables

This work is licensed under the Apache 2 license.

Variables

- Powerful type inference
  - Let the compiler infer the type
  - You can explicitly declare the type if needed
- Mutable and immutable variables
  - Immutability not enforced, but recommended

Kotlin is a statically-typed language. The type is resolved at compile time and never changes.

This work is licensed under the Apache 2 license.

Specifying the variable type

Colon Notation

```
var width: Int = 12
var length: Double = 2.5
```

Important: Once a type has been assigned by you or the compiler, you can't change the type or you get an error.

This work is licensed under the Apache 2 license.

Mutable and immutable variables

- Mutable (Changeable)

```
var score = 10
```

- Immutable (Unchangeable)

```
val name = "Jennifer"
```

Although not strictly enforced, using immutable variables is recommended in most cases.

This work is licensed under the Apache 2 license.

var and val

```
var count = 1
count = 2

val size = 1
size = 2

=> Error: val cannot be reassigned
```


This work is licensed under the Apache 2 license.

Conditionals

This work is licensed under the Apache 2 license.

Control flow

Kotlin features several ways to implement conditional logic:

- If/Else statements
- When statements
- For loops
- While loops

This work is licensed under the Apache 2 license.

if/else statements

```
val numberOfCups = 30
val numberOfPlates = 50

if (numberOfCups > numberOfPlates) {
    println("Too many cups!")
} else {
    println("Not enough cups!")
}
```

```
=> Not enough cups!
```


This work is licensed under the Apache 2 license.

if statement with multiple cases

```
val guests = 30
if (guests == 0) {
    println("No guests")
} else if (guests < 20) {
    println("Small group of people")
} else {
    println("Large group of people!")
}
⇒ Large group of people!
```


This work is licensed under the Apache 2 license.

Ranges

- Data type containing a span of comparable values (e.g., integers from 1 to 100 inclusive)
- Ranges are bounded
- Objects within a range can be mutable or immutable

This work is licensed under the Apache 2 license.

Ranges in if/else statements

```
val numberOfStudents = 50
if (numberOfStudents in 1..100) {
    println(numberOfStudents)
}
=> 50
```

Note: There are no spaces around the "range to" operator `(1..100)`

This work is licensed under the Apache 2 license.

when statement

```
when (results) {
    0  -> println("No results")
    in 1..39 -> println("Got results!")
    else -> println("That's a lot of results!")
}
⇒ That's a lot of results!
```

As well as a `when` statement, you can also define a `when` expression that provides a return value.

This work is licensed under the Apache 2 license.

for loops

```
val pets = arrayOf("dog", "cat", "canary")
for (element in pets) {
    print(element + " ")
}
⇒ dog cat canary
```

You don’t need to define an iterator variable and increment it for each pass.

This work is licensed under the Apache 2 license.

for loops: elements and indexes

```
for ((index, element) in pets.withIndex()) {
    println("Item at $index is $element\n")
}
⇒ Item at 0 is dog
Item at 1 is cat
Item at 2 is canary
```


This work is licensed under the Apache 2 license.

for loops: step sizes and ranges

```
for (i in 1..5) print(i)
⇒ 12345

for (i in 5 downTo 1) print(i)
⇒ 54321

for (i in 3..6 step 2) print(i)
⇒ 35

for (i in 'd'..'g') print (i)
⇒ defg
```


This work is licensed under the Apache 2 license.

while loops

```
var bicycles = 0
while (bicycles < 50) {
    bicycles++
}
println("$bicycles bicycles in the bicycle rack\n")
⇒ 50 bicycles in the bicycle rack

do {
    bicycles--
} while (bicycles > 50)
println("$bicycles bicycles in the bicycle rack\n")
⇒ 49 bicycles in the bicycle rack
```


This work is licensed under the Apache 2 license.

repeat loops

```
repeat(2) {
      print("Hello!")
}
```

```
⇒ Hello!Hello!
```


This work is licensed under the Apache 2 license.

Lists and arrays

This work is licensed under the Apache 2 license.

Lists

- Lists are ordered collections of elements
- List elements can be accessed programmatically through their indices
- Elements can occur more than once in a list

An example of a list is a sentence: it's a group of words, their order is important, and they can repeat.

This work is licensed under the Apache 2 license.

Immutable list using listOf()

Declare a list using `listOf()` and print it out.

```
val instruments = listOf("trumpet", "piano", "violin")
println(instruments)
```

```
⇒ [trumpet, piano, violin]
```


This work is licensed under the Apache 2 license.

Mutable list using mutableListOf()

Lists can be changed using `mutableListOf()`

```
val myList = mutableListOf("trumpet", "piano", "violin")
myList.remove("violin")

⇒ kotlin.Boolean = true
```

With a list defined with `val`, you can't change which list the variable refers to, but you can still change the contents of the list.

This work is licensed under the Apache 2 license.

Arrays

- Arrays store multiple items
- Array elements can be accessed programmatically through their indices
- Array elements are mutable
- Array size is fixed

This work is licensed under the Apache 2 license.

Array using arrayOf()

An array of strings can be created using `arrayOf()`

```
val pets = arrayOf("dog", "cat", "canary")
println(java.util.Arrays.toString(pets))
⇒ [dog, cat, canary]
```

With an array defined with `val`, you can't change which array the variable refers to, but you can still change the contents of the array.

This work is licensed under the Apache 2 license.

Arrays with mixed or single types

An array can contain different types.

```
val mix = arrayOf("hats", 2)
```

An array can also contain just one type (integers in this case).

```
val numbers = intArrayOf(1, 2, 3)
```


This work is licensed under the Apache 2 license.

Combining arrays

Use the + operator.

```
val numbers = intArrayOf(1,2,3)
val numbers2 = intArrayOf(4,5,6)
val combined = numbers2 + numbers
println(Arrays.toString(combined))

=> [4, 5, 6, 1, 2, 3]
```


This work is licensed under the Apache 2 license.

Null safety

This work is licensed under the Apache 2 license.

Null safety

- In Kotlin, variables cannot be null by default
- You can explicitly assign a variable to null using the safe call operator
- Allow null-pointer exceptions using the `!!` operator
- You can test for null using the elvis (`?:`) operator

This work is licensed under the Apache 2 license.

Variables cannot be null

In Kotlin, `null` variables are not allowed by default.

Declare an `Int` and assign `null` to it.

```
var numberOfBooks: Int = null
```

không khai báo được kiểu này

```
⇒ error: null can not be a value of a non-null type Int
```


This work is licensed under the Apache 2 license.

Safe call operator

The safe call operator (`?`), after the type indicates that a variable can be `null`.

Declare an `Int?` as nullable

```
var numberOfBooks: Int? = null
```

khai báo hợp lệ

In general, do not set a variable to null as it may have unwanted consequences.

This work is licensed under the Apache 2 license.

Testing for null

Check whether the `numberOfBooks` variable is not `null`. Then decrement that variable.

```
var numberOfBooks = 6
if (numberOfBooks != null) {
    numberOfBooks = numberOfBooks.dec()
}
```

nếu biến khác null -> giảm đi 1 đơn vị

Now look at the Kotlin way of writing it, using the safe call operator.

```
var numberOfBooks = 6
numberOfBooks = numberOfBooks?.dec()
```

viết ngắn hơn

This work is licensed under the Apache 2 license.

The !! operator

nếu chắc chắn biến không null

buộc

If you're certain a variable won't be null, use `!!` to force the variable into a non-null type. Then you can call methods/properties on it.

```
val len = s!!.length
```

biến s buộc phải không null (nếu null thì ném ra ngoại lệ)

throws NullPointerException if s is null

Warning: Because `!!` will throw an exception, it should only be used when it would be exceptional to hold a null value.

This work is licensed under the Apache 2 license.

Elvis operator

Chain null tests with the `?:` operator.

```
numberOfBooks = numberOfBooks?.dec() ?: 0
```

The `?:` operator is sometimes called the "Elvis operator," because it's like a smiley on its side with a pompadour hairstyle, like Elvis Presley styled his hair.

This work is licensed under the Apache 2 license.

Summary

This work is licensed under the Apache 2 license.

Summary

In Lesson 1, you learned how to:

- Create an IntelliJ IDEA project, opening REPL, and execute a function
- Use operators and numeric operator methods
- Use data types, type casting, strings, and string templates
- Use variables and type inference, and mutable and immutable variables
- Use conditionals, control flow, and looping structures
- Use lists and arrays
- Use Kotlin's null safety features

This work is licensed under the Apache 2 license.

Pathway

Practice what you've learned by completing the pathway:

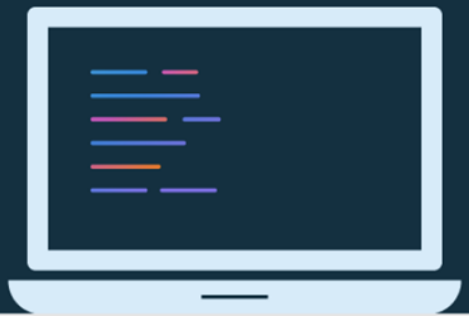
Lesson 1: Kotlin basics



This work is licensed under the Apache 2 license.



Lesson 2: Functions



This work is licensed under the Apache 2 license.

About this lesson

Lesson 2: Functions

- Programs in Kotlin
- (Almost) Everything has a value
- Functions in Kotlin
- Compact functions
- Lambdas and higher-order functions
- List filters
- Summary

This work is licensed under the Apache 2 license.

Programs in Kotlin

This work is licensed under the Apache 2 license.

Setting up

Before you can write code and run programs, you need to:

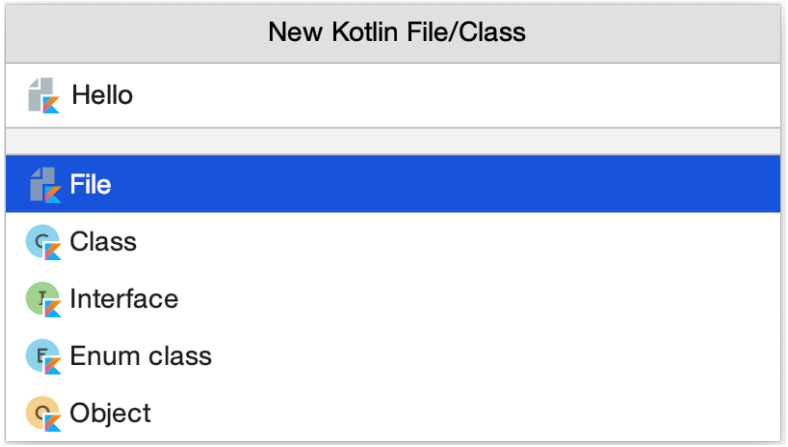
- Create a file in your project
- Create a `main()` function
- Pass arguments to `main()` (Optional)
- Use any passed arguments in function calls (Optional)
- Run your program

This work is licensed under the Apache 2 license.

Create a new Kotlin file

In IntelliJ IDEA's Project pane, under **Hello World**, right-click the `src` folder.

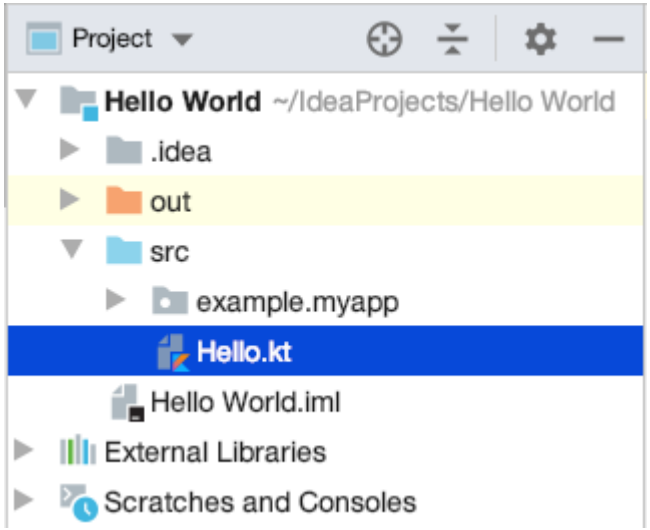
- Select **New > Kotlin File/Class**.
- Select **File**, name the file `Hello,` and press **Enter**.



This work is licensed under the Apache 2 license.

Create a Kotlin file

You should now see a file in the `src` folder called `Hello.kt`.



This work is licensed under the Apache 2 license.

Create a main() function

`main()` is the entry point for execution for a Kotlin program.

In the `Hello.kt` file:

```
fun main(args: Array<String>) {
    println("Hello, world!")
}
```

The args in the `main()` function are optional.

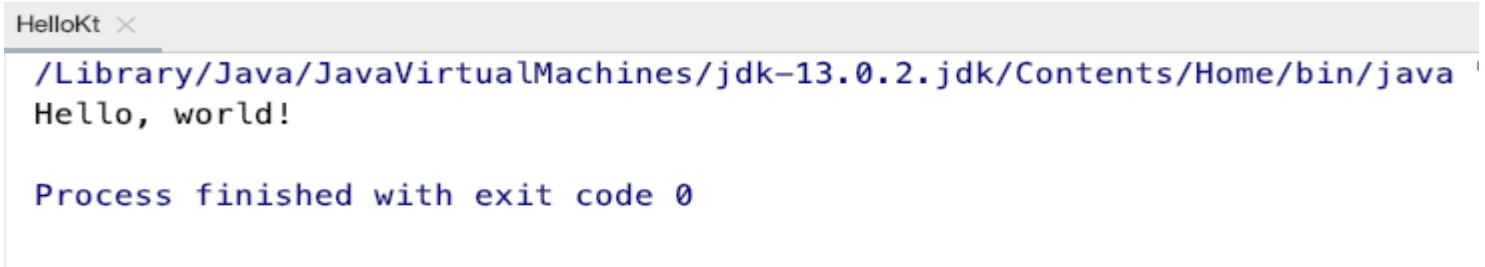
This work is licensed under the Apache 2 license.

Run your Kotlin program

To run your program, click the Run icon (▶) to the left of the `main()` function.

```
1 ▶ fun main(args: Array<String>) {
2       println("Hello, world!")
3   }
4
```

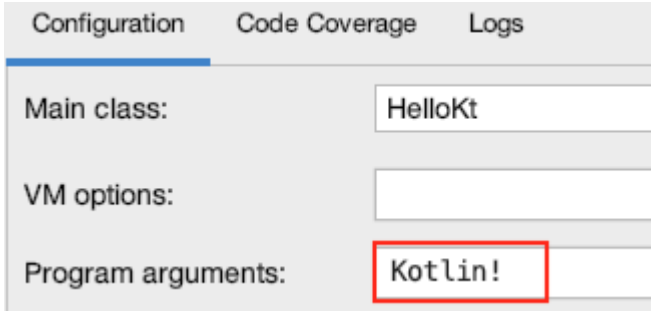
IntelliJ IDEA runs the program, and displays the results in the console.



This work is licensed under the Apache 2 license.

Pass arguments to main()

Select **Run > Edit Configurations** to open the **Run/Debug Configurations** window.



This work is licensed under the Apache 2 license.

Use arguments in `main()`

Use `args[0]` to access the first input argument passed to `main()`.

```
fun main(args: Array<String>) {
    println("Hello, ${args[0]}")
}
```

```
⇒ Hello, Kotlin!
```


This work is licensed under the Apache 2 license.

(Almost) Everything has a value

This work is licensed under the Apache 2 license.

(Almost) Everything is an expression

In Kotlin, almost everything is an expression and has a value. Even an `if` expression has a value.

```
val temperature = 20

val isHot = if (temperature > 40) true else false

println(isHot)

⇒ false
```


This work is licensed under the Apache 2 license.

Expression values

Sometimes, that value is `kotlin.Unit`.

```
val isUnit = println("This is an expression")

println(isUnit)

⇒ This is an expression
  kotlin.Unit
```


This work is licensed under the Apache 2 license.

Functions in Kotlin

This work is licensed under the Apache 2 license.

About functions

- A block of code that performs a specific task
- Breaks a large program into smaller modular chunks
- Declared using the `fun` keyword
- Can take arguments with either named or default values

This work is licensed under the Apache 2 license.

Parts of a function

Earlier, you created a simple function that printed "Hello World".

```
fun printHello() {
    println("Hello World")
}

printHello()
```


This work is licensed under the Apache 2 license.

Unit returning functions

If a function does not return any useful value, its return type is `Unit`.

đối số truyền vào chấp nhận null

```
fun printHello(name: String?): Unit {
    println("Hi there!")
}
```

`Unit` is a type with only one value: `Unit`.

This work is licensed under the Apache 2 license.

Unit returning functions

The `Unit` return type declaration is optional.

```
fun printHello(name: String?): Unit {
    println("Hi there!")
}
```

is equivalent to:

```
fun printHello(name: String?) {
    println("Hi there!")
}
```


This work is licensed under the Apache 2 license.

Function arguments

Functions may have:

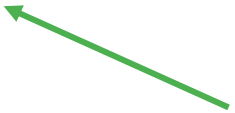
- Default parameters
- Required parameters
- Named arguments

This work is licensed under the Apache 2 license.

Default parameters

Default values provide a fallback if no parameter value is passed.

```
fun drive(speed: String = "fast") {
    println("driving $speed")
}
```



Use "=" after the type to define default values

nếu như không truyền đối số vào thì mặc định speep = "fast"

```
drive() ⇒ driving fast

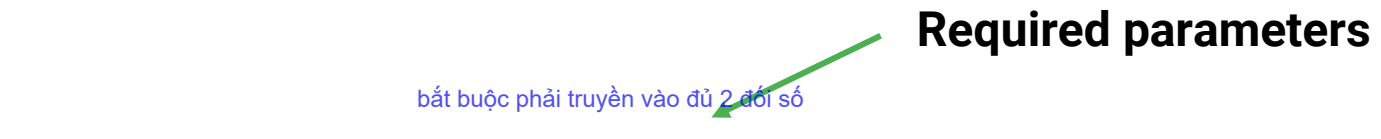
drive("slow") ⇒ driving slowly

drive(speed = "turtle-like") ⇒ driving turtle-like
```


This work is licensed under the Apache 2 license.

Required parameters

If no default is specified for a parameter, the corresponding argument is required.




```
fun tempToday(day: String, temp: Int) {
    println("Today is $day and it's $temp degrees.")
}
```


This work is licensed under the Apache 2 license.

Default versus required parameters

func có thể vừa có default vừa có mặc định

Functions can have a mix of default and required parameters.

```
fun reformat(str: String,
                        divideByCamelHumps: Boolean,
             wordSeparator: Char,
             normalizeCase: Boolean = true){
```



Pass in required arguments.

```
reformat("Today is a day like no other day", false, '_')
```


This work is licensed under the Apache 2 license.

Named arguments

To improve readability, use named arguments for required arguments.

```
reformat(str, divideByCamelHumps = false, wordSeparator = '_')
```

It's considered good style to put default arguments after positional arguments, that way callers only have to specify the required arguments.

This work is licensed under the Apache 2 license.

Compact functions

This work is licensed under the Apache 2 license.

Single-expression functions

Compact functions, or single-expression functions, make your code more concise and readable.

```
fun double(x: Int): Int {
    x * 2
}
```



Complete version

```
fun double(x: Int):Int = x * 2
```



Compact version

This work is licensed under the Apache 2 license.

Lambdas and higher-order functions

This work is licensed under the Apache 2 license.

Kotlin functions are first-class

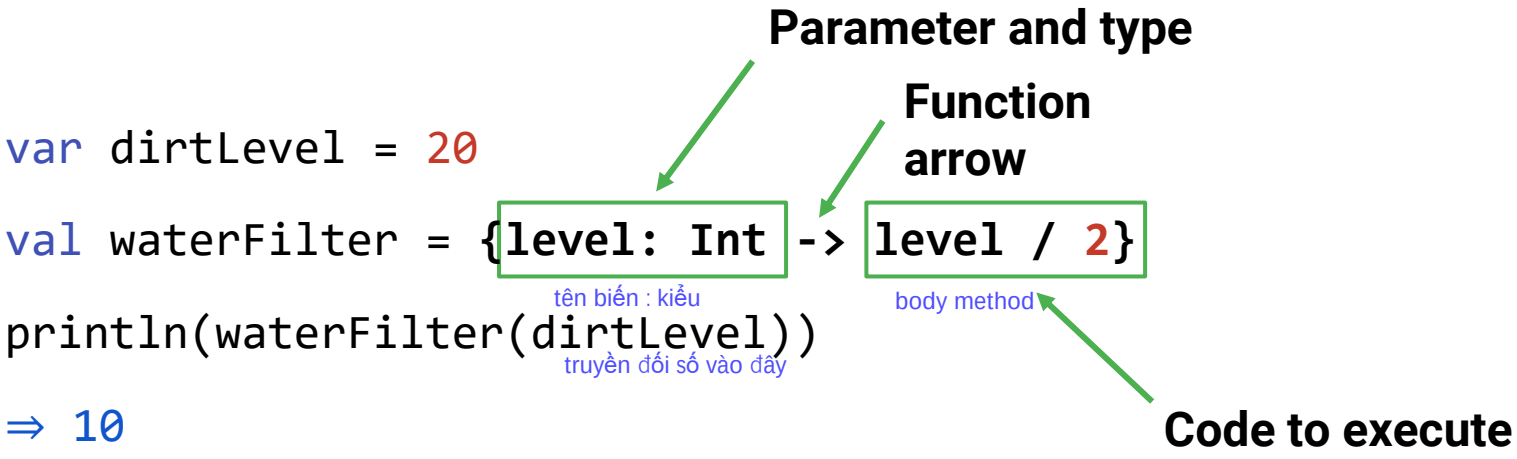
hàm trong kotlin có thể chứa biến và cả data

- Kotlin functions can be stored in variables and data structures
- They can be passed as arguments to, and returned from, other higher-order functions
- You can use higher-order functions to create new "built-in" functions

This work is licensed under the Apache 2 license.

Lambda functions

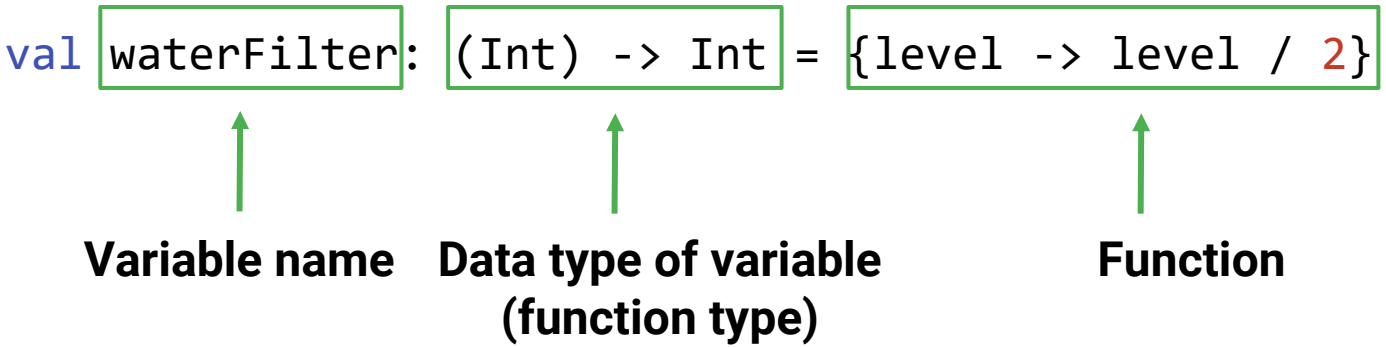
A lambda is an expression that makes a function that has no name.



This work is licensed under the Apache 2 license.

Syntax for function types

Kotlin's syntax for function types is closely related to its syntax for lambdas. Declare a variable that holds a function.



This work is licensed under the Apache 2 license.

Higher-order functions

Higher-order functions take functions as parameters, or return a function.

```
fun encodeMsg(msg: String, encode: (String) -> String): String {
    return encode(msg)
}
```

The body of the code calls the function that was passed as the second argument, and passes the first argument along to it.

This work is licensed under the Apache 2 license.

Higher-order functions

To call this function, pass it a string and a function.

```
val enc1: (String) -> String = { input -> input.toUpperCase() }
println(encodeMsg("abc", enc1))
```

Using a function type separates its implementation from its usage.

This work is licensed under the Apache 2 license.

Passing a function reference

Use the `::` operator to pass a named function as an argument to another function.

```
fun enc2(input:String): String = input.reversed()

encodeMessage("abc", ::enc2)
```

Passing a named function, not a lambda

The `::` operator lets Kotlin know that you are passing the function reference as an argument, and not trying to call the function.

This work is licensed under the Apache 2 license.

Last parameter call syntax

Kotlin prefers that any parameter that takes a function is the last parameter.

```
encodeMessage("acronym", { input -> input.toUpperCase() })
```

You can pass a lambda as a function parameter without putting it inside the parentheses.

```
encodeMsg("acronym") { input -> input.toUpperCase() }
```


This work is licensed under the Apache 2 license.

Using higher-order functions

Many Kotlin built-in functions are defined using last parameter call syntax.

```
inline fun repeat(times: Int, action: (Int) -> Unit)
```

```
repeat(3) {
    println("Hello")
}
```


This work is licensed under the Apache 2 license.

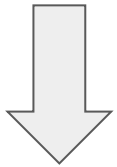
List filters

This work is licensed under the Apache 2 license.

List filters

Get part of a list based on some condition

red	red-orange	dark red	orange	bright orange	saffron



Apply `filter()` on list

Condition: element contains “red”

red	red-orange	dark red

This work is licensed under the Apache 2 license.

Iterating through lists

If a function literal has only one parameter, you can omit its declaration and the "`->`". The parameter is implicitly declared under the name `it`.

```
val ints = listOf(1, 2, 3)
ints.filter { it > 0 }
```

Filter iterates through a collection, where `it` is the value of the element during the iteration. This is equivalent to:

```
ints.filter { n: Int -> n > 0 }   OR   ints.filter { n -> n > 0 }
```


This work is licensed under the Apache 2 license.

List filters

The filter condition in curly braces `{}` tests each item as the filter loops through. If the expression returns `true`, the item is included.

```
val books = listOf("nature", "biology", "birds")

println(books.filter { it[0] == 'b' })
```

lọc ra các phần tử bắt đầu bằng chữ 'b'

```
⇒ [biology, birds]
```


This work is licensed under the Apache 2 license.

Eager and lazy filters

Evaluation of expressions in lists:

- **Eager:** occurs regardless of whether the result is ever used
- **Lazy:** occurs only if necessary at runtime

chỉ chạy nếu cần thiết

Lazy evaluation of lists is useful if you don't need the entire result, or if the list is exceptionally large and multiple copies wouldn't wouldn't fit into RAM.

This work is licensed under the Apache 2 license.

Eager filters

Filters are eager by default. A new list is created each time you use a filter.

```
val instruments = listOf("viola", "cello", "violin")
val eager = instruments.filter { it [0] == 'v' }
println("eager: " + eager)
⇒ eager: [viola, violin]
```


This work is licensed under the Apache 2 license.

Lazy filters

Sequences are data structures that use lazy evaluation, and can be used with filters to make them lazy.

```
val instruments = listOf("viola", "cello", "violin")
val filtered = instruments.asSequence().filter { it[0] == 'v'}
println("filtered: " + filtered)
```

```
⇒ filtered: kotlin.sequences.FilteringSequence@386cc1c4
```


This work is licensed under the Apache 2 license.

Sequences -> lists

Sequences can be turned back into lists using `toList()`.

```
val filtered = instruments.asSequence().filter { it[0] == 'v'}
val newList = filtered.toList()
println("new list: " + newList)
```

```
⇒ new list: [viola, violin]
```


This work is licensed under the Apache 2 license.

Other list transformations

- `map()` performs the same transform on every item and returns the list.

```
val numbers = setOf(1, 2, 3)
println(numbers.map { it * 3 })
=> [3, 6, 9]
```

map dùng để biến đổi giá trị

- `flatten()` returns a single list of all the elements of nested collections.

```
val numberSets = listOf(setOf(1, 2, 3), setOf(4, 5), setOf(1, 2))
println(numberSets.flatten())
=> [1, 2, 3, 4, 5, 1, 2]
```


This work is licensed under the Apache 2 license.

Summary

This work is licensed under the Apache 2 license.

Summary

In Lesson 2, you learned how to:

- Create a file and a `main()` function in your project, and run a program
- Pass arguments to the `main()` function
- Use the returned value of an expression
- Use default arguments to replace multiple versions of a function
- Use compact functions, to make code more readable
- Use lambdas and higher-order functions
- Use eager and lazy list filters

This work is licensed under the Apache 2 license.

Pathway

Practice what you've learned by completing the pathway:

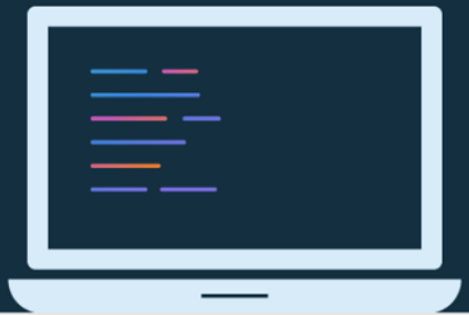
Lesson 2: Functions



This work is licensed under the Apache 2 license.



Lesson 3: Classes and objects



This work is licensed under the Apache 2 license.

About this lesson

Lesson 3: Classes and objects

- Classes
- Inheritance
- Extension functions
- Special classes
- Organizing your code
- Summary

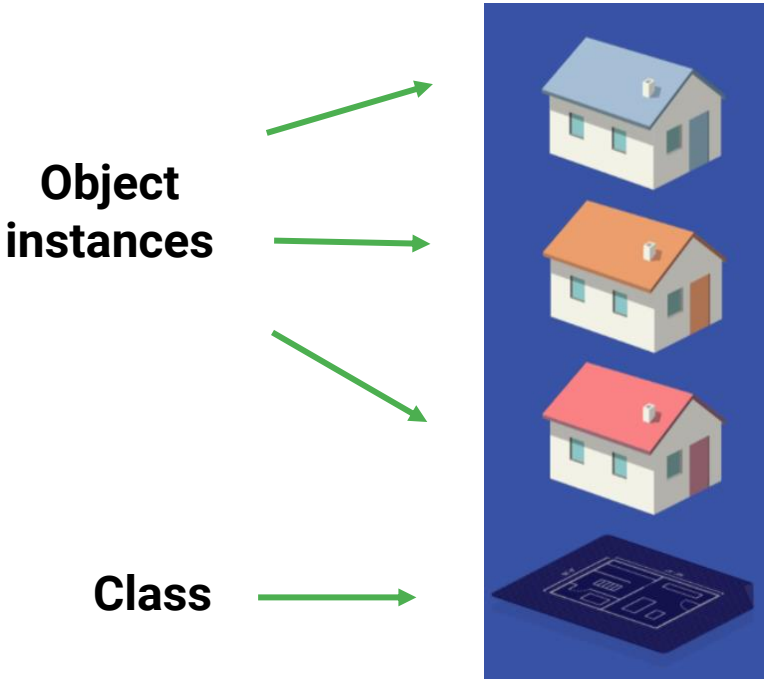
This work is licensed under the Apache 2 license.

Classes

This work is licensed under the Apache 2 license.

Class

- Classes are blueprints for objects
- Classes define methods that operate on their object instances



This work is licensed under the Apache 2 license.

Class versus object instance

House Class

Data

- House color (String)
- Number of windows (Int)
- Is for sale (Boolean)

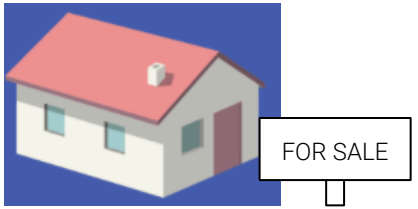
Behavior

- `updateColor()`
- `putOnSale()`

Object Instances







This work is licensed under the Apache 2 license.

Define and use a class

Class Definition

```
class House {
  val color: String = "white"
  val numberOfWindows: Int = 2
  val isForSale: Boolean = false

  fun updateColor(newColor: String){...}
  ...
}
```

Create New Object Instance

```
val myHouse = House()
println(myHouse)
```


This work is licensed under the Apache 2 license.

Constructors

When a constructor is defined in the class header, it can contain:

- No parameters

```
class A
```

- Parameters
  - Not marked with `var` or `val` → copy exists only within scope of the constructor

```
class B(x: Int)
```

  - Marked `var` or `val` → copy exists in all instances of the class

```
class C(val y: Int)
```


This work is licensed under the Apache 2 license.

Constructor examples

```
class A
```

```
val aa = A()
```

```
class B(x: Int)
```

```
val bb = B(12)
println(bb.x)
=> compiler error unresolved
reference
```

```
class C(val y: Int)
```

```
val cc = C(42)
println(cc.y)
=> 42
```


This work is licensed under the Apache 2 license.

Default parameters

Class instances can have default values.

sử dụng default value làm giảm số lương constructor cần thiết

- Use default values to reduce the number of constructors needed

các tham số mặc định có thể trộn lẫn với tham số bắt buộc

- Default parameters can be mixed with required parameters
- More concise (don't need to have multiple constructor versions)

nhắn gọn hơn (không cần phải có nhiều constructor version_

```
class Box(val length: Int, val width:Int = 20, val height:Int = 40)

val box1 = Box(100, 20, 40)
val box2 = Box(length = 100)
val box3 = Box(length = 100, width = 20, height = 40)
```

viết như thế này thì đỡ phải viết nhiều constructor -> đỡ dài code

This work is licensed under the Apache 2 license.

Primary constructor

Declare the primary constructor within the class header.

```
class Circle(i: Int) {
    init {
        ...
    }
}
```

This is technically equivalent to:

```
class Circle {
    constructor(i: Int) {
        ...
    }
}
```


This work is licensed under the Apache 2 license.

Initializer block

- Any required initialization code is run in a special `init` block
- Multiple `init` blocks are allowed
- `init` blocks become the body of the primary constructor

This work is licensed under the Apache 2 license.

Initializer block example

Use the `init` keyword:

```
class Square(val side: Int) {
    init {
        println(side * 2)
    }
}

val s = Square(10)
=> 20
```


This work is licensed under the Apache 2 license.

Multiple constructors

sử dụng từ khóa ..... để khởi tạo secondary constructor

- Use the `constructor` keyword to define secondary constructors
- Secondary constructors must call:
  - The primary constructor using `this` keyword

    OR

  - Another secondary constructor that calls the primary constructor
- Secondary constructor body is not required

This work is licensed under the Apache 2 license.

Multiple constructors example

```
class Circle(val radius:Double) {
    constructor(name:String) : this(1.0)
    constructor(diameter:Int) : this(diameter / 2.0) {
        println("in diameter constructor")
    }
    init {
        println("Area: ${Math.PI * radius * radius}")
    }
}
val c = Circle(3)
```


This work is licensed under the Apache 2 license.

Properties

- Define properties in a class using `val` or `var`
- Access these properties using
  dot `.` notation with property name
- Set these properties using
  dot `.` notation with property name (only if declared a `var`)

This work is licensed under the Apache 2 license.

Person class with name property

```
class Person(var name: String)

fun main() {
    val person = Person("Alex")
    println(person.name)
    person.name = "Joey"
    println(person.name)
}
```

Access with .<property name>

Set with .<property name>

This work is licensed under the Apache 2 license.

Custom getters and setters

If you don't want the default `get/set` behavior:

- Override `get()` for a property
- Override `set()` for a property (if defined as a `var`)

Format:

```
var propertyName: DataType = initialValue
        get() = ...
        set(value) {
            ...
        }
```


This work is licensed under the Apache 2 license.

Custom getter

```
class Person(val firstName: String, val lastName:String) {
    val fullName:String
        get() {
            return "$firstName $lastName"
        }
}

val person = Person("John", "Doe")
println(person.fullName)
=> John Doe
```


This work is licensed under the Apache 2 license.

Custom setter

```
var fullName:String = ""
    get() = "$firstName $lastName"
    set(value) {
        val components = value.split(" ")
        firstName = components[0]
        lastName = components[1]
        field = value
    }

person.fullName = "Jane Smith"
```


This work is licensed under the Apache 2 license.

Member functions

- Classes can also contain functions
- Declare functions as shown in ***Functions*** in Lesson 2
  - `fun` keyword
  - Can have default or required parameters
  - Specify return type (if not `Unit`)

This work is licensed under the Apache 2 license.

Inheritance

kế thừa

This work is licensed under the Apache 2 license.

Inheritance

- Kotlin has single-parent class inheritance
- Each class has exactly one parent class, called a superclass
- Each subclass inherits all members of its superclass including ones that the superclass itself has inherited

If you don't want to be limited by only inheriting a single class, you can define an interface since you can implement as many of those as you want.

This work is licensed under the Apache 2 license.

Interfaces

- Provide a contract all implementing classes must adhere to
- Can contain method signatures and property names
- Can derive from other interfaces

Format: `interface NameOfInterface { interfaceBody }`

This work is licensed under the Apache 2 license.

Interface example

```
interface Shape {
    fun computeArea() : Double
}

class Circle(val radius:Double) : Shape {
    override fun computeArea() = Math.PI * radius * radius
}


val c = Circle(3.0)
println(c.computeArea())
=> 28.274333882308138
```


This work is licensed under the Apache 2 license.

Extending classes

To extend a class:

- Create a new class that uses an existing class as its core (subclass)
- Add functionality to a class without creating a new one (extension functions)

This work is licensed under the Apache 2 license.

Creating a new class

- Kotlin classes by default are not subclassable
- Use `open` keyword to allow subclassing
- Properties and functions are redefined with the `override` keyword

This work is licensed under the Apache 2 license.

Classes are final by default

Declare a class

```
class A
```

Try to subclass `A`

```
class B : A
```

```
=>Error: A is final and cannot be inherited from
```


This work is licensed under the Apache 2 license.

Use open keyword

Use `open` to declare a class so that it can be subclassed.

muốn con kết thừa cha thì cha phải open

Declare a class

```
open class C
```

Subclass from C

```
class D : C()
```


This work is licensed under the Apache 2 license.

Overriding

sử dụng từ khóa open để cho lớp con ghi đè lại phuonwg thức hay att của cha

- Must use `open` for properties and methods that can be overridden (otherwise you get compiler error)

cần có từ kháo overide khi ghi đè phương thức hay thuộc tính

- Must use `override` when overriding properties and methods
- Something marked `override` can be overridden in subclasses (unless marked `final`)

nếu đánh dấu là final thì không ghi đè đc

This work is licensed under the Apache 2 license.

Abstract classes

đánh dấu

- Class is marked as `abstract`

khởi tạo kế thừa

- Cannot be instantiated, must be subclassed
- Similar to an interface with the added the ability to store state
- Properties and functions marked with `abstract` must be overridden

các thuộc tính và chưcs năng đc đánh dấu là abstract cần phải đc ghi đè lại

- Can include non-abstract properties and functions

có thể chứa cả nhưng att hay method non-abstract

This work is licensed under the Apache 2 license.

Example abstract classes

```
abstract class Food {
    abstract val kcal : Int
    abstract val name : String
    fun consume() = println("I'm eating ${name}")
}
class Pizza() : Food() {
    override val kcal = 600
    override val name = "Pizza"
}
fun main() {
    Pizza().consume()    // "I'm eating Pizza"
}
```

khi một lớp con kế thừa cha nó
nếu nó ghi đè lại phương thức của cha nó thì khi gọi nó trong main thì sẽ gọi đến phương thức nó đã ghi đè,còn khi nó ko ghi đè lại phương thức của cha nó thì khi gọi nó sẽ dùng đến method của cha gần nhất của nó

This work is licensed under the Apache 2 license.

When to use each

- Defining a broad spectrum of behavior or types? Consider an interface.
- Will the behavior be specific to that type? Consider a class.
- Need to inherit from multiple classes? Consider refactoring code to see if some behavior can be isolated into an interface.
- Want to leave some properties / methods abstract to be defined by subclasses? Consider an abstract class.
- You can extend only one class, but implement one or more interfaces.

chỉ có đơn kế thừa

kotlin cho phép đa implement như java

This work is licensed under the Apache 2 license.

Extension functions

This work is licensed under the Apache 2 license.

Extension functions

Add functions to an existing class that you cannot modify directly.

- Appears as if the implementer added it
- Not actually modifying the existing class
- Cannot access private instance variables

Format: `fun ClassName.functionName( params ) { body }`

This work is licensed under the Apache 2 license.

Why use extension functions?

- Add functionality to classes that are not **open**
- Add functionality to classes you don't own
- Separate out core API from helper methods for classes you own

Define extension functions in an easily discoverable place such as in the same file as the class, or a well-named function.

This work is licensed under the Apache 2 license.

Extension function example

Add `isOdd()` to `Int` class:

```
fun Int.isOdd(): Boolean { return this % 2 == 1 }
```

Call `isOdd()` on an `Int`:

```
3.isOdd()
```

Extension functions are very powerful in Kotlin!

ngayf 2/7 hoc den day roi

This work is licensed under the Apache 2 license.

Special classes

This work is licensed under the Apache 2 license.

Data class

- Special class that exists just to store a set of data

class đặc biệt dành cho việc lưu trữ data

- Mark the class with the `data` keyword

khi khai báo data class
-> 2 đối tượng sẽ so sánh với nhau về nội dụng chứ không so sánh về địa chỉ ô nhớ

- Generates getters for each property (and setters for vars too)
- Generates `toString()`, `equals()`, `hashCode()`, `copy()` methods, and destructuring operators

Format: `data class <NameOfClass>( parameterList )`

This work is licensed under the Apache 2 license.

Data class example

Define the data class:

```
data class Player(val name: String, val score: Int)
```

Use the data class:

```
val firstPlayer = Player("Lauren", 10)
println(firstPlayer)
=> Player(name=Lauren, score=10)
```

Data classes make your code much more concise! lamf cho code súc tích/ngắn gọn hơn

This work is licensed under the Apache 2 license.

Pair and Triple

- `Pair` and `Triple` are predefined data classes that store 2 or 3 pieces of data respectively
- Access variables with `.first`, `.second`, `.third` respectively
- Usually named `data` classes are a better option (more meaningful names for your use case)

This work is licensed under the Apache 2 license.

Pair and Triple examples

```
val bookAuthor = Pair("Harry Potter", "J.K. Rowling")

println(bookAuthor)
=> (Harry Potter, J.K. Rowling)

val bookAuthorYear = Triple("Harry Potter", "J.K. Rowling", 1997)
println(bookAuthorYear)

println(bookAuthorYear.third)
=> (Harry Potter, J.K. Rowling, 1997)
    1997
```


This work is licensed under the Apache 2 license.

Pair to

`Pair`'s special `to` variant lets you omit parentheses and periods (infix function).

It allows for more readable code

```
val bookAuth1 = "Harry Potter".to("J. K. Rowling")
val bookAuth2 = "Harry Potter" to "J. K. Rowling"
=> bookAuth1 and bookAuth2 are Pair (Harry Potter, J. K. Rowling)
```

Also used in collections like Map and HashMap

```
val map = mapOf(1 to "x", 2 to "y", 3 to "zz")
=> map of Int to String {1=x, 2=y, 3=zz}
```


This work is licensed under the Apache 2 license.

Enum class

User-defined data type for a set of named values

- Use `this` to require instances be one of several constant values
- The constant value is, by default, not visible to you
- Use `enum` before the `class` keyword

Format: `enum class EnumName { NAME1, NAME2, … NAMEn }`

Referenced via `EnumName.<ConstantName>`

This work is licensed under the Apache 2 license.

Enum class example

Define an `enum` with red, green, and blue colors.

```
enum class Color(val r: Int, val g: Int, val b: Int) {
    RED(255, 0, 0), GREEN(0, 255, 0), BLUE(0, 0, 255)
}

println("" + Color.RED.r + " " + Color.RED.g + " " + Color.RED.b)
=> 255 0 0
```


This work is licensed under the Apache 2 license.

Object/singleton

- Sometimes you only want one instance of a class to ever exist
- Use the `object` keyword instead of the `class` keyword
- Accessed with `NameOfObject.<function or variable>`

This work is licensed under the Apache 2 license.

Object/singleton example

```
object Calculator {
    fun add(n1: Int, n2: Int): Int {
        return n1 + n2
    }
}

println(Calculator.add(2,4))
=> 6
```


This work is licensed under the Apache 2 license.

Companion objects

- Lets all instances of a class share a single instance of a set of variables or functions
- Use `companion` keyword
- Referenced via `ClassName.PropertyOrFunction`

This work is licensed under the Apache 2 license.

Companion object example

```
class PhysicsSystem {
    companion object WorldConstants {
        val gravity = 9.8
        val unit = "metric"
        fun computeForce(mass: Double, accel: Double): Double {
            return mass * accel
        }
    }
}
println(PhysicsSystem.WorldConstants.gravity)
println(PhysicsSystem.WorldConstants.computeForce(10.0, 10.0))
=> 9.8100.0
```


This work is licensed under the Apache 2 license.

Organizing your code

This work is licensed under the Apache 2 license.

Single file, multiple entities

- Kotlin DOES NOT enforce a single entity (class/interface) per file convention
- You can and should group related structures in the same file
- Be mindful of file length and clutter

This work is licensed under the Apache 2 license.

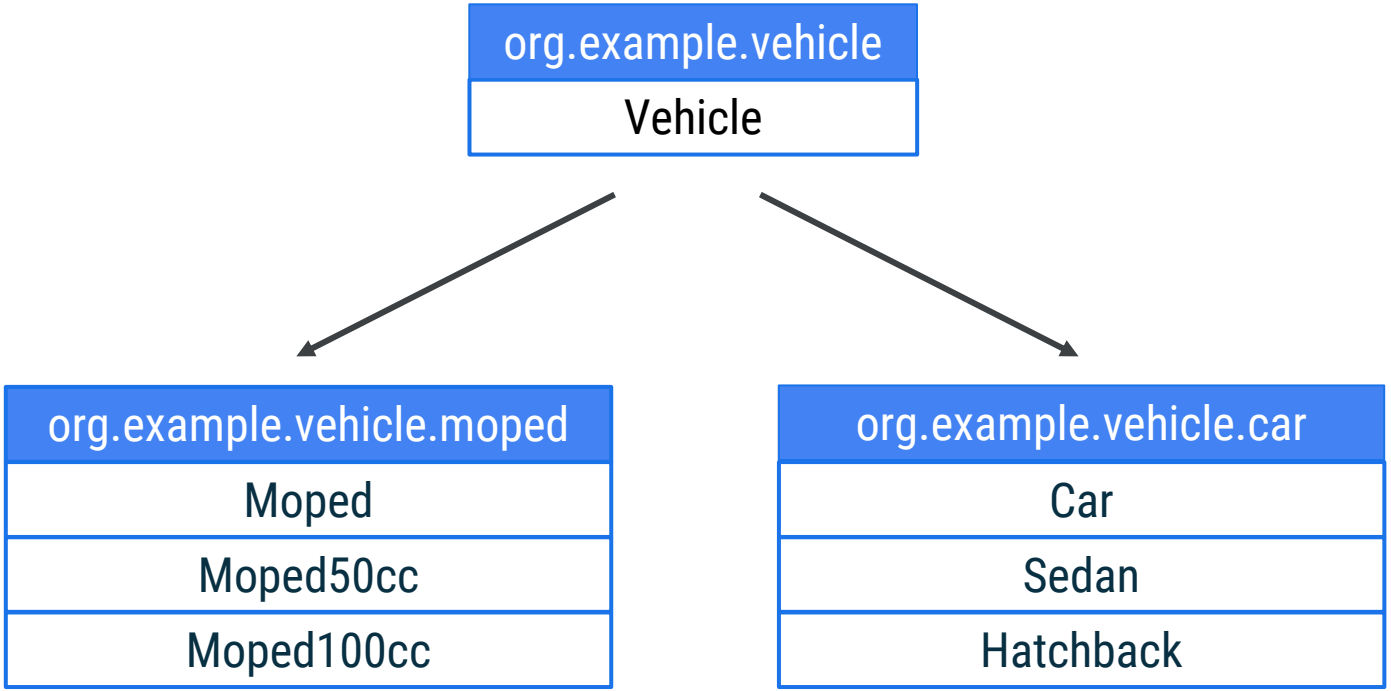
Packages

- Provide means for organization
- Identifiers are generally lower case words separated by periods
- Declared in the first non-comment line of code in a file following the `package` keyword

```
package org.example.game
```


This work is licensed under the Apache 2 license.

Example class hierarchy



This work is licensed under the Apache 2 license.

Visibility modifiers

Use visibility modifiers to limit what information you expose.

rộng nhất : ở đâu cx có thể truy cập được

- `public` means visible outside the class. Everything is public by default, including variables and methods of the class.

private chỉ có thể truy cập trong class

- `private` means it will only be visible in that class (or source file if you are working with functions).

protected cho phép truy cập trong class và các subclass của nó

- `protected` is the same as `private`, but it will also be visible to any subclasses.

This work is licensed under the Apache 2 license.

Summary

This work is licensed under the Apache 2 license.

Summary

In Lesson 3, you learned about:

- Classes, constructors, and getters and setters
- Inheritance, interfaces, and how to extend classes
- Extension functions
- Special classes: data classes, enums, object/singletons, companion objects
- Packages
- Visibility modifiers

This work is licensed under the Apache 2 license.

Pathway

Practice what you've learned by completing the pathway:

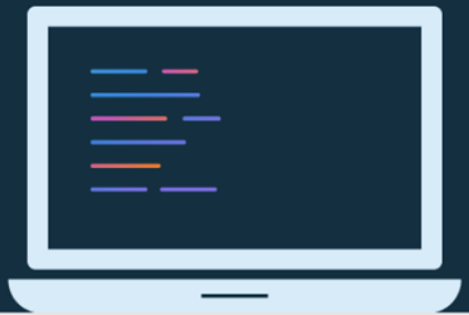
Lesson 3: Classes and objects



This work is licensed under the Apache 2 license.



Lesson 4: Build your first Android app



This work is licensed under the Apache 2 license.

About this lesson

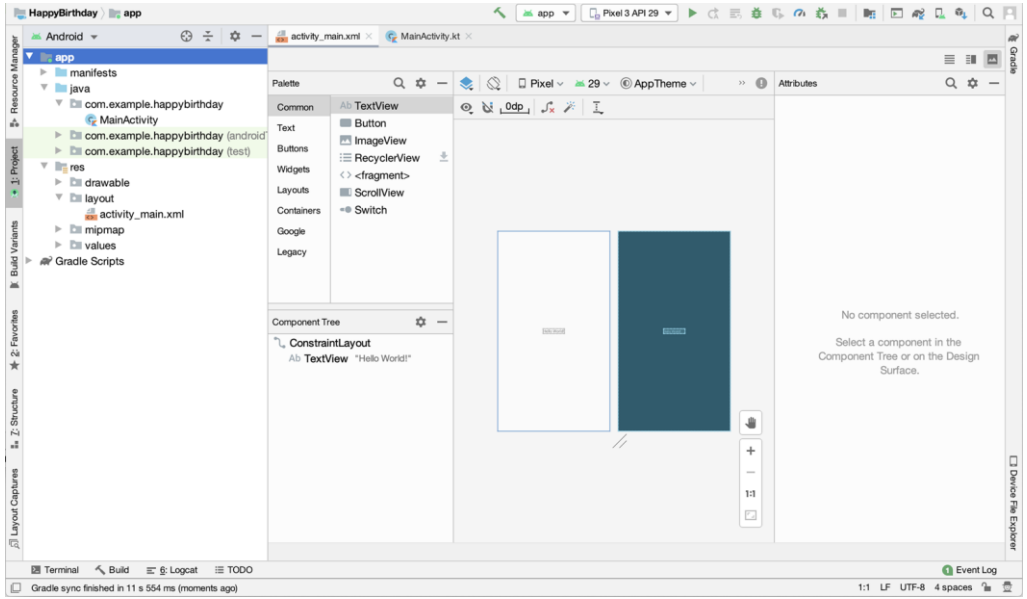
Lesson 4: Build your first Android app

- Your first app
- Anatomy of an Android app
- Layouts and resources in Android
- Activities
- Make an app interactive
- Gradle: Building an Android app
- Accessibility
- Summary

This work is licensed under the Apache 2 license.

Android Studio

Official IDE for building Android apps

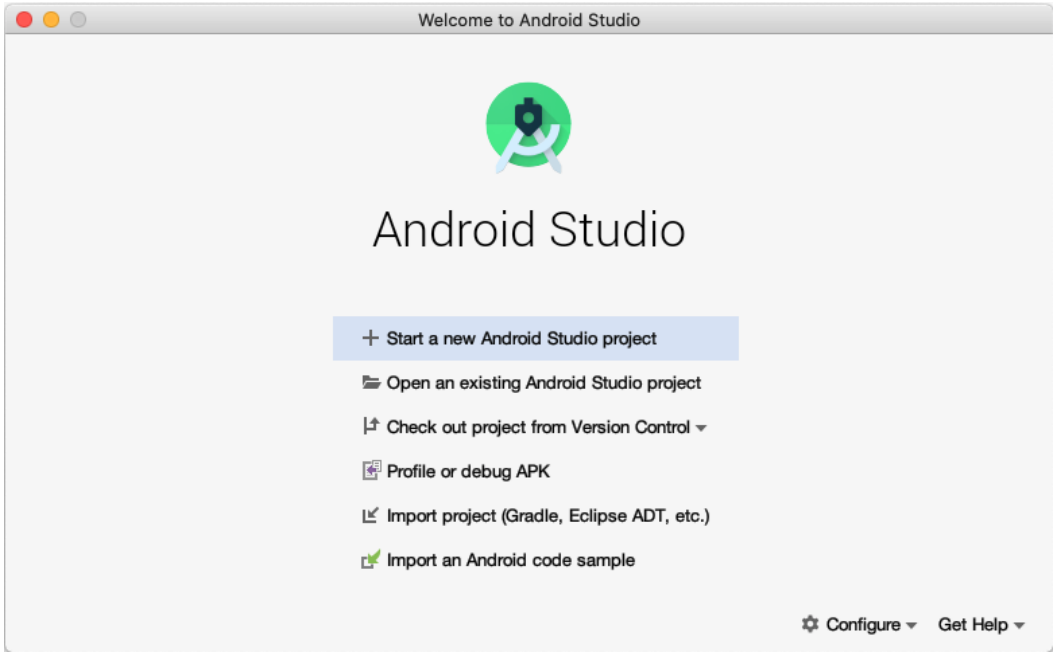


This work is licensed under the Apache 2 license.

Your first app

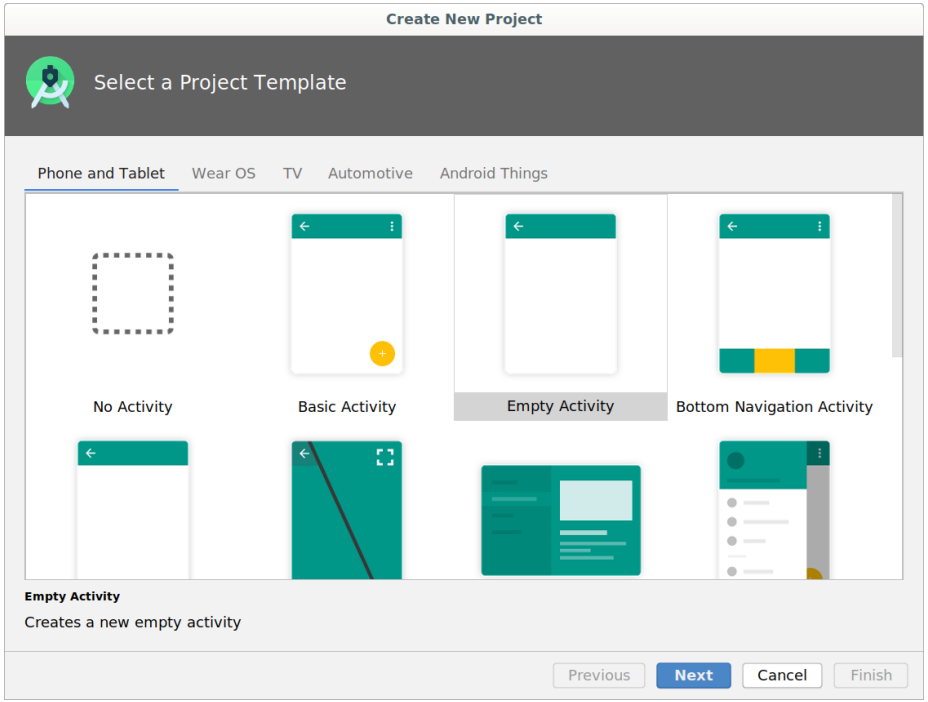
This work is licensed under the Apache 2 license.

Open Android Studio



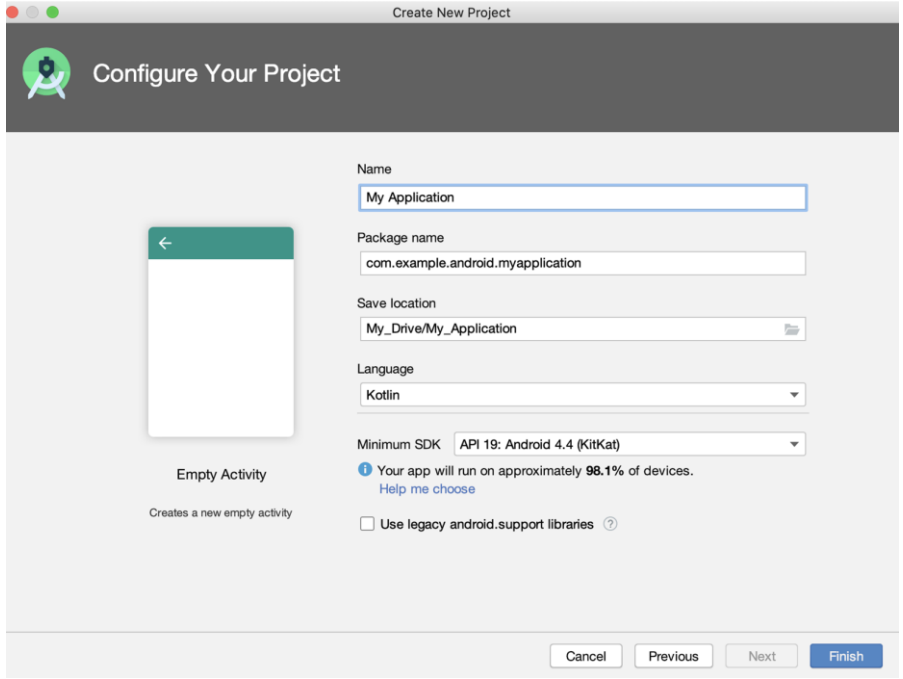
This work is licensed under the Apache 2 license.

Create new project



This work is licensed under the Apache 2 license.

Enter your project details



This work is licensed under the Apache 2 license.

Android releases and API levels

Platform Version	API Level	VERSION_CODE
Android 10.0	29	Q
Android 9	28	P
Android 8.1	27	O_MR1
Android 8.0	26	O
Android 7.1.1 Android 7.1	25	N_MR1
Android 7.0	24	N
Android 6.0	23	M
Android 5.1	22	LOLLIPOP_MR1
Android 5.0	21	LOLLIPOP

This work is licensed under the Apache 2 license.

Choose API levels for your app

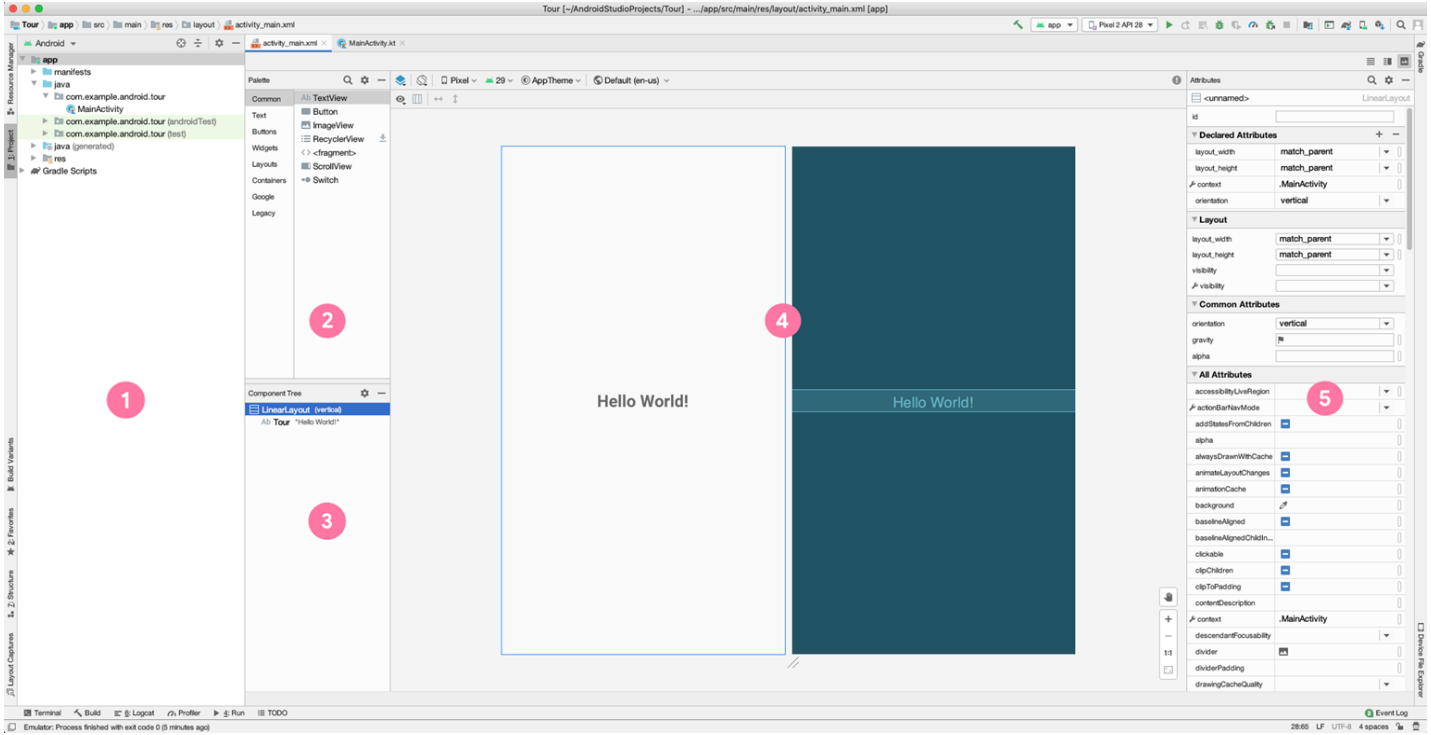
- Minimum SDK: Device needs at least this API level to install
- Target SDK: API version and highest Android version tested
- Compile SDK: Android OS library version compiled with

```
minSdkVersion <= targetSdkVersion <= compileSdkVersion
```

The API level identifies the framework API version of the Android SDK.

This work is licensed under the Apache 2 license.

Tour of Android Studio



This work is licensed under the Apache 2 license.

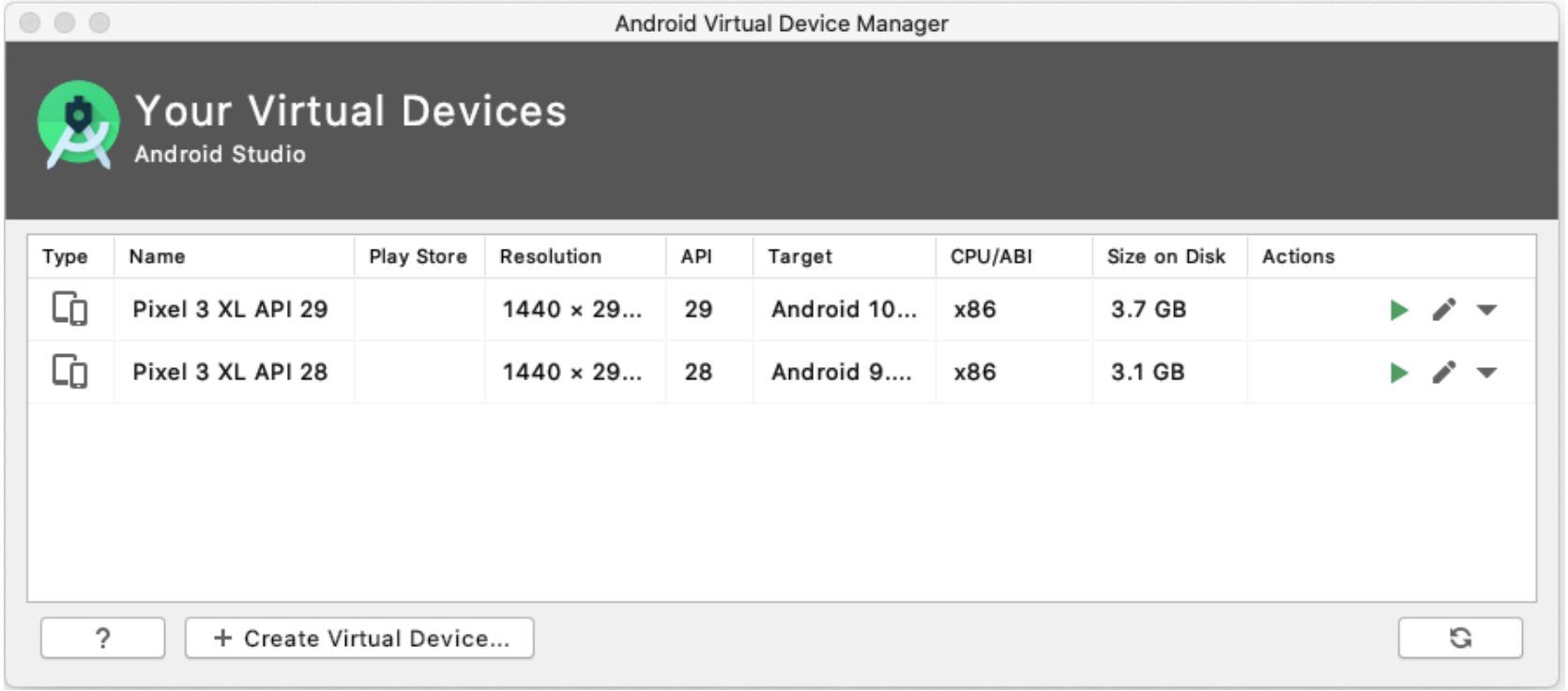
Run your app



- Android device (phone, tablet)
- Emulator on your computer

This work is licensed under the Apache 2 license.

Android Virtual Device (AVD) Manager



This work is licensed under the Apache 2 license.

Anatomy of an Android App project

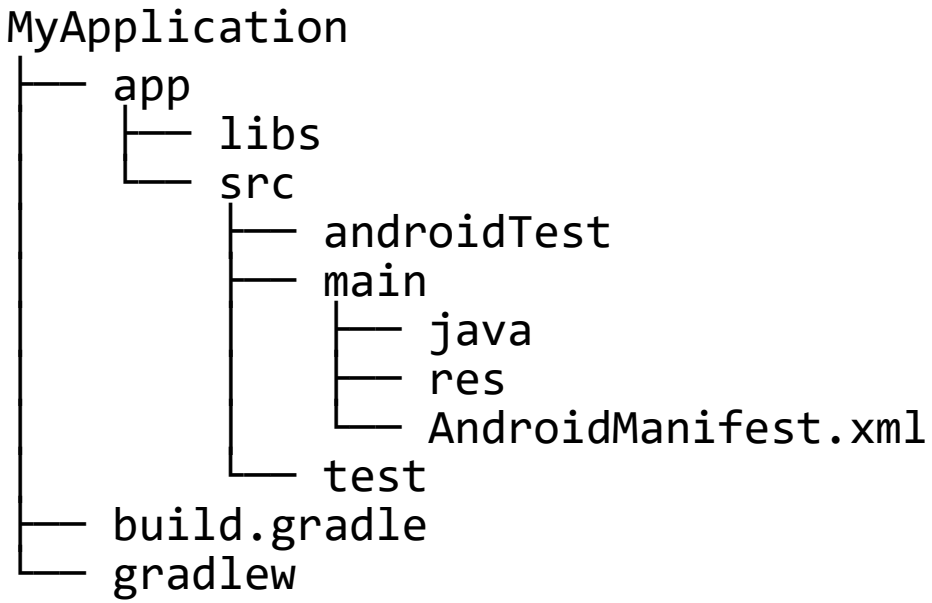
This work is licensed under the Apache 2 license.

Anatomy of a basic app project

- Activity
- Resources (layout files, images, audio files, themes, and colors)
- Gradle files

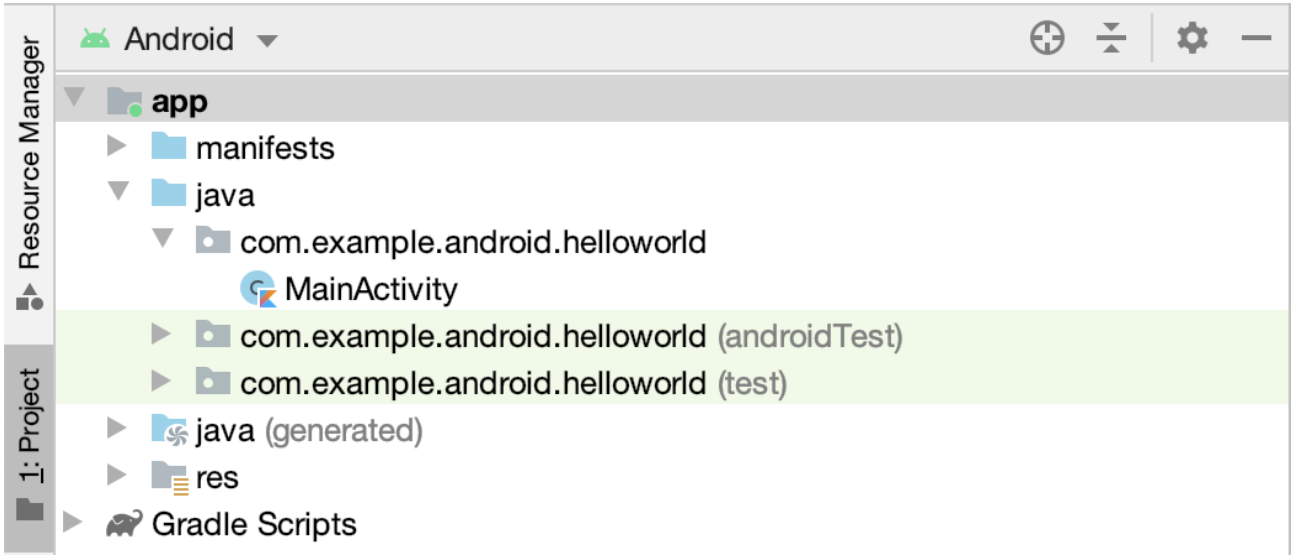
This work is licensed under the Apache 2 license.

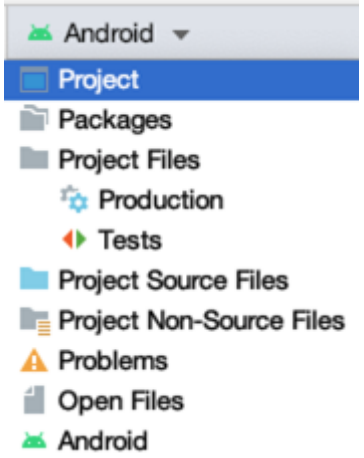
Android app project structure



This work is licensed under the Apache 2 license.

Browse files in Android Studio





This work is licensed under the Apache 2 license.

Layouts and resources in Android

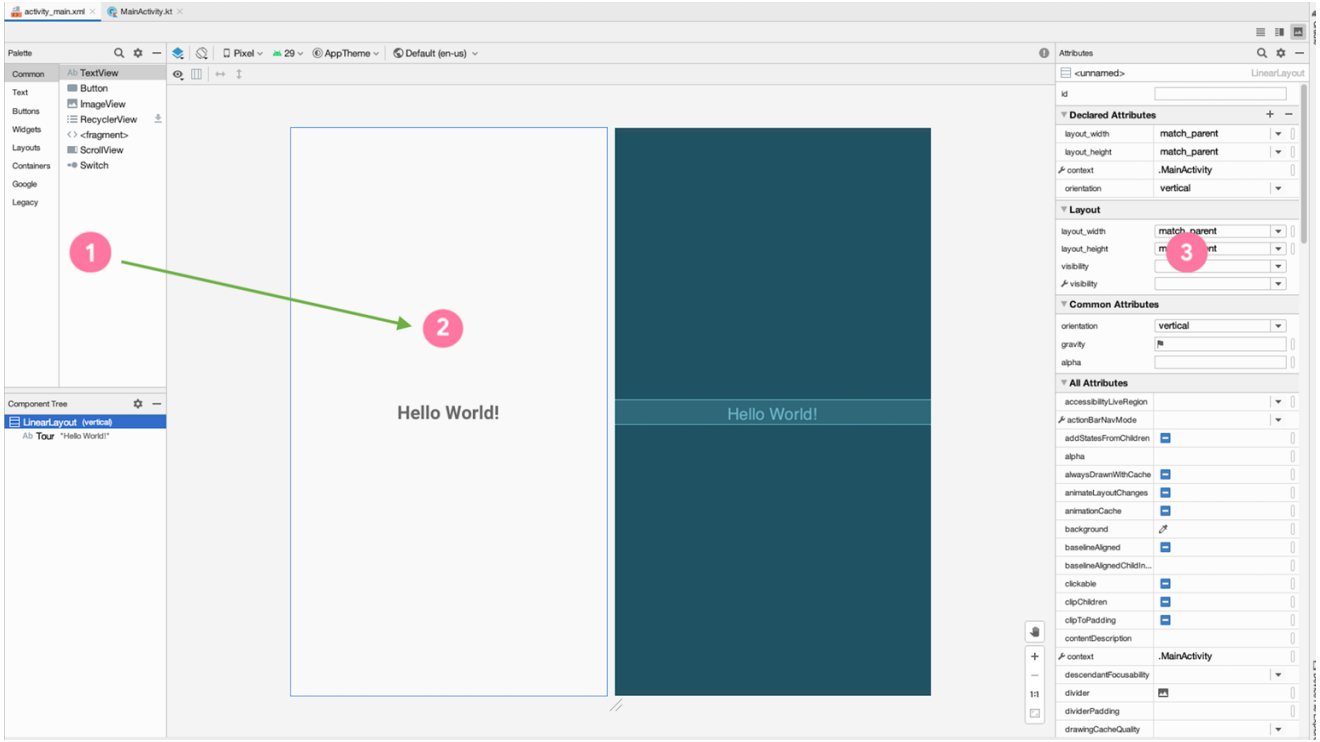
This work is licensed under the Apache 2 license.

Views

- Views are the user interface building blocks in Android
  - Bounded by a rectangular area on the screen
  - Responsible for drawing and event handling
  - Examples: TextView, ImageView, Button
- Can be grouped to form more complex user interfaces

This work is licensed under the Apache 2 license.

Layout Editor



This work is licensed under the Apache 2 license.

XML Layouts

You can also edit your layout in XML.

- Android uses XML to specify the layout of user interfaces (including View attributes)

- Each View in XML corresponds to a class in Kotlin that controls how that View functions

This work is licensed under the Apache 2 license.

XML for a TextView

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!"/>
```



This work is licensed under the Apache 2 license.

Size of a View

- wrap_content

```
android:layout_width="wrap_content"
```

- match_parent

```
android:layout_width="match_parent"
```

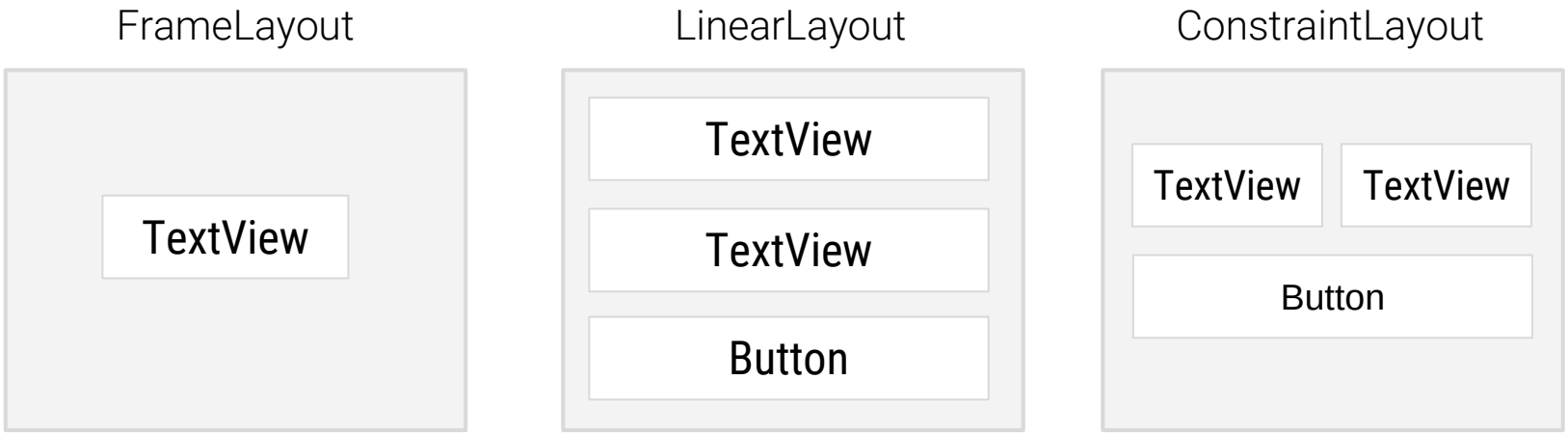
- Fixed value (use dp units)

```
android:layout_width="48dp"
```


This work is licensed under the Apache 2 license.

ViewGroups

A `ViewGroup` is a container that determines how views are displayed.



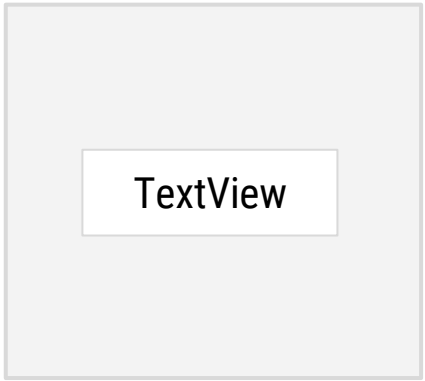
The ViewGroup is the parent and the views inside it are its children.

This work is licensed under the Apache 2 license.

FrameLayout example

A `FrameLayout` generally holds a single child `View`.

```
<FrameLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:text="Hello World!"/>
</FrameLayout>
```

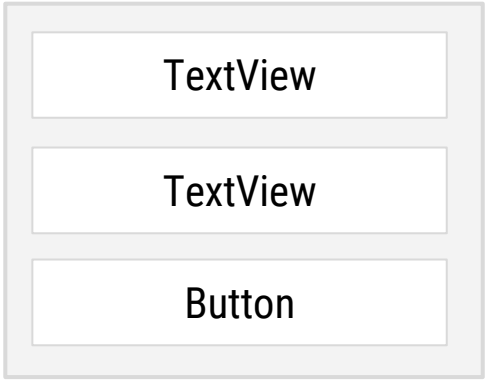


This work is licensed under the Apache 2 license.

LinearLayout example

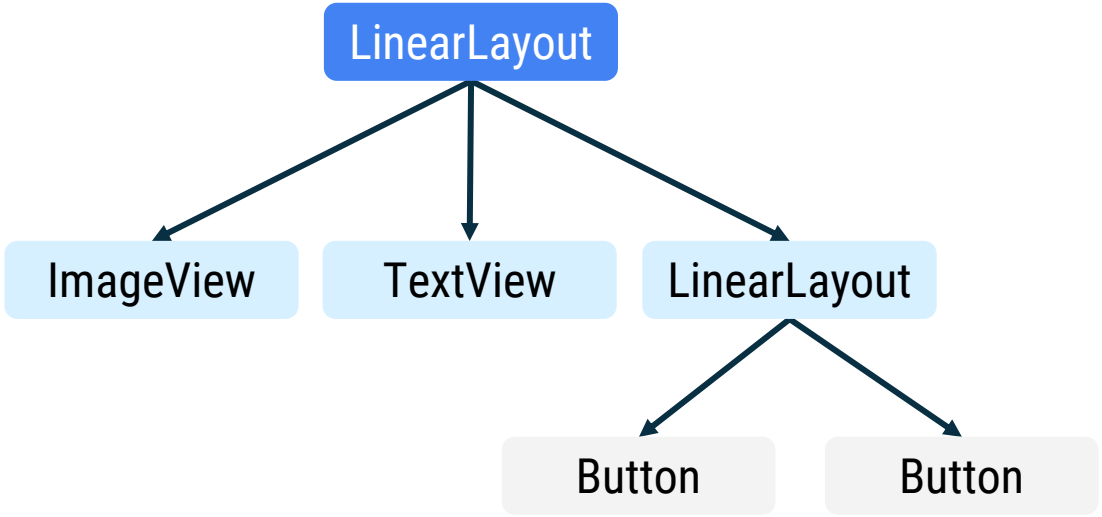
- Aligns child views in a row or column
- Set `android:orientation` to `horizontal` or `vertical`

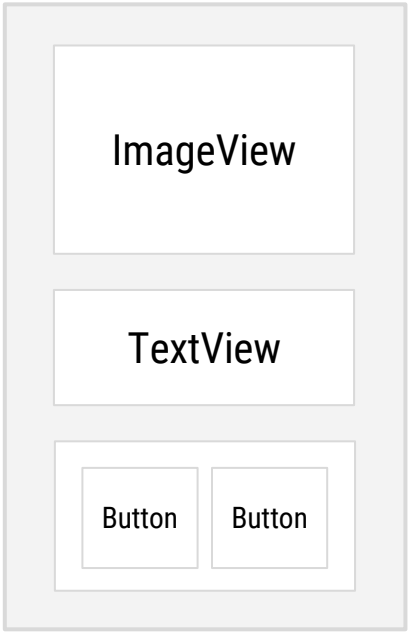
```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    <TextView ... />
    <TextView ... />
    <Button ... />
</LinearLayout>
```



This work is licensed under the Apache 2 license.

View hierarchy





This work is licensed under the Apache 2 license.

App resources

Static content or additional files that your code uses

- Layout files
- Images
- Audio files
- User interface strings
- App icon

This work is licensed under the Apache 2 license.

Common resource directories

Add resources to your app by including them in the appropriate resource directory under the parent `res` folder.

```
main
├── java
└── res
    ├── drawable ảnh / icon
    ├── layout layout
    ├── mipmap
    └── values màu sắc/string
```


This work is licensed under the Apache 2 license.

Resource IDs

- Each resource has a resource ID to access it.
- When naming resources, the convention is to use all lowercase with underscores (for example, `activity_main.xml`).
- Android autogenerates a class file named `R.java` with references to all resources in the app.
- Individual items are referenced with:
  `R.<resource_type>.<resource_name>`

Examples:
```
R.drawable.ic_launcher (res/drawable/ic_launcher.xml)
R.layout.activity_main (res/layout/activity_main.xml)
```


This work is licensed under the Apache 2 license.

Resource IDs for views

Individual views can also have resource IDs.

Add the `android:id` attribute to the View in XML. Use `@+id/name` syntax.

```
<TextView
     android:id="@+id/helloTextView"
     android:layout_width="wrap_content"
     android:layout_height="wrap_content"
     android:text="Hello World!"/>
```

Within your app, you can now refer to this specific TextView using:

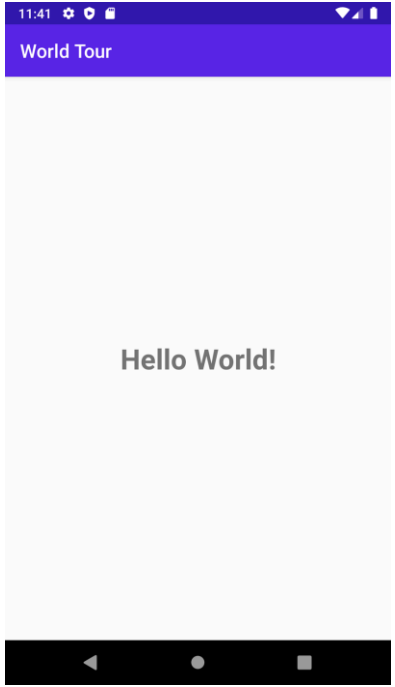
`R.id.helloTextView`

This work is licensed under the Apache 2 license.

Activities

This work is licensed under the Apache 2 license.

What's an Activity?



- An Activity is a means for the user to accomplish one main goal.
- An Android app is composed of one or more activities.

app có thể có 1 or nhiều act

This work is licensed under the Apache 2 license.

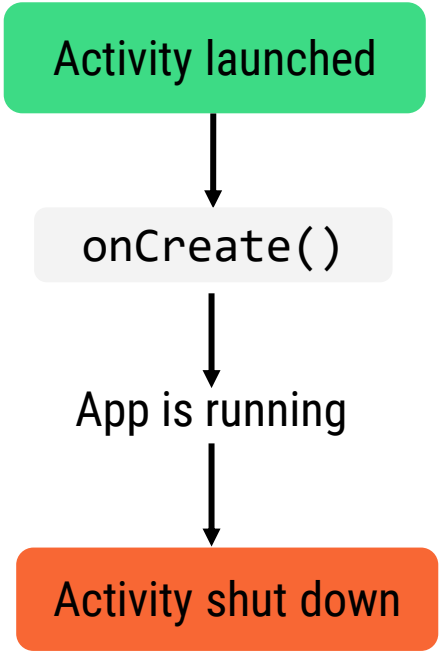
MainActivity.kt

```
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```


This work is licensed under the Apache 2 license.

How an Activity runs



This work is licensed under the Apache 2 license.

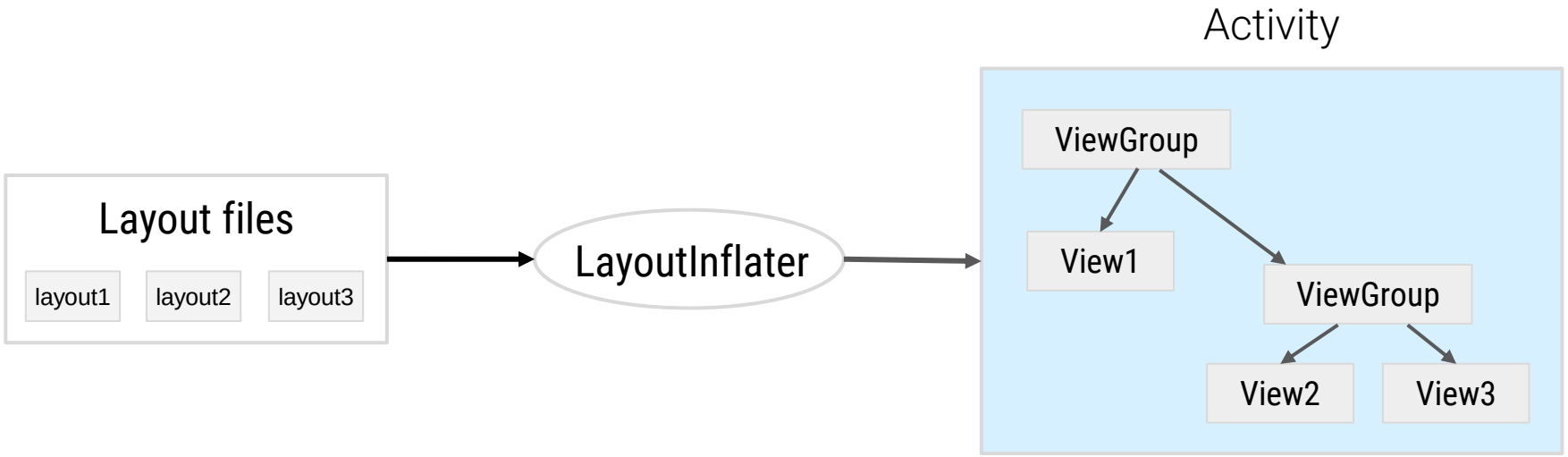
Implement the onCreate() callback

Called when the system creates your Activity

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
}
```


This work is licensed under the Apache 2 license.

Layout inflation



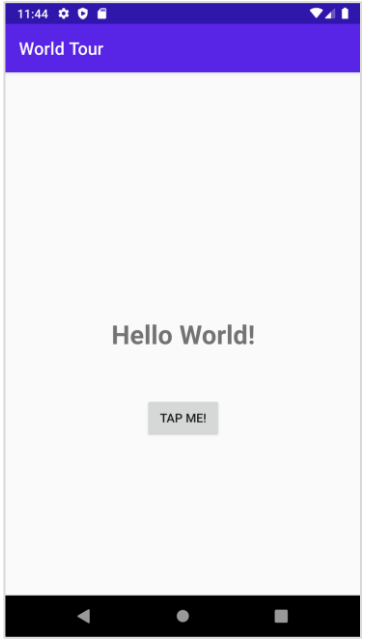
This work is licensed under the Apache 2 license.

Make an app interactive

This work is licensed under the Apache 2 license.

Define app behavior in Activity

Modify the Activity so the app responds to user input, such as a button tap.



This work is licensed under the Apache 2 license.

Modify a View dynamically

Within `MainActivity.kt`:

Get a reference to the View in the view hierarchy:

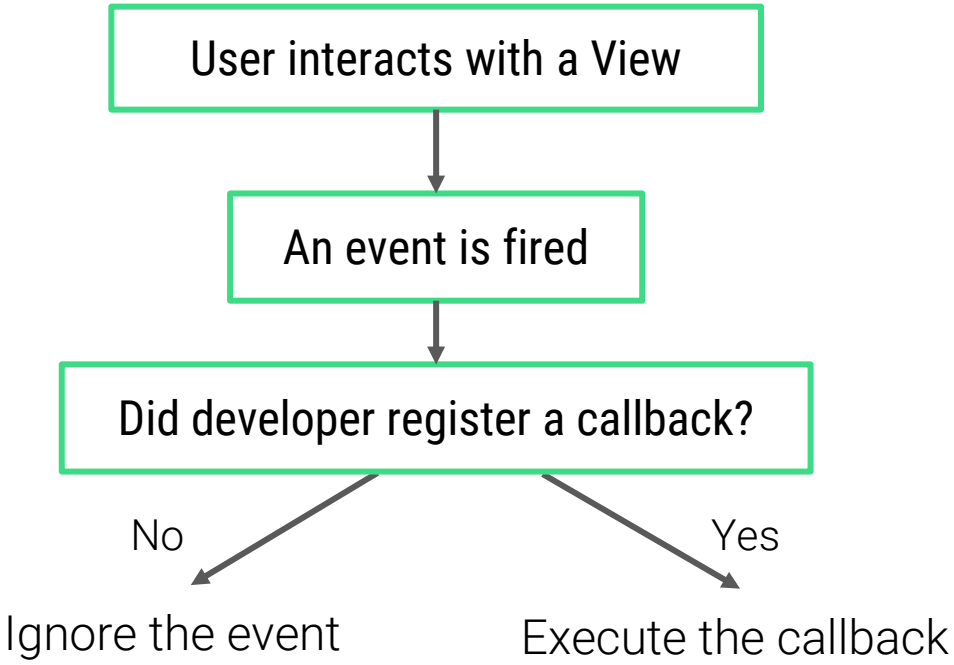
```
val resultTextView: TextView = findViewById(R.id.textView)
```

Change properties or call methods on the View instance:

```
resultTextView.text = "Goodbye!"
```


This work is licensed under the Apache 2 license.

Set up listeners for specific events



This work is licensed under the Apache 2 license.

View.OnClickListener

```
class MainActivity : AppCompatActivity(), View.OnClickListener {

    override fun onCreate(savedInstanceState: Bundle?) {
        ...
        val button: Button = findViewById(R.id.button)
        button.setOnClickListener(this)
    }

    override fun onClick(v: View?) {
        TODO("not implemented")
    }
}
```


This work is licensed under the Apache 2 license.

SAM (single abstract method)

Converts a function into an implementation of an interface

Format: `InterfaceName { lambda body }`

```
val runnable = Runnable { println("Hi there") }
```

is equivalent to

```
val runnable = (object: Runnable {
    override fun run() {
        println("Hi there")
    }
})
```


This work is licensed under the Apache 2 license.

View.OnClickListener as a SAM

A more concise way to declare a click listener

```
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        ...

        val button: Button = findViewById(R.id.button)
        button.setOnClickListener({ view -> /* do something*/ })
    }
}
```


This work is licensed under the Apache 2 license.

Late initialization

```
class Student(val id: String) {

    lateinit var records: HashSet<Any>

    init {
        // retrieve records given an id
    }
}
```


This work is licensed under the Apache 2 license.

Lateinit example in Activity

```
class MainActivity : AppCompatActivity() {

    lateinit var result: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        ...
        result = findViewById(R.id.result_text_view)
    }
}
```


This work is licensed under the Apache 2 license.

Gradle: Building an Android app

This work is licensed under the Apache 2 license.

What is Gradle?

- Builds automation system
- Manages the build cycle via a series of tasks (for example, compiles Kotlin sources, runs tests, installs app to device)
- Determines the proper order of tasks to run
- Manages dependencies between projects and third-party libraries

This work is licensed under the Apache 2 license.

Gradle build file

- Declare plugins
- Define Android properties
- Handle dependencies
- Connect to repositories

This work is licensed under the Apache 2 license.

Plugins

Provide libraries and infrastructure needed by your app

```
apply plugin: 'com.android.application'

apply plugin: 'kotlin-android'

apply plugin: 'kotlin-android-extensions'
```


This work is licensed under the Apache 2 license.

Android configuration

```
android {
    compileSdkVersion 30
    buildToolsVersion "30.0.2"

    defaultConfig {
        applicationId "com.example.sample"
        minSdkVersion 19
        targetSdkVersion 30
    }
}
```


This work is licensed under the Apache 2 license.

Dependencies

```
dependencies {
    implementation "org.jetbrains.kotlin:kotlin-stdlib-
jdk7:$kotlin_version"
    implementation 'androidx.core:core-ktx:1.3.2'
    implementation 'androidx.appcompat:appcompat:1.2.0'
    implementation 'com.google.android.material:material:1.2.1'
    ...
}
```


This work is licensed under the Apache 2 license.

Repositories

```
repositories {
    google()
    jcenter()
    maven {
        url "https://maven.example.com"
    }
}
```


This work is licensed under the Apache 2 license.

Common Gradle tasks

- Clean
- Tasks
- InstallDebug

This work is licensed under the Apache 2 license.

Accessibility

This work is licensed under the Apache 2 license.

Accessibility

- Refers to improving the design and functionality of your app to make it easier for more people, including those with disabilities, to use
- Making your app more accessible leads to an overall better user experience and benefits all your users

This work is licensed under the Apache 2 license.

Make apps more accessible

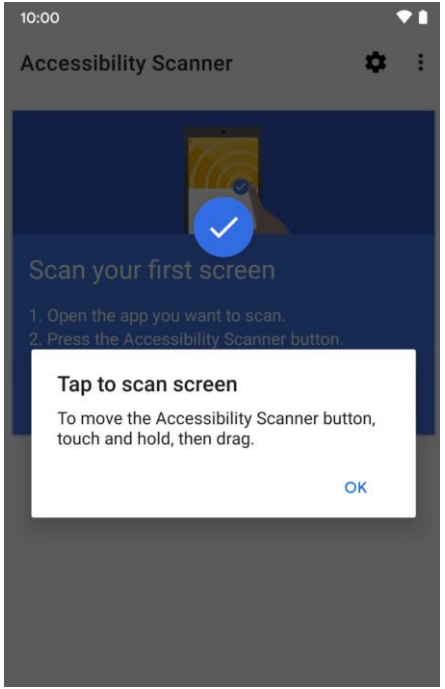
- Increase text visibility with foreground and background color contrast ratio:
  - At least 4.5:1 for small text against the background
  - At least 3.0:1 for large text against the background
- Use large, simple controls
  - Touch target size should be at least 48dp x 48dp
- Describe each UI element
  - Set content description on images and controls

This work is licensed under the Apache 2 license.

Accessibility Scanner

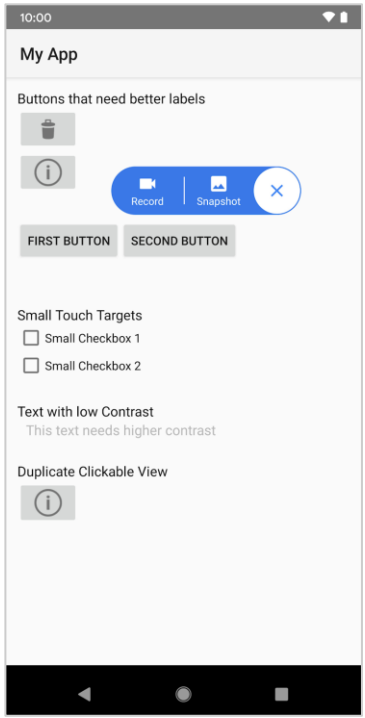
Tool that scans your screen and suggests improvements to make your app more accessible, based on:

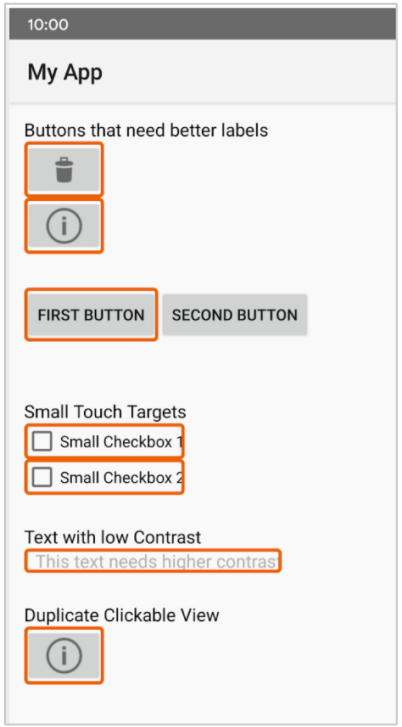
- Content labels
- Touch target sizes
- Clickable views
- Text and image contrast

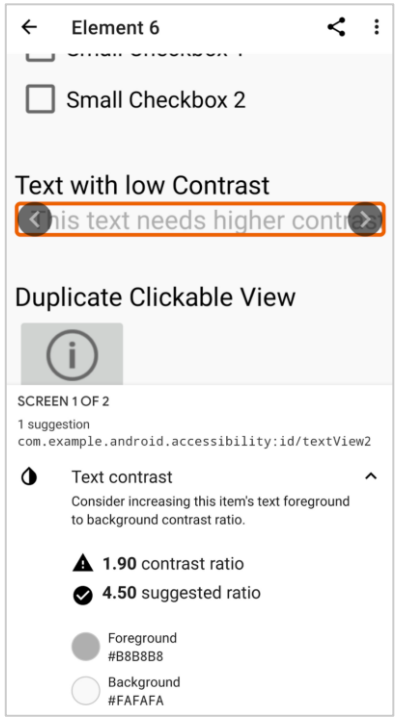


This work is licensed under the Apache 2 license.

Accessibility Scanner example







This work is licensed under the Apache 2 license.

Add content labels

- Set `contentDescription` attribute → read aloud by screen reader

```
<ImageView
    ...
    android:contentDescription="@string/stop_sign" />
```

- Text in TextView already provided to accessibility services, no additional label needed

This work is licensed under the Apache 2 license.

No content label needed

- For graphical elements that are purely for decorative purposes, you can set

```
android:importantForAccessibility="no"
```

- Removing unnecessary announcements is better for the user

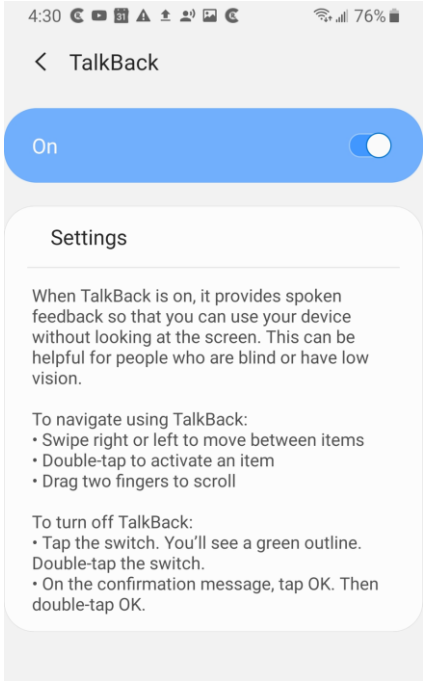
This work is licensed under the Apache 2 license.

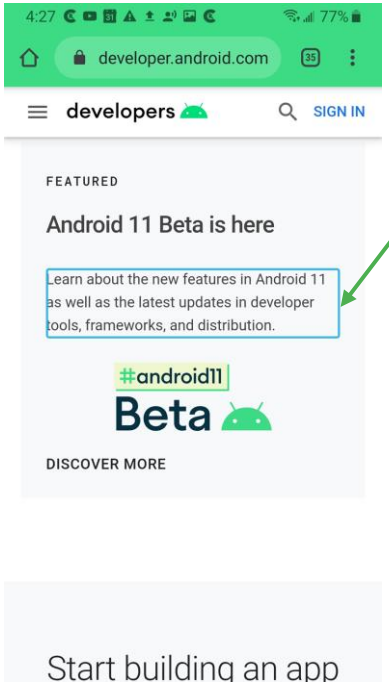
TalkBack

- Google screen reader included on Android devices
- Provides spoken feedback so you don't have to look at the screen to use your device
- Lets you navigate the device using gestures
- Includes braille keyboard for Unified English Braille

This work is licensed under the Apache 2 license.

TalkBack example





Reads text aloud as user navigates the screen

This work is licensed under the Apache 2 license.

Switch access

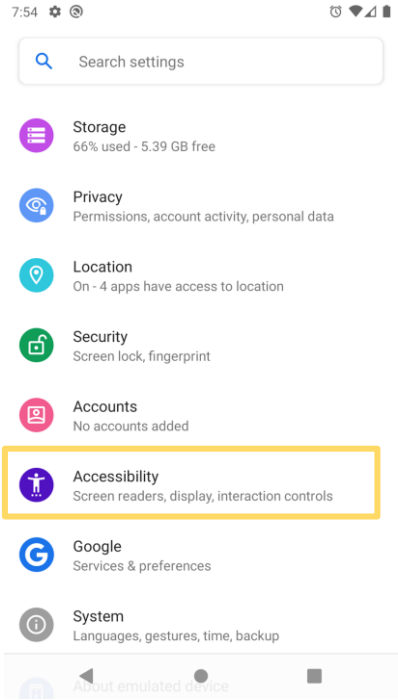
- Allows for controlling the device using one or more switches instead of the touchscreen
- Scans your app UI and highlights each item until you make a selection
- Use with external switch, external keyboard, or buttons on the Android device (e.g., volume buttons)

This work is licensed under the Apache 2 license.

Android Accessibility Suite

Collection of accessibility apps that help you use your Android device eyes-free, or with a switch device. It includes:

- Talkback screen reader
- Switch Access
- Accessibility Menu
- Select to Speak



This work is licensed under the Apache 2 license.

Accessibility Resources

- Build more accessible apps
- Principles for improving app accessibility
- Basic Android Accessibility codelab
- Material Design best practices on accessibility

This work is licensed under the Apache 2 license.

Summary

This work is licensed under the Apache 2 license.

Summary

In Lesson 4, you learned how to:

- Use Views and `ViewGroups` to build the user interface of your app
- Access resources in your app from `R.<resource_type>.<resource_name>`
- Define app behavior in the Activity (for example, register `OnClickListener`)
- Use Gradle as the build system to build your app
- Follow best practices to make your apps more accessible

This work is licensed under the Apache 2 license.

Learn more

- Layouts
- LinearLayout
- Input events overview
- View
- ViewGroup

This work is licensed under the Apache 2 license.

Pathway

Practice what you've learned by completing the pathway:

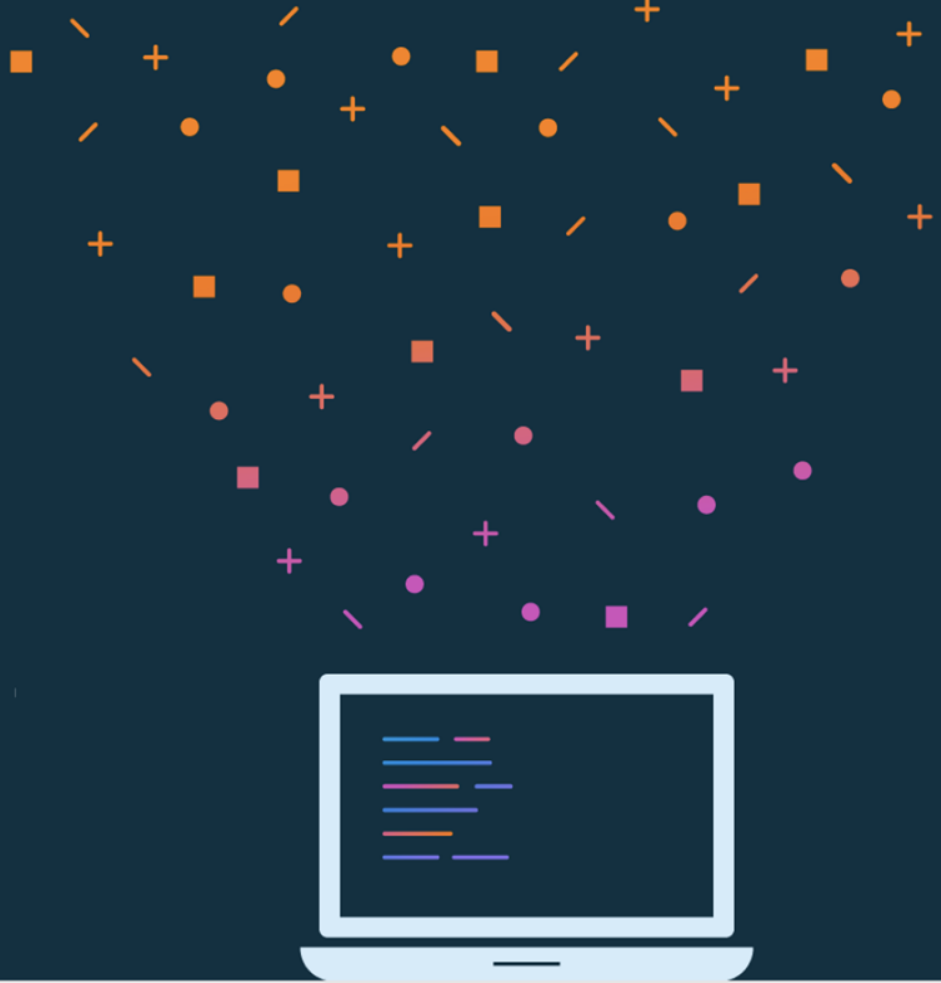
Lesson 4: Build your first Android app



This work is licensed under the Apache 2 license.



Lesson 5: Layouts



This work is licensed under the Apache 2 license.

About this lesson

Lesson 5: Layouts

- Layouts in Android
- ConstraintLayout
- Additional topics for ConstraintLayout
- Data binding
- Displaying lists with RecyclerView
- Summary

This work is licensed under the Apache 2 license.

Layouts in Android

This work is licensed under the Apache 2 license.

Android devices

- Android devices come in many different form factors.
- More and more pixels per inch are being packed into device screens.
- Developers need the ability to specify layout dimensions that are consistent across devices.

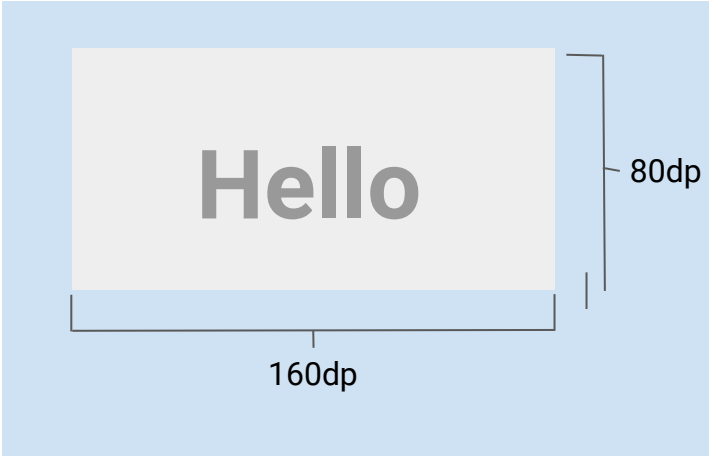


This work is licensed under the Apache 2 license.

Density-independent pixels (dp)

Use dp when specifying sizes in your layout, such as the width or height of views.

- Density-independent pixels (dp) take screen density into account.
- Android views are measured in density-independent pixels.
- dp = $\frac{\text{(width in pixels * 160)}}{\text{screen density}}$



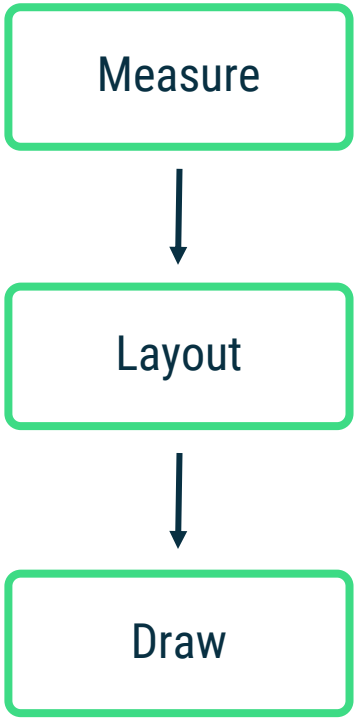
This work is licensed under the Apache 2 license.

Screen-density buckets

Density qualifier	Description	DPI estimate
ldpi (mostly unused)	Low density	~120dpi
mdpi (baseline density)	Medium density	~160dpi
hdpi	High density	~240dpi
xhdpi	Extra-high density	~320dpi
xxhdpi	Extra-extra-high density	~480dpi
xxxhdpi	Extra-extra-extra-high density	~640dpi

This work is licensed under the Apache 2 license.

Android View rendering cycle



This work is licensed under the Apache 2 license.

Drawing region

What we see:

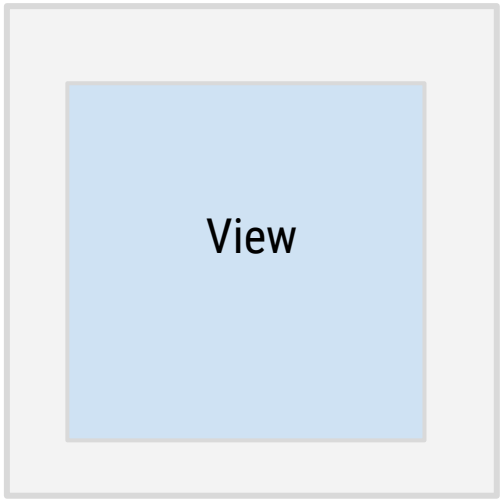
How it's drawn:



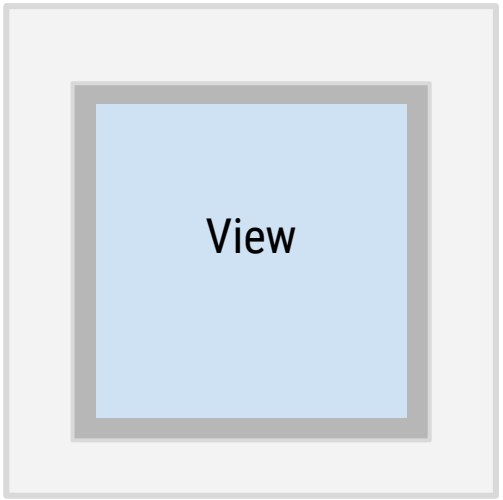
This work is licensed under the Apache 2 license.

View margins and padding

View with margin



View with margin and padding



This work is licensed under the Apache 2 license.

ConstraintLayout

This work is licensed under the Apache 2 license.

Deeply nested layouts are costly

- Deeply nested ViewGroups require more computation
- Views may be measured multiple times
- Can cause UI slowdown and lack of responsiveness

Use ConstraintLayout to avoid some of these issues!

This work is licensed under the Apache 2 license.

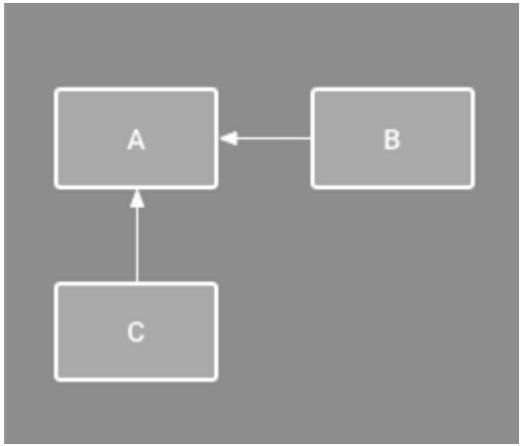
What is ConstraintLayout?

- Recommended default layout for Android
- Solves costly issue of too many nested layouts, while allowing complex behavior
- Position and size views within it using a set of constraints

This work is licensed under the Apache 2 license.

What is a constraint?

A restriction or limitation on the properties of a View that the layout attempts to respect



This work is licensed under the Apache 2 license.

Relative positioning constraints

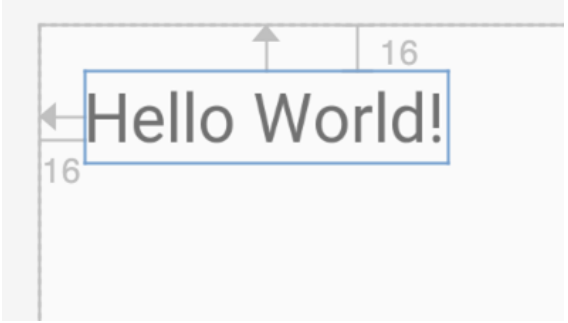
Can set up a constraint relative to the parent container

Format:

`layout_constraint<`**`SourceConstraint`**`>_to<`**`TargetConstraint`**`>Of`

Example attributes on a TextView:

```
app:layout_constraintTop_toTopOf="parent"
app:layout_constraintLeft_toLeftOf="parent"
```



This work is licensed under the Apache 2 license.

Relative positioning constraints



This work is licensed under the Apache 2 license.

Relative positioning constraints



This work is licensed under the Apache 2 license.

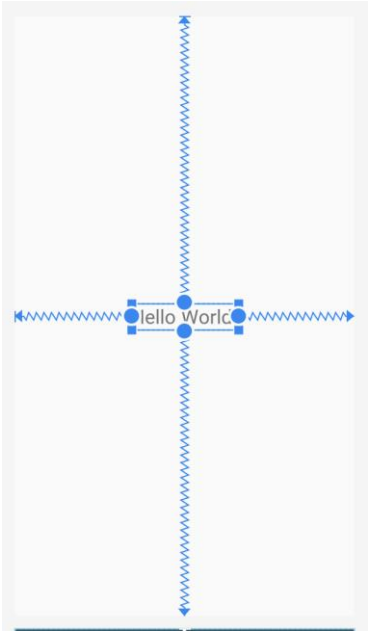
Simple ConstraintLayout example

```
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        ...

app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

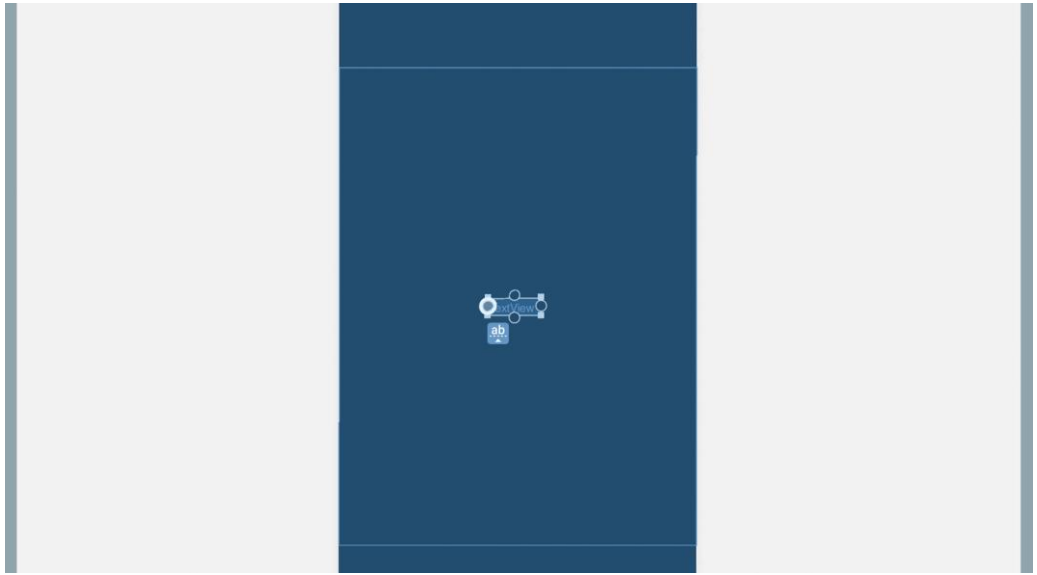
</androidx.constraintlayout.widget.ConstraintLayout>
```



This work is licensed under the Apache 2 license.

Layout Editor in Android Studio

You can click and drag to add constraints to a View.



This work is licensed under the Apache 2 license.

Constraint Widget in Layout Editor



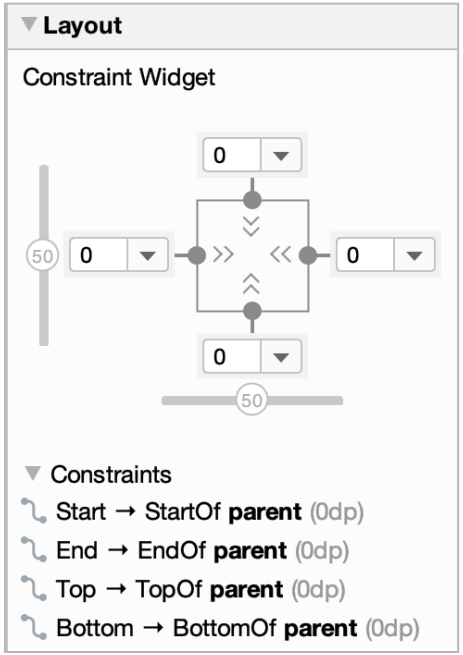
Fixed



Wrap content

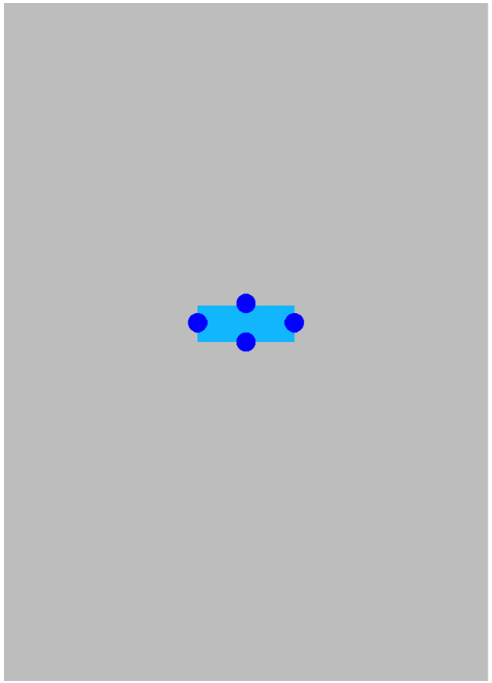


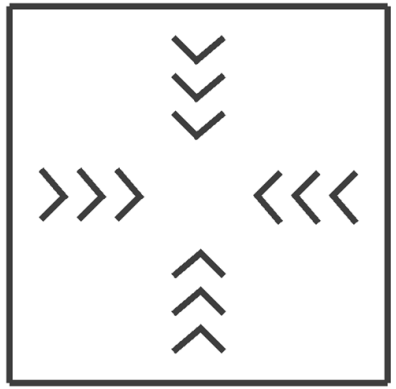
Match constraints



This work is licensed under the Apache 2 license.

Wrap content for width and height

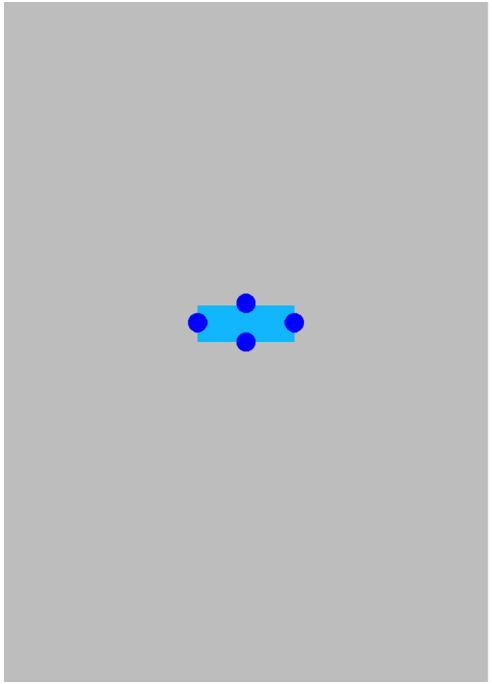


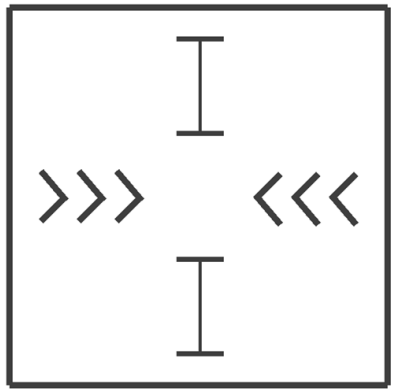


```
layout_width    wrap_content
layout_height   wrap_content
```


This work is licensed under the Apache 2 license.

Wrap content for width, fixed height

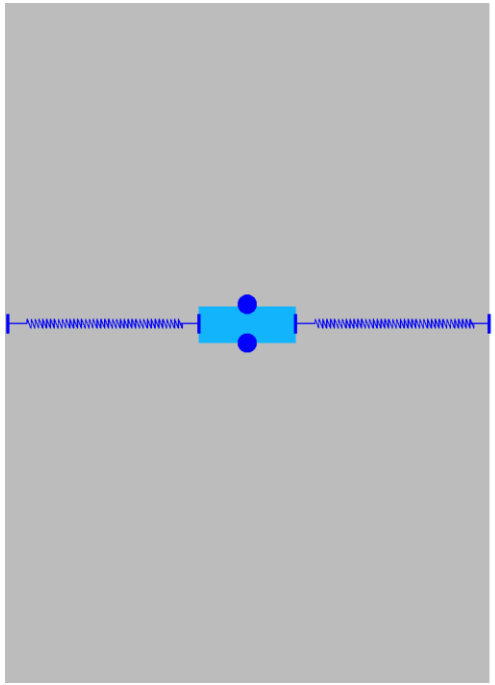


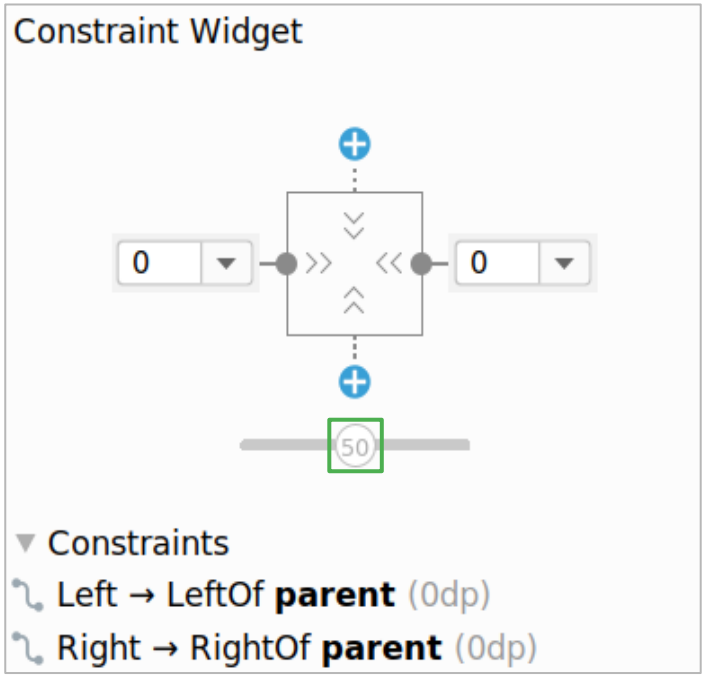


```
layout_width    wrap_content
layout_height   48dp
```


This work is licensed under the Apache 2 license.

Center a view horizontally

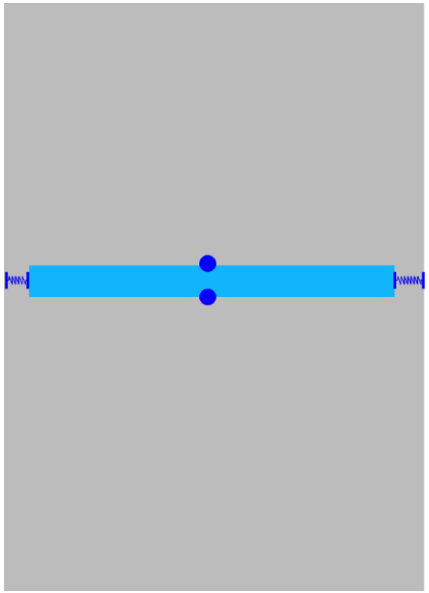


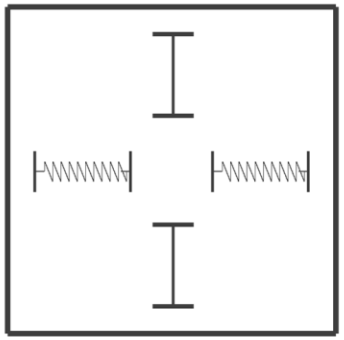


This work is licensed under the Apache 2 license.

Use match_constraint

Can’t use `match_parent` on a child view, use `match_constraint` instead





```
layout_width    0dp(match_constraint)
layout_height   48dp
```


This work is licensed under the Apache 2 license.

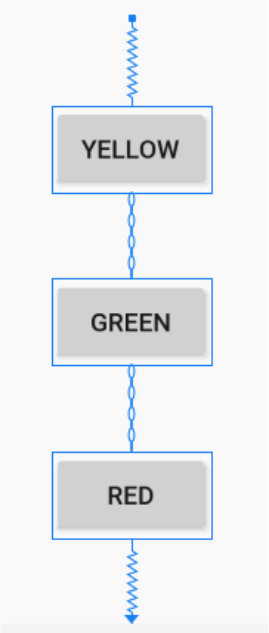
Chains

- Let you position views in relation to each other
- Can be linked horizontally or vertically
- Provide much of LinearLayout functionality

This work is licensed under the Apache 2 license.

Create a Chain in Layout Editor

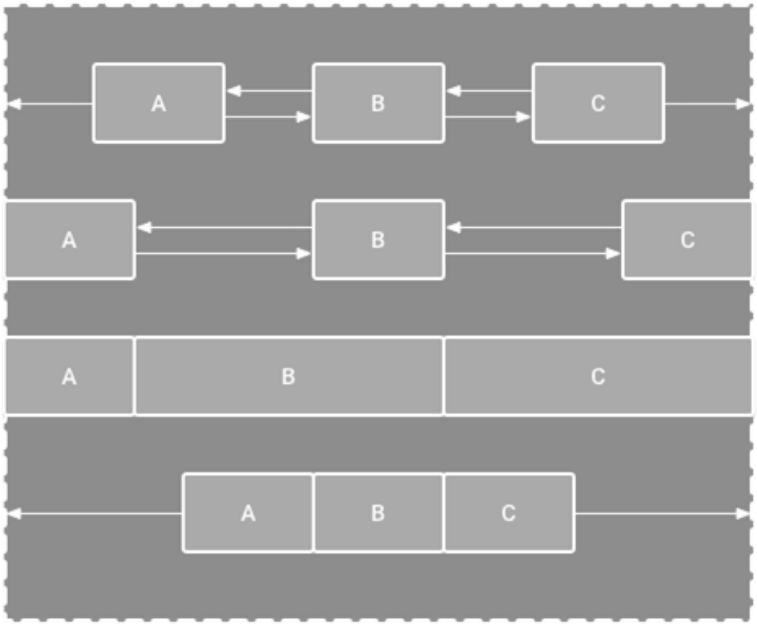
1. Select the objects you want to be in the chain.
2. Right-click and select **Chains.**
3. Create a horizontal or vertical chain.



This work is licensed under the Apache 2 license.

Chain styles

Adjust space between views with these different chain styles.



Spread Chain

Spread Inside Chain

Weighted Chain

Packed Chain

This work is licensed under the Apache 2 license.

Additional topics for ConstraintLayout

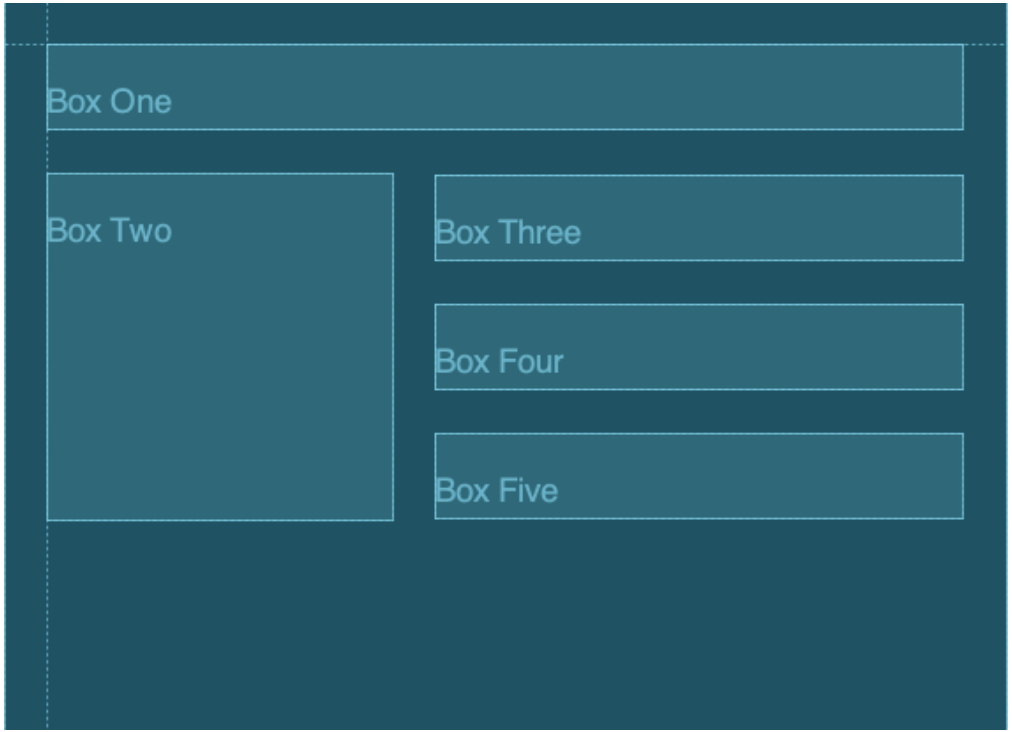
This work is licensed under the Apache 2 license.

Guidelines

- Let you position multiple views relative to a single guide
- Can be vertical or horizontal
- Allow for greater collaboration with design/UX teams
- Aren't drawn on the device

This work is licensed under the Apache 2 license.

Guidelines in Android Studio



This work is licensed under the Apache 2 license.

Example Guideline

```
<ConstraintLayout>
    <androidx.constraintlayout.widget.Guideline
        android:id="@+id/start_guideline"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        app:layout_constraintGuide_begin="16dp" />
    <TextView ...
        app:layout_constraintStart_toEndOf="@id/start_guideline" />
</ConstraintLayout>
```


This work is licensed under the Apache 2 license.

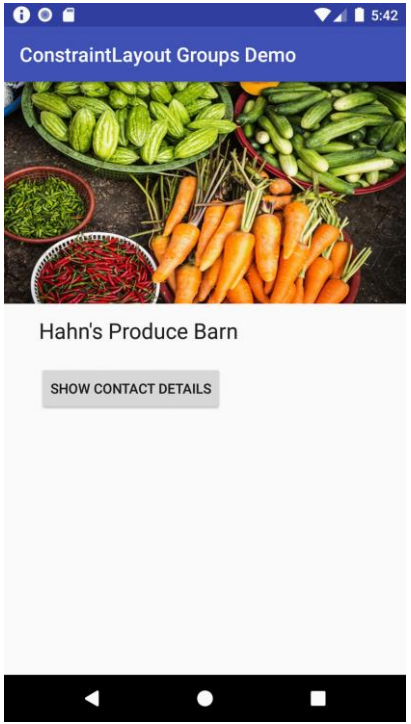
Creating Guidelines

- `layout_constraintGuide_begin`
- `layout_constraintGuide_end`
- `layout_constraintGuide_percent`

This work is licensed under the Apache 2 license.

Groups

- Control the visibility of a set of widgets
- Group visibility can be toggled in code



This work is licensed under the Apache 2 license.

Example group

```
<androidx.constraintlayout.widget.Group
    android:id="@+id/group"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:constraint_referenced_ids="locationLabel,locationDetails"/>
```


This work is licensed under the Apache 2 license.

Groups app code

```
override fun onClick(v: View?) {
    if (group.visibility == View.GONE) {
        group.visibility = View.VISIBLE
        button.setText(R.string.hide_details)
    } else {
        group.visibility = View.GONE
        button.setText(R.string.show_details)
    }
}
```


This work is licensed under the Apache 2 license.

Data binding

This work is licensed under the Apache 2 license.

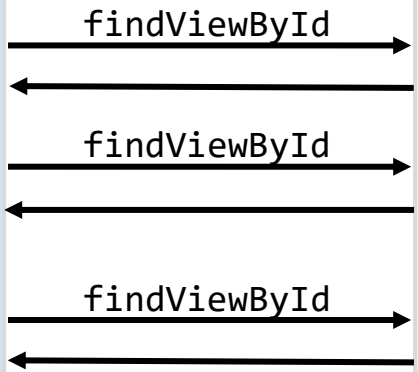
Current approach: findViewById()

Traverses the `View` hierarchy each time

MainActivity.kt

```
val name = findViewById(...)
val age = findViewById(...)
val loc = findViewById(...)

name.text = …
age.text = …
loc.text = …
```



activity_main.xml

```
<ConstraintLayout … >
  <TextView
      android:id="@+id/name"/>
  <TextView
      android:id="@+id/age"/>
  <TextView
      android:id="@+id/loc"/>
</ConstraintLayout>
```


This work is licensed under the Apache 2 license.

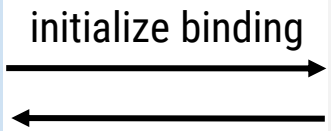
Use data binding instead

Bind UI components in your layouts to data sources in your app.

MainActivity.kt

```
Val binding:ActivityMainBinding

binding.name.text = …
binding.age.text = …
binding.loc.text = …
```



activity_main.xml

```
<layout>
   <ConstraintLayout … >
       <TextView
          android:id="@+id/name"/>
       <TextView
          android:id="@+id/age"/>
       <TextView
          android:id="@+id/loc"/>
   </ConstraintLayout>
</layout>
```


This work is licensed under the Apache 2 license.

Modify build.gradle file

```
android {
    ...
    buildFeatures {
        dataBinding true
    }
}
```


This work is licensed under the Apache 2 license.

Add layout tag

```
<layout>
    <androidx.constraintlayout.widget.ConstraintLayout>
        <TextView ... android:id="@+id/username" />
        <EditText ... android:id="@+id/password" />
    </androidx.constraintlayout.widget.ConstraintLayout>
</layout>
```


This work is licensed under the Apache 2 license.

Layout inflation with data binding

Replace this

```
setContentView(R.layout.activity_main)
```

with this

```
val binding: ActivityMainBinding = DataBindingUtil.setContentView(
    this, R.layout.activity_main)

binding.username = "Melissa"
```


This work is licensed under the Apache 2 license.

Data binding layout variables

```
<layout>
    <data>
        <variable name="name" type="String"/>
    </data>
    <androidx.constraintlayout.widget.ConstraintLayout>
        <TextView
            android:id="@+id/textView"
            android:text="@{name}" />
    </androidx.constraintlayout.widget.ConstraintLayout>
</layout>
```

In `MainActivity.kt`:

```
binding.name = "John"
```


This work is licensed under the Apache 2 license.

Data binding layout expressions

```
<layout>
    <data>
        <variable name="name" type="String"/>
    </data>

    <androidx.constraintlayout.widget.ConstraintLayout>
        <TextView
            android:id="@+id/textView"
            android:text="@{name.toUpperCase()}" />
    </androidx.constraintlayout.widget.ConstraintLayout>
</layout>
```

3/7

This work is licensed under the Apache 2 license.

Displaying lists with RecyclerView

This work is licensed under the Apache 2 license.

RecyclerView

- Widget for displaying lists of data
- "Recycles" (reuses) item views to make scrolling more performant
- Can specify a list item layout for each item in the dataset
- Supports animations and transitions

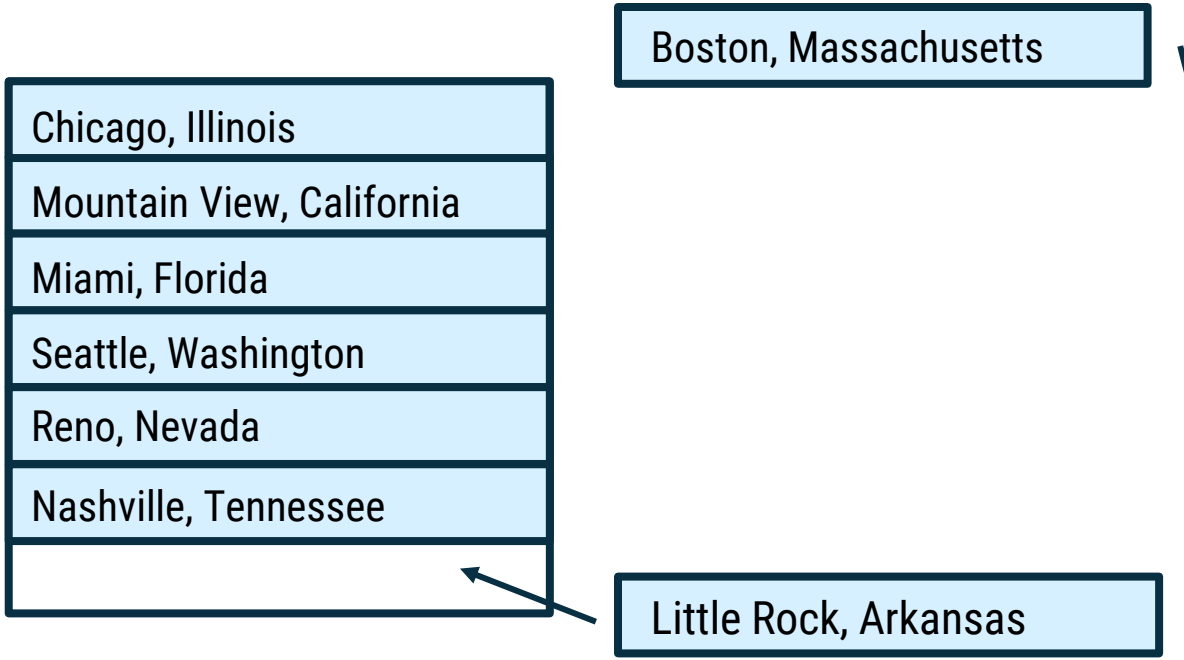
This work is licensed under the Apache 2 license.

RecyclerView.Adapter

- Supplies data and layouts that the RecyclerView displays
- A custom Adapter extends from `RecyclerView.Adapter` and overrides these three functions:
  - `getItemCount`
  - `onCreateViewHolder`
  - `onBindViewHolder`

This work is licensed under the Apache 2 license.

View recycling in RecyclerView



If item is scrolled offscreen, it isn't destroyed. Item is put in a pool to be recycled.

onBindViewHolder binds the view with the new values, and then the view gets reinserted in the list.

This work is licensed under the Apache 2 license.

Add RecyclerView to your layout

```
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/rv"
    android:scrollbars="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>
```


This work is licensed under the Apache 2 license.

Create a list item layout

res/layout/item_view.xml

```
<FrameLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content">
    <TextView
        android:id="@+id/number"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
</FrameLayout>
```


This work is licensed under the Apache 2 license.

Create a list adapter

```
class MyAdapter(val data: List<Int>) : RecyclerView.Adapter<MyAdapter.MyViewHolder>()
{
    class MyViewHolder(val row: View) : RecyclerView.ViewHolder(row) {
        val textView = row.findViewById<TextView>(R.id.number)
    }

   override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MyViewHolder {
        val layout = LayoutInflater.from(parent.context).inflate(R.layout.item_view,
                     parent, false)
        return MyViewHolder(layout)
    }
    override fun onBindViewHolder(holder: MyViewHolder, position: Int) {
        holder.textView.text = data.get(position).toString()
    }
    override fun getItemCount(): Int = data.size
```


This work is licensed under the Apache 2 license.

Set the adapter on the RecyclerView

In `MainActivity.kt`:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    val rv: RecyclerView = findViewById(R.id.rv)
    rv.layoutManager = LinearLayoutManager(this)

    rv.adapter = MyAdapter(IntRange(0, 100).toList())
}
```


This work is licensed under the Apache 2 license.

Summary

This work is licensed under the Apache 2 license.

Summary

In Lesson 5, you learned how to:

- Specify lengths in dp for your layout
- Work with screen densities for different Android devices
- Render Views to the screen of your app
- Layout views within a ConstraintLayout using constraints
- Simplify getting View references from layout with data binding
- Display a list of text items using a RecyclerView and custom adapter

This work is licensed under the Apache 2 license.

Learn more

- Pixel density on Android
- Spacing
- Device metrics
- Type scale
- Build a Responsive UI with ConstraintLayout
- Data Binding Library
- Create dynamic lists with RecyclerView

This work is licensed under the Apache 2 license.

Pathway

Practice what you've learned by completing the pathway:

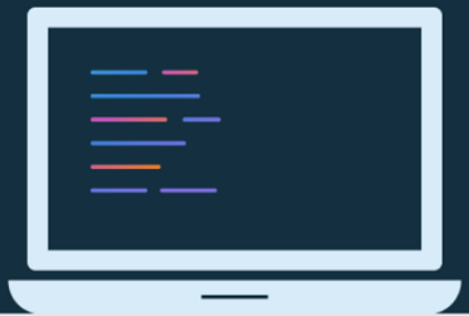
Lesson 5: Layouts



This work is licensed under the Apache 2 license.



Lesson 6: App navigation



This work is licensed under the Apache 2 license.

About this lesson

Lesson 6: App navigation

- Multiple activities and intents
- App bar, navigation drawer, and menus
- Fragments
- Navigation in an app
- More custom navigation behavior
- Navigation UI
- Summary

This work is licensed under the Apache 2 license.

Multiple activities and intents

This work is licensed under the Apache 2 license.

Multiple screens in an app

Sometimes app functionality may be separated into multiple screens.

Examples:

- View details of a single item (for example, product in a shopping app)
- Create a new item (for example, new email)
- Show settings for an app
- Access services in other apps (for example, photo gallery or browse documents)

This work is licensed under the Apache 2 license.

Intent

Requests an action from another app component, such as another Activity

- An `Intent` usually has two primary pieces of information:
  - Action to be performed (for example, `ACTION_VIEW`, `ACTION_EDIT`, `ACTION_MAIN`)
  - Data to operate on (for example, a person's record in the contacts database)
- Commonly used to specify a request to transition to another Activity

This work is licensed under the Apache 2 license.

Explicit intent

- Fulfills a request **using a specific component**
- Navigates internally to an Activity in your app
- Navigates to a specific third-party app or another app you've written

This work is licensed under the Apache 2 license.

Explicit intent examples

Navigate between activities in your app:

```
fun viewNoteDetail() {
    val intent = Intent(this, NoteDetailActivity::class.java)
    intent.putExtra(NOTE_ID, note.id)
    startActivity(intent)
}
```

Navigate to a specific external app:

```
fun openExternalApp() {
    val intent = Intent("com.example.workapp.FILE_OPEN")
    if (intent.resolveActivity(packageManager) != null) {
        startActivity(intent)
    }
}
```


This work is licensed under the Apache 2 license.

Implicit intent

- Provides generic action the app can perform
- Resolved using mapping of the data type and action to known components
- Allows any app that matches the criteria to handle the request

This work is licensed under the Apache 2 license.

Implicit intent example

```
fun sendEmail() {
    val intent = Intent(Intent.ACTION_SEND)
    intent.type = "text/plain"
    intent.putExtra(Intent.EXTRA_EMAIL, emailAddresses)
    intent.putExtra(Intent.EXTRA_TEXT, "How are you?")

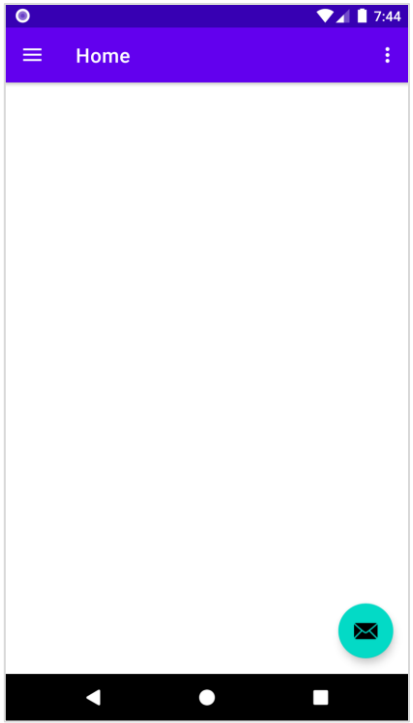
    if (intent.resolveActivity(packageManager) != null) {
        startActivity(intent)
    }
}
```

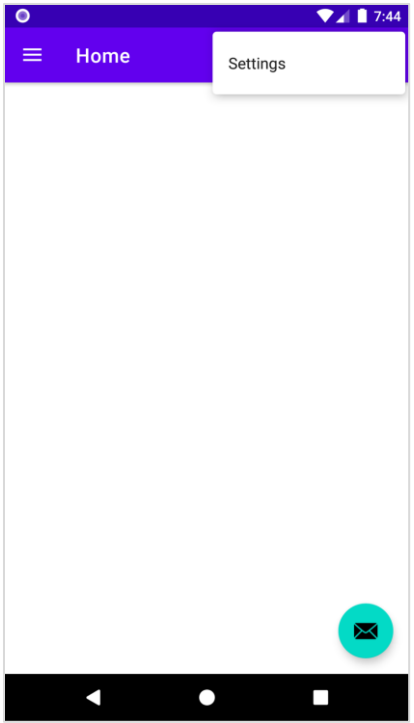

This work is licensed under the Apache 2 license.

App bar, navigation drawer, and menus

This work is licensed under the Apache 2 license.

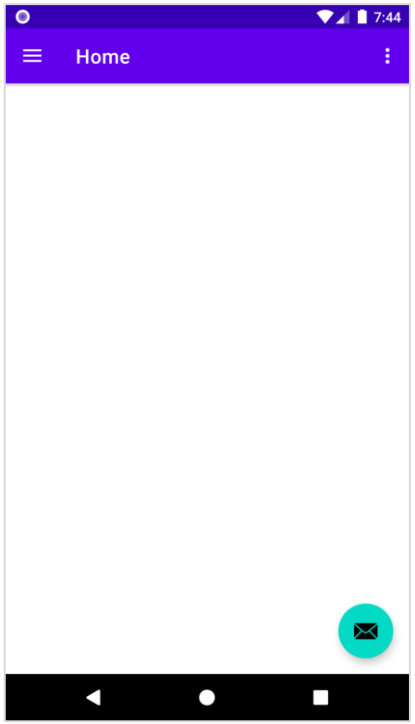
App bar

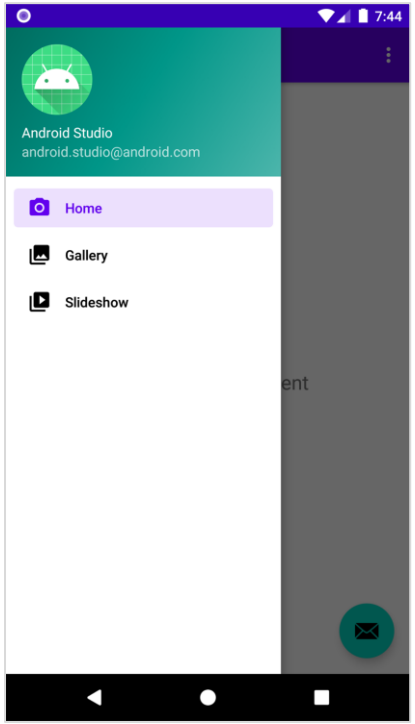




This work is licensed under the Apache 2 license.

Navigation drawer





This work is licensed under the Apache 2 license.

Menu

Define menu items in XML menu resource (located in `res/menu` folder)

```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">
    <item
        android:id="@+id/action_settings"
        android:orderInCategory="100"
        android:title="@string/action_settings"
        app:showAsAction="never" />
</menu>
```


This work is licensed under the Apache 2 license.

More menu options

```
<menu>
    <group android:checkableBehavior="single">
        <item
            android:id="@+id/nav_home"
            android:icon="@drawable/ic_menu_camera"
            android:title="@string/menu_home" />
        <item
            android:id="@+id/nav_gallery"
            android:icon="@drawable/ic_menu_gallery"
            android:title="@string/menu_gallery" />
        <item
            android:id="@+id/nav_slideshow"
            android:icon="@drawable/ic_menu_slideshow"
            android:title="@string/menu_slideshow" />
    </group>
</menu>
```

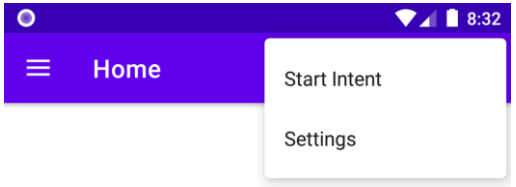

This work is licensed under the Apache 2 license.

Options menu example

```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">

    <item android:id="@+id/action_intent"
        android:title="@string/action_intent" />

    <item
        android:id="@+id/action_settings"
        android:orderInCategory="100"
        android:title="@string/action_settings"
        app:showAsAction="never" />
</menu>
```



This work is licensed under the Apache 2 license.

Inflate options menu

```
override fun onCreateOptionsMenu(menu: Menu): Boolean {
    menuInflater.inflate(R.menu.main, menu)
    return true
}
```


This work is licensed under the Apache 2 license.

Handle menu options selected

```
override fun onOptionsItemSelected(item: MenuItem): Boolean {

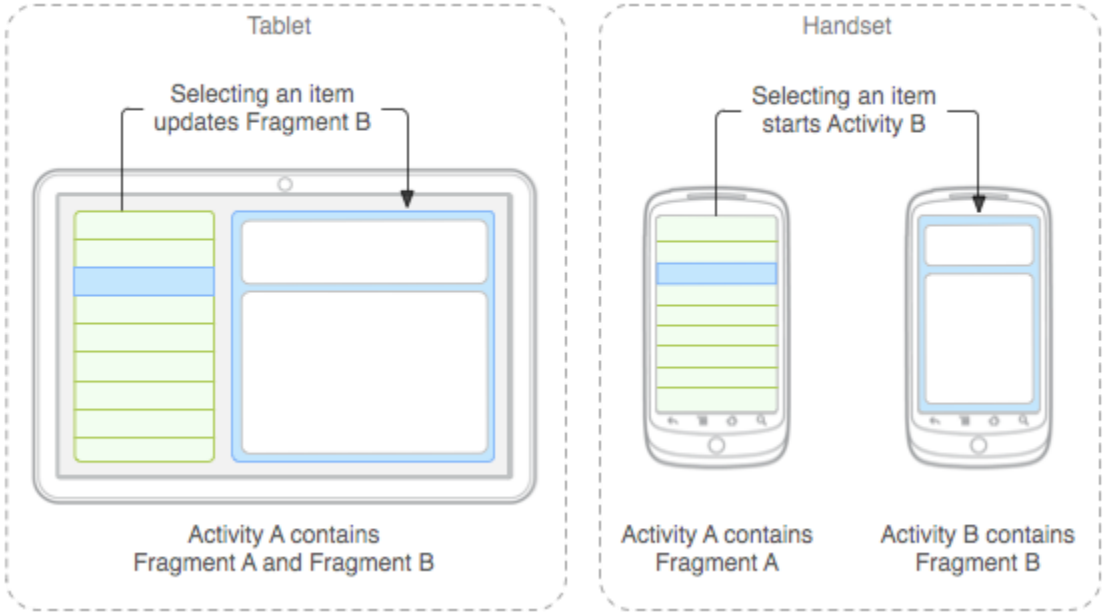
    when (item.itemId) {
        R.id.action_intent -> {
            val intent = Intent(Intent.ACTION_WEB_SEARCH)
            intent.putExtra(SearchManager.QUERY, "pizza")
            if (intent.resolveActivity(packageManager) != null) {
                startActivity(intent)
            }
        }
        else -> Toast.makeText(this, item.title, Toast.LENGTH_LONG).show()
    ...
```


This work is licensed under the Apache 2 license.

Fragments

This work is licensed under the Apache 2 license.

Fragments for tablet layouts



This work is licensed under the Apache 2 license.

Fragment

- Represents a behavior or portion of the UI in an activity ("microactivity")
- Must be hosted in an activity
- Lifecycle tied to host activity's lifecycle
- Can be added or removed at runtime

This work is licensed under the Apache 2 license.

Note about fragments

Use the AndroidX version of the `Fragment` class. (`androidx.fragment.app.Fragment`).

Don't use the platform version of the `Fragment` class (`android.app.Fragment`), which was deprecated.

This work is licensed under the Apache 2 license.

Navigation within an app

This work is licensed under the Apache 2 license.

Navigation component

- Collection of libraries and tooling, including an integrated editor, for creating navigation paths through an app
- Assumes one `Activity` per graph with many `Fragment` destinations
- Consists of three major parts:
  - Navigation graph
  - Navigation Host (`NavHost`)
  - Navigation Controller (`NavController`)

This work is licensed under the Apache 2 license.

Add dependencies

In `build.gradle`, under `dependencies`:

```
implementation "androidx.navigation:navigation-fragment-ktx:$nav_version"

implementation "androidx.navigation:navigation-ui-ktx:$nav_version"
```


This work is licensed under the Apache 2 license.

Navigation host (NavHost)

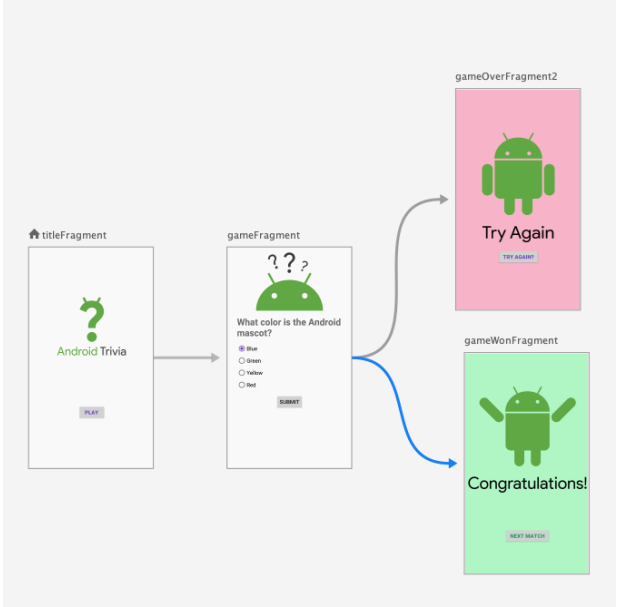
```
<fragment
    android:id="@+id/nav_host"
    android:name="androidx.navigation.fragment.NavHostFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:defaultNavHost="true"
    app:navGraph="@navigation/nav_graph_name"/>
```


This work is licensed under the Apache 2 license.

Navigation graph

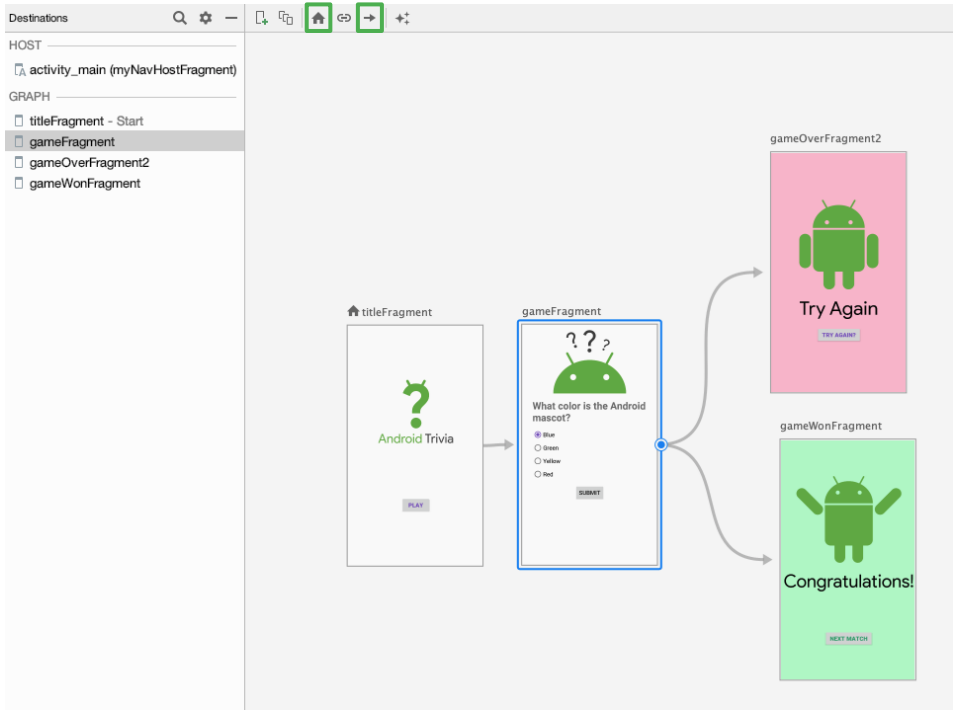
New resource type located in `res/navigation` directory

- XML file containing all of your navigation destinations and actions
- Lists all the (Fragment/Activity) destinations that can be navigated to
- Lists the associated actions to traverse between them
- Optionally lists animations for entering or exiting



This work is licensed under the Apache 2 license.

Navigation Editor in Android Studio



This work is licensed under the Apache 2 license.

Creating a Fragment

- Extend `Fragment` class
- Override `onCreateView()`
- Inflate a layout for the Fragment that you have defined in XML

```
class DetailFragment : Fragment() {

    override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?,
            savedInstanceState: Bundle?): View? {
        return inflater.inflate(R.layout.detail_fragment, container, false)
    }
}
```


This work is licensed under the Apache 2 license.

Specifying Fragment destinations

- Fragment destinations are denoted by the `action` tag in the navigation graph.
- Actions can be defined in XML directly or in the Navigation Editor by dragging from source to destination.
- Autogenerated action IDs take the form of `action_<sourceFragment>_to_<destinationFragment>`.

This work is licensed under the Apache 2 license.

Example fragment destination

```
<fragment
    android:id="@+id/welcomeFragment"
    android:name="com.example.android.navigation.WelcomeFragment"
    android:label="fragment_welcome"
    tools:layout="@layout/fragment_welcome" >

    <action
        android:id="@+id/action_welcomeFragment_to_detailFragment"
        app:destination="@id/detailFragment" />

</fragment>
```


This work is licensed under the Apache 2 license.

Navigation Controller (NavController)

`NavController` manages UI navigation in a navigation host.

- Specifying a destination path only names the action, but it doesn't execute it.
- To follow a path, use `NavController`.

This work is licensed under the Apache 2 license.

Example NavController

```
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        ...
        val navController = findNavController(R.id.myNavHostFragment)
    }

    fun navigateToDetail() {
        navController.navigate(R.id.action_welcomeFragment_to_detailFragment)
    }
}
```


This work is licensed under the Apache 2 license.

More custom navigation behavior

This work is licensed under the Apache 2 license.

Passing data between destinations

Using Safe Args:

- Ensures arguments have a valid type
- Lets you provide default values
- Generates a `<SourceDestination>Directions` class with methods for every action in that destination
- Generates a class to set arguments for every named action
- Generates a `<TargetDestination>Args` class providing access to the destination's arguments

This work is licensed under the Apache 2 license.

Setting up Safe Args

In the project `build.gradle` file:

```
buildscript {
    repositories {
        google()
    }
    dependencies {
        classpath "androidx.navigation:navigation-safe-args-gradle-plugin:$nav_version"
    }
}
```

In the app's or module's `build.gradle` file:

```
apply plugin: "androidx.navigation.safeargs.kotlin"
```


This work is licensed under the Apache 2 license.

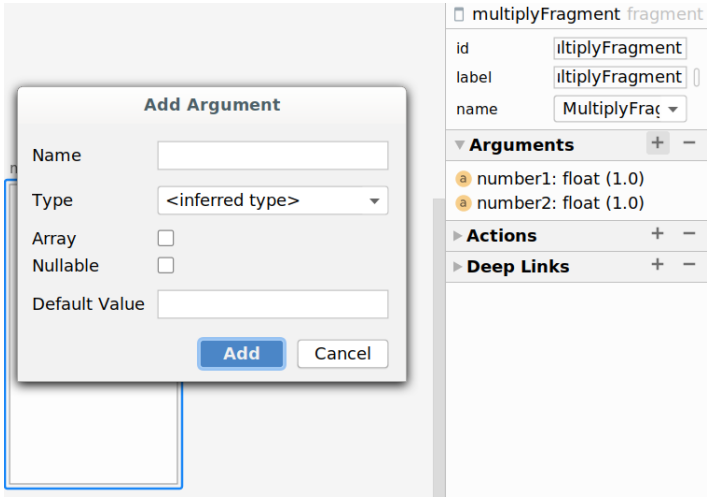
Sending data to a Fragment

1. Create arguments the destination fragment will expect.
2. Create action to link from source to destination.
3. Set the arguments in the action method on `<Source>FragmentDirections`.
4. Navigate according to that action using the Navigation Controller.
5. Retrieve the arguments in the destination fragment.

This work is licensed under the Apache 2 license.

Destination arguments

```
<fragment
    android:id="@+id/multiplyFragment"
    android:name="com.example.arithmetic.MultiplyFragment"
    android:label="MultiplyFragment" >
    <argument
        android:name="number1"
        app:argType="float"
        android:defaultValue="1.0" />
    <argument
        android:name="number2"
        app:argType="float"
        android:defaultValue="1.0" />
</fragment>
```



This work is licensed under the Apache 2 license.

Supported argument types

Type	**Type Syntax** `app:argType=<type>`	**Supports Default Values**	**Supports Null Values**
Integer	`"integer"`	Yes	No
Float	`"float"`	Yes	No
Long	`"long"`	Yes	No
Boolean	`"boolean"`	Yes (`"true"` or `"false"`)	No
String	`"string"`	Yes	Yes
Array	above type + `"[]"` (for example, `"string[]"` `"long[]"`)	Yes (only `"@null"`)	Yes
Enum	Fully qualified name of the enum	Yes	No
Resource reference	`"reference"`	Yes	No

This work is licensed under the Apache 2 license.

Supported argument types: Custom classes

Type	**Type Syntax** `app:argType=<type>`	**Supports Default Values**	**Supports Null Values**
Serializable	Fully qualified class name	Yes (only "`@null`")	Yes
Parcelable	Fully qualified class name	Yes (only "`@null`")	Yes

This work is licensed under the Apache 2 license.

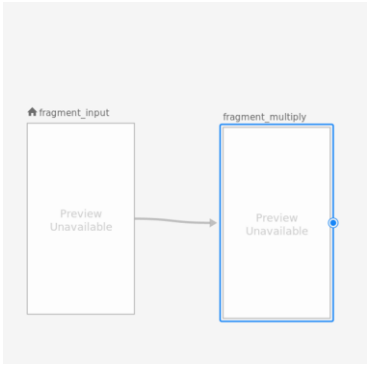
Create action from source to destination

In `nav_graph.xml`:

```
<fragment
    android:id="@+id/fragment_input"
    android:name="com.example.arithmetic.InputFragment">

    <action
        android:id="@+id/action_to_multiplyFragment"
        app:destination="@id/multiplyFragment" />

</fragment>
```



This work is licensed under the Apache 2 license.

Navigating with actions

In `InputFragment.kt`:

```
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    super.onViewCreated(view, savedInstanceState)
    binding.button.setOnClickListener {
        val n1 = binding.number1.text.toString().toFloatOrNull() ?: 0.0
        val n2 = binding.number2.text.toString().toFloatOrNull() ?: 0.0

        val action = InputFragmentDirections.actionToMultiplyFragment(n1, n2)
        view.findNavController().navigate(action)
    }
}
```


This work is licensed under the Apache 2 license.

Retrieving Fragment arguments

```
class MultiplyFragment : Fragment() {
    val args: MultiplyFragmentArgs by navArgs()
    lateinit var binding: FragmentMultiplyBinding
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        val number1 = args.number1
        val number2 = args.number2
        val result = number1 * number2
        binding.output.text = "${number1} * ${number2} = ${result}"
    }
}
```


This work is licensed under the Apache 2 license.

Navigation UI

This work is licensed under the Apache 2 license.

Menus revisited

```
override fun onOptionsItemSelected(item: MenuItem): Boolean {
    val navController = findNavController(R.id.nav_host_fragment)
    return item.onNavDestinationSelected(navController) ||
            super.onOptionsItemSelected(item)
}
```


This work is licensed under the Apache 2 license.

DrawerLayout for navigation drawer

```
<androidx.drawerlayout.widget.DrawerLayout
    android:id="@+id/drawer_layout" ...>

    <fragment
        android:name="androidx.navigation.fragment.NavHostFragment"
        android:id="@+id/nav_host_fragment" ... />

    <com.google.android.material.navigation.NavigationView
        android:id="@+id/nav_view"
        app:menu="@menu/activity_main_drawer" ... />


</androidx.drawerlayout.widget.DrawerLayout>
```


This work is licensed under the Apache 2 license.

Finish setting up navigation drawer

Connect `DrawerLayout` to navigation graph:

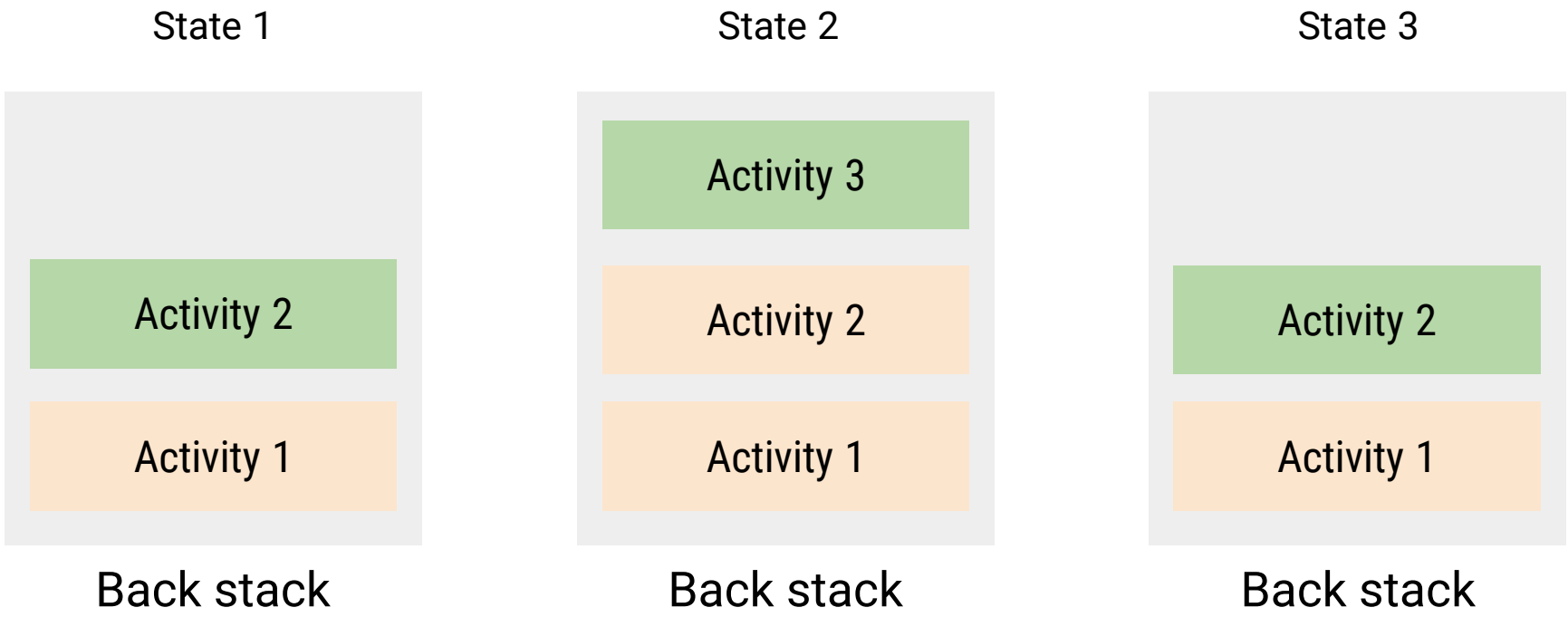
```
val appBarConfiguration = AppBarConfig(navController.graph, drawer)
```

Set up `NavigationView` for use with a `NavController`:

```
val navView = findViewById<NavigationView>(R.id.nav_view)
navView.setupWithNavController(navController)
```

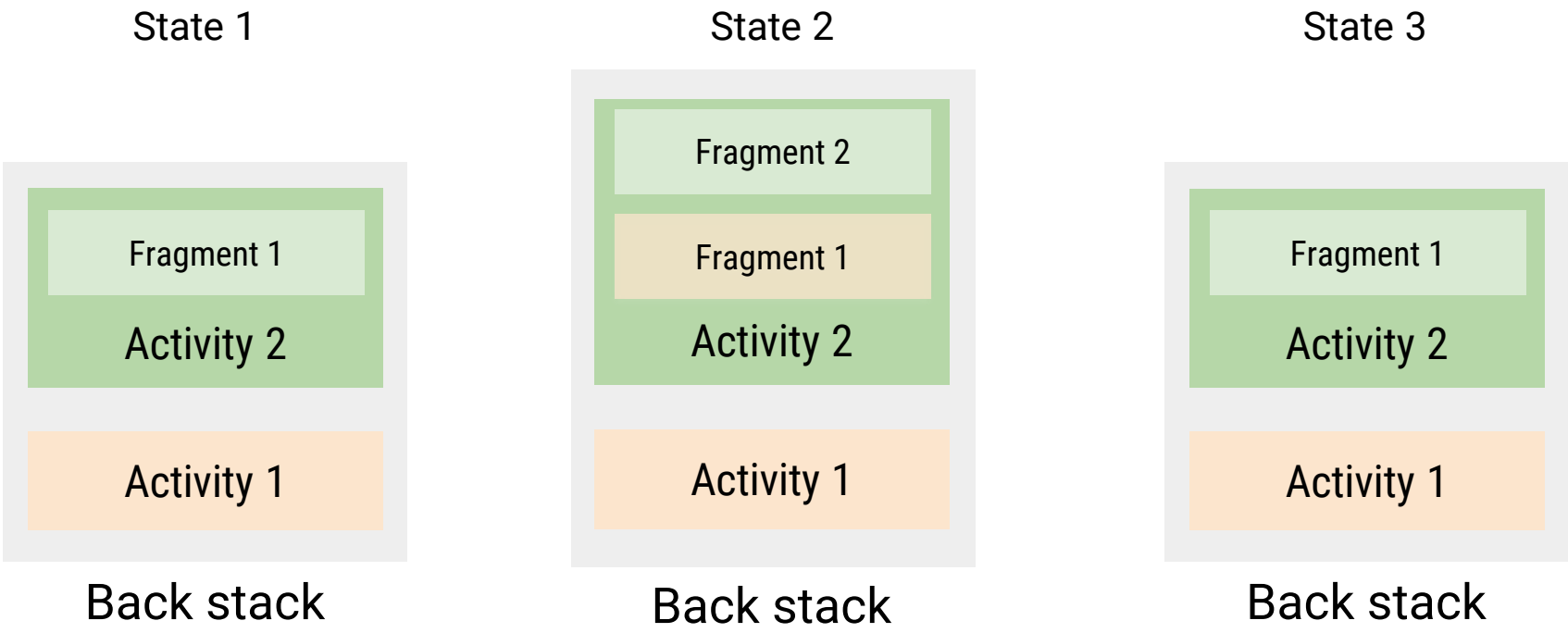

This work is licensed under the Apache 2 license.

Understanding the back stack



This work is licensed under the Apache 2 license.

Fragments and the back stack



This work is licensed under the Apache 2 license.

Summary

This work is licensed under the Apache 2 license.

Summary

In Lesson 6, you learned how to:

- Use explicit and implicit intents to navigate between Activities
- Structure apps using fragments instead of putting all UI code in the Activity
- Handle navigation with NavGraph, NavHost, and NavController
- Use Safe Args to pass data between fragment destinations
- Use NavigationUI to hook up top app bar, navigation drawer, and bottom navigation
- Android keeps a back stack of all the destinations you've visited, with each new destination being pushed onto the stack.

This work is licensed under the Apache 2 license.

Learn more

- Principles of navigation
- Navigation component
- Pass data between destinations
- NavigationUI

This work is licensed under the Apache 2 license.

Pathway

Practice what you've learned by completing the pathway:

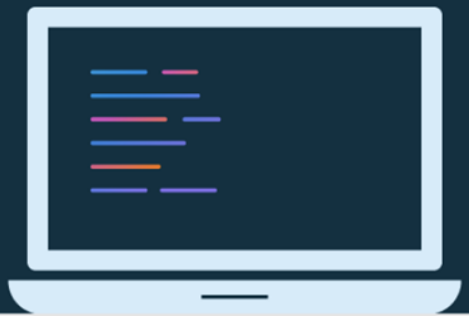
Lesson 6: App navigation



This work is licensed under the Apache 2 license.



Lesson 7: Activity and fragment lifecycles



This work is licensed under the Apache 2 license.

About this lesson

Lesson 7: Activity and fragment lifecycles

- Activity lifecycle
- Logging
- Fragment lifecycle
- Lifecycle-aware components
- Tasks and back stack
- Summary

This work is licensed under the Apache 2 license.

Activity lifecycle

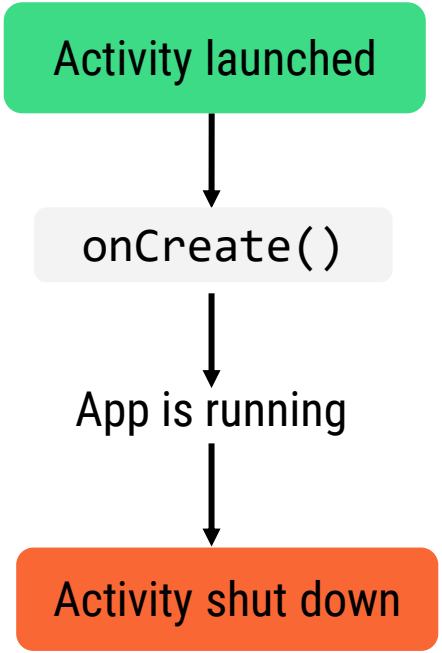
This work is licensed under the Apache 2 license.

Why it matters

- Preserve user data and state if:
  - User temporarily leaves app and then returns
  - User is interrupted (for example, a phone call)
  - User rotates device
- Avoid memory leaks and app crashes.

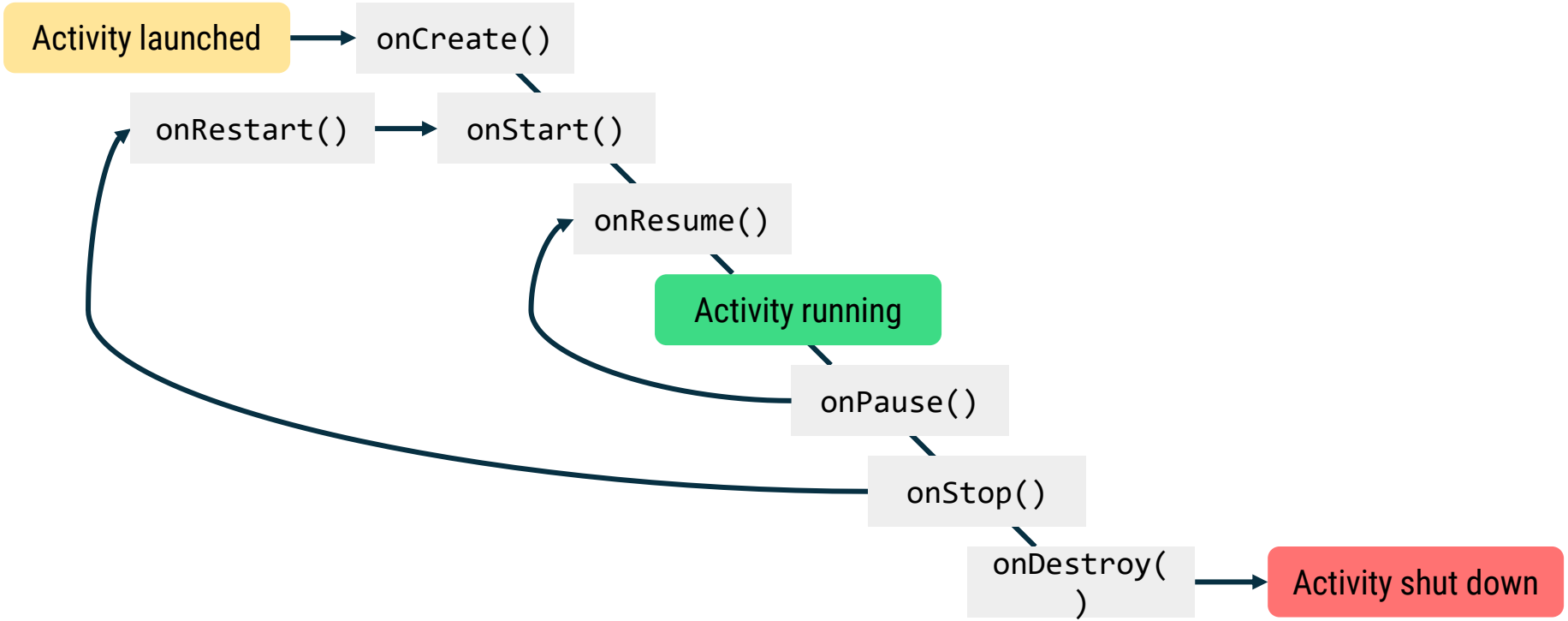
This work is licensed under the Apache 2 license.

Simplified activity lifecycle



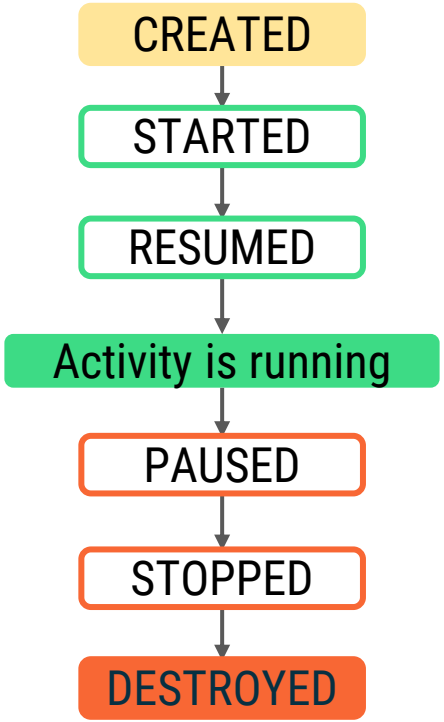
This work is licensed under the Apache 2 license.

Activity lifecycle



This work is licensed under the Apache 2 license.

Activity states



This work is licensed under the Apache 2 license.

onCreate()

- Activity is created and other initialization work occurs
- You must implement this callback
- Inflate activity UI and perform other app startup logic

This work is licensed under the Apache 2 license.

onStart()

- Activity becomes visible to the user
- Called after activity:
  - `onCreate()`

    or
  - `onRestart()` if activity was previously stopped

This work is licensed under the Apache 2 license.

onResume()

- Activity gains input focus:
  - User can interact with the activity
- Activity stays in resumed state until system triggers activity to be paused

This work is licensed under the Apache 2 license.

onPause()

- Activity has lost focus (not in foreground)
- Activity is still visible, but user is not actively interacting with it
- Counterpart to `onResume()`

This work is licensed under the Apache 2 license.

onStop()

- Activity is no longer visible to the user
- Release resources that aren't needed anymore
- Save any persistent state that the user is in the process of editing so they don't lose their work

This work is licensed under the Apache 2 license.

onDestroy()

- Activity is about to be destroyed, which can be caused by:
  - Activity has finished or been dismissed by the user
  - Configuration change
- Perform any final cleanup of resources.
- Don’t rely on this method to save user data (do that earlier)

This work is licensed under the Apache 2 license.

Summary of activity states

State	Callbacks	Description
Created	`onCreate()`	Activity is being initialized.
Started	`onStart()`	Activity is visible to the user.
Resumed	`onResume()`	Activity has input focus.
Paused	`onPause()`	Activity does not have input focus.
Stopped	`onStop()`	Activity is no longer visible.
Destroyed	`onDestroy()`	Activity is destroyed.

This work is licensed under the Apache 2 license.

Save state

User expects UI state to stay the same after a config change or if the app is terminated when in the background.

- Activity is destroyed and restarted, or app is terminated and activity is started.
- Store user data needed to reconstruct app and activity Lifecycle changes:
  - Use `Bundle` provided by `onSaveInstanceState()`.
  - `onCreate()` receives the `Bundle` as an argument when activity is created again.

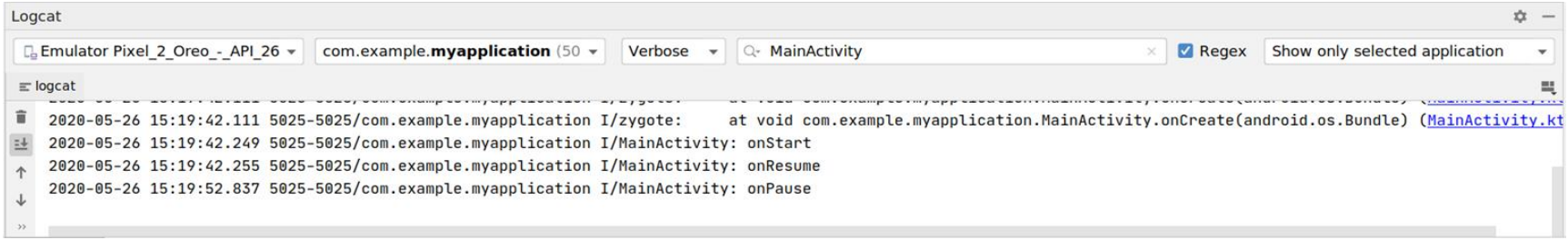
This work is licensed under the Apache 2 license.

Logging

This work is licensed under the Apache 2 license.

Logging in Android

- Monitor the flow of events or state of your app.
- Use the built-in `Log` class or third-party library.
- Example `Log` method call: `Log.d(TAG, "Message")`



This work is licensed under the Apache 2 license.

Write logs

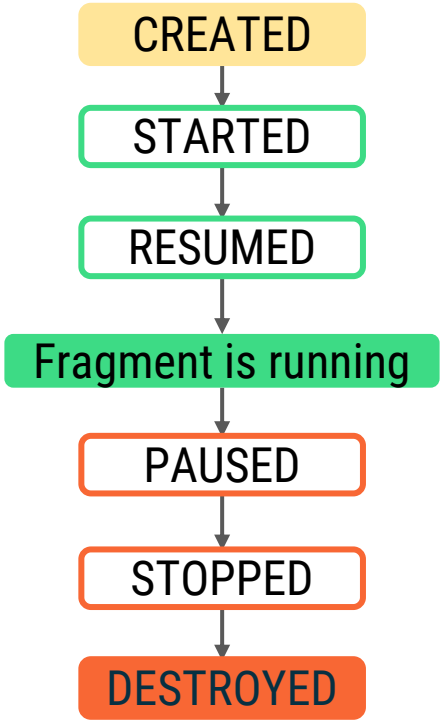
Priority level	Log method
Verbose	`Log.v(String, String)`
Debug	`Log.d(String, String)`
Info	`Log.i(String, String)`
Warning	`Log.w(String, String)`
Error	`Log.e(String, String)`

This work is licensed under the Apache 2 license.

Fragment lifecycle

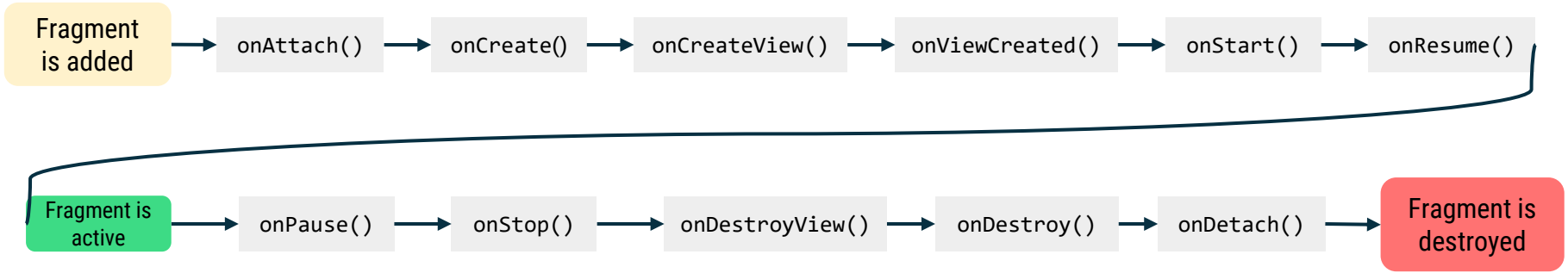
This work is licensed under the Apache 2 license.

Fragment states



This work is licensed under the Apache 2 license.

Fragment lifecycle diagram



This work is licensed under the Apache 2 license.

onAttach()

- Called when a fragment is attached to a context
- Immediately precedes `onCreate()`

This work is licensed under the Apache 2 license.

onCreateView()

- Called to create the view hierarchy associated with the fragment
- Inflate the fragment layout here and return the root view

This work is licensed under the Apache 2 license.

onViewCreated()

- Called when view hierarchy has already been created
- Perform any remaining initialization here (for example, restore state from `Bundle`)

This work is licensed under the Apache 2 license.

onDestroyView() and onDetach()

- `onDestroyView()` is called when view hierarchy of fragment is removed.
- `onDetach()` is called when fragment is no longer attached to the host.

This work is licensed under the Apache 2 license.

Summary of fragment states

State	Callbacks	Description
Initialized	`onAttach()`	Fragment is attached to host.
Created	`onCreate()`, `onCreateView()`, `onViewCreated()`	Fragment is created and layout is being initialized.
Started	`onStart()`	Fragment is started and visible.
Resumed	`onResume()`	Fragment has input focus.
Paused	`onPause()`	Fragment no longer has input focus.
Stopped	`onStop()`	Fragment is not visible.
Destroyed	`onDestroyView()`, `onDestroy()`, `onDetach()`	Fragment is removed from host.

This work is licensed under the Apache 2 license.

Save fragment state across config changes

Preserve UI state in fragments by storing state in `Bundle`:

- `onSaveInstanceState(outState: Bundle)`

Retrieve that data by receiving the Bundle in these fragment callbacks:

- `onCreate()`
- `onCreateView()`
- `onViewCreated()`

This work is licensed under the Apache 2 license.

Lifecycle-aware components

This work is licensed under the Apache 2 license.

Lifecycle-aware components

Adjust their behavior based on activity or fragment lifecycle

- Use the `androidx.lifecycle` library
- `Lifecycle` tracks the lifecycle state of an activity or fragment
  - Holds current lifecycle state
  - Dispatches lifecycle events (when there are state changes)

This work is licensed under the Apache 2 license.

LifecycleOwner

- Interface that says this class has a lifecycle
- Implementers must implement `getLifecycle()` method

  Examples: `Fragment` and `AppCompatActivity` are implementations of `LifecycleOwner`

This work is licensed under the Apache 2 license.

LifecycleObserver

Implement `LifecycleObserver` interface:

```
class MyObserver : LifecycleObserver {
    @OnLifecycleEvent(Lifecycle.Event.ON_RESUME)
    fun connectListener() {
        ...
    }
```

Add the observer to the lifecycle:

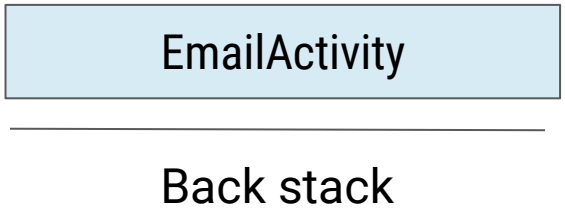
```
myLifecycleOwner.getLifecycle().addObserver(MyObserver())
```


This work is licensed under the Apache 2 license.

Tasks and back stack

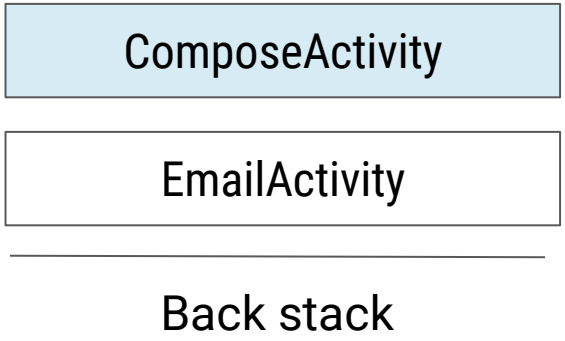
This work is licensed under the Apache 2 license.

Back stack of activities



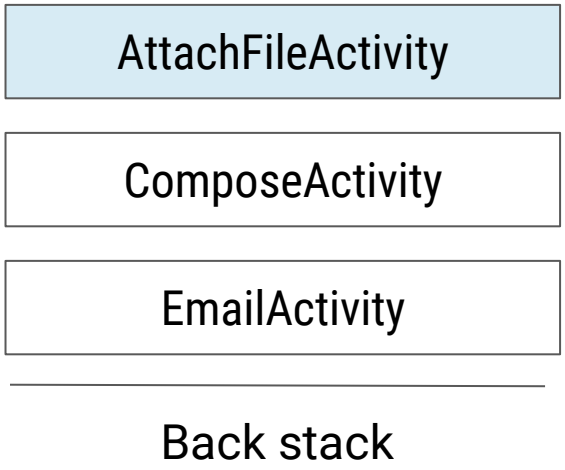
This work is licensed under the Apache 2 license.

Add to the back stack



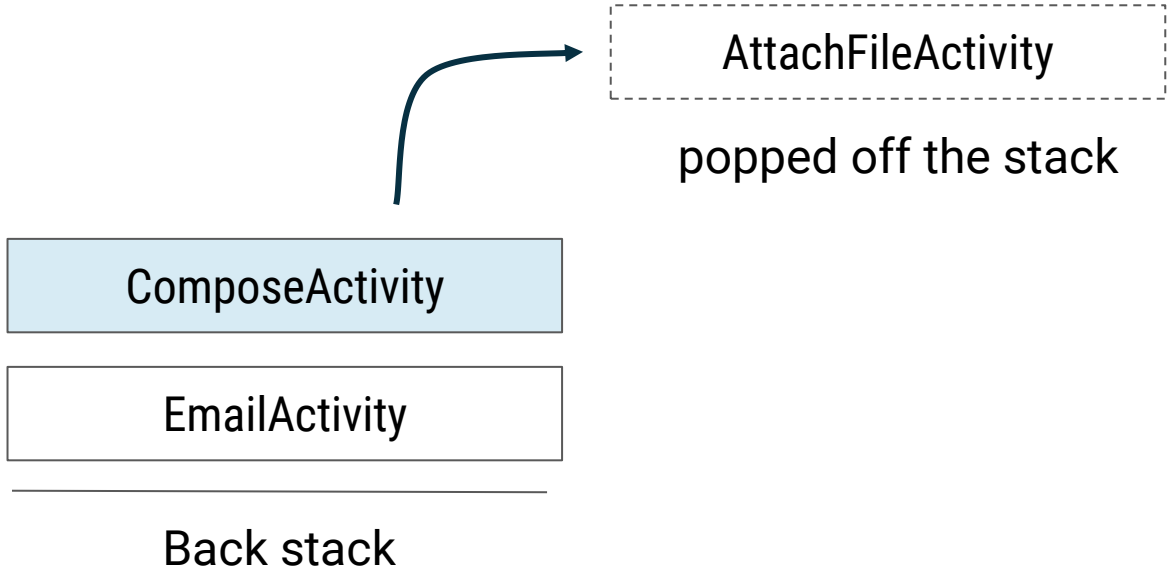
This work is licensed under the Apache 2 license.

Add to the back stack again



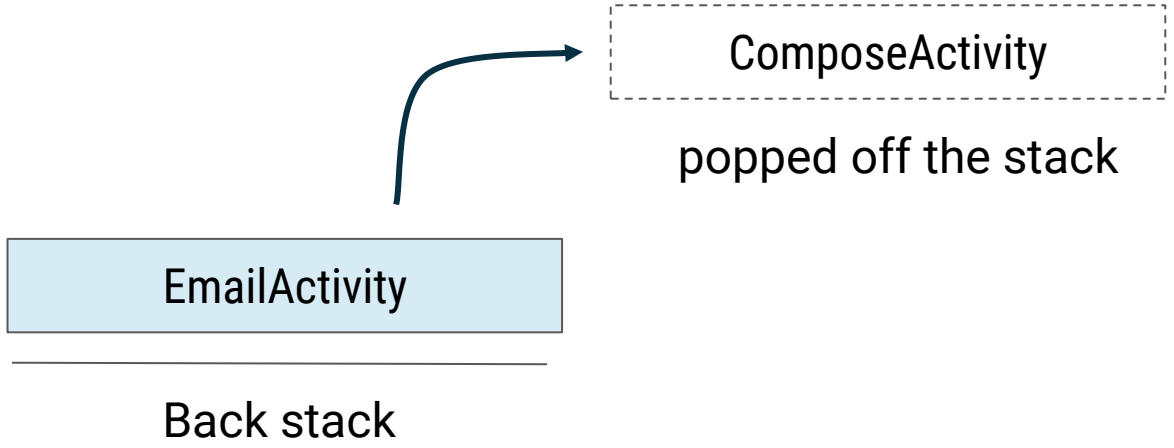
This work is licensed under the Apache 2 license.

Tap Back button



This work is licensed under the Apache 2 license.

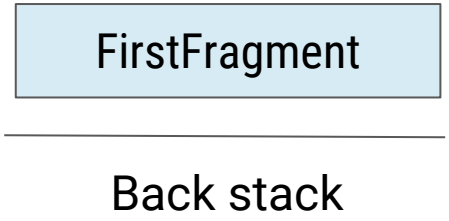
Tap Back button again



This work is licensed under the Apache 2 license.

First destination in the back stack

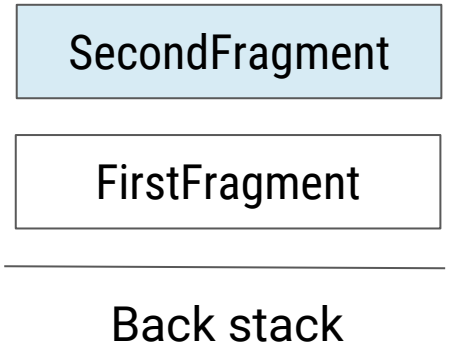




This work is licensed under the Apache 2 license.

Add a destination to the back stack

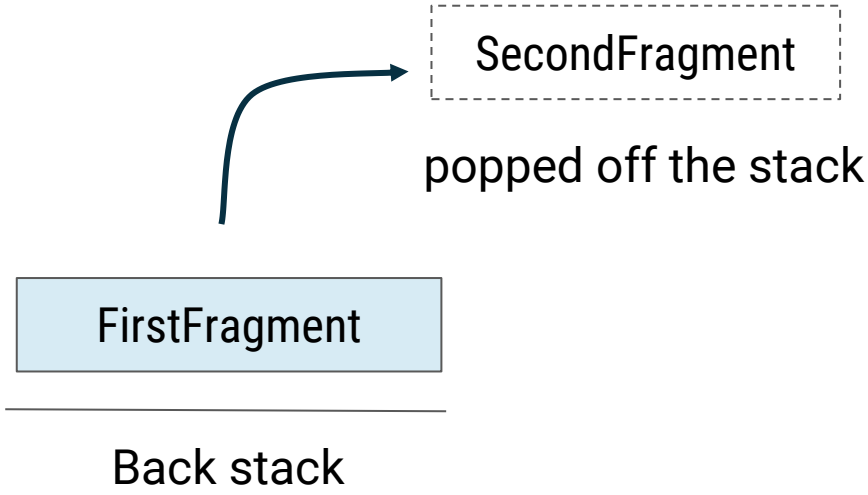




This work is licensed under the Apache 2 license.

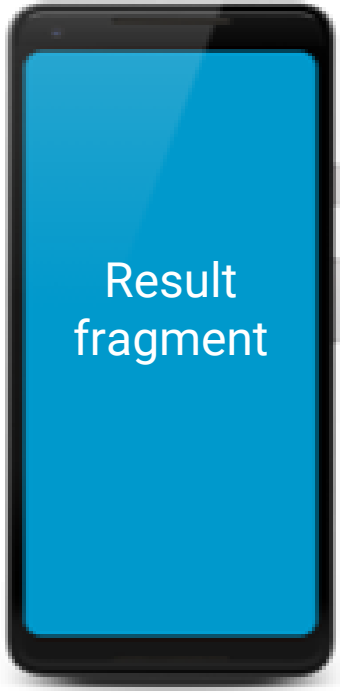
Tap Back button





This work is licensed under the Apache 2 license.

Another back stack example



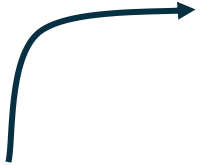


This work is licensed under the Apache 2 license.

Modify Back button behavior



pop additional destinations
off the back stack



ResultFragment

Question3Fragment

Question2Fragment

Question1Fragment

WelcomeFragment

Back stack

This work is licensed under the Apache 2 license.

Summary

This work is licensed under the Apache 2 license.

Summary

In Lesson 7, you learned how to:

- Understand how an activity instance transitions through different lifecycle states as the user interacts with or leaves your app
- Reserve UI state across configuration changes using a `Bundle`
- Fragment lifecycle callback methods similar to activity, but with additions
- Use lifecycle-aware components help organize your app code
- Use default or custom back stack behavior
- Use logging to help debug and track the state of the app

This work is licensed under the Apache 2 license.

Learn more

- Understand the Activity Lifecycle
- Activity class
- Fragments guide and lifecycle
- Fragment class

This work is licensed under the Apache 2 license.

Pathway

Practice what you've learned by completing the pathway:

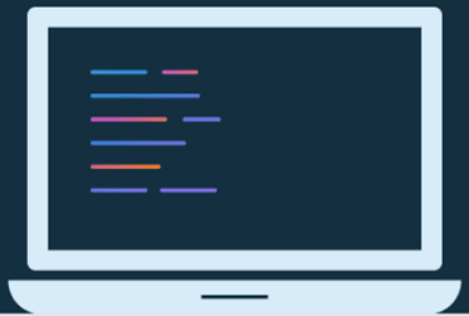
Lesson 7: Activity and Fragment Lifecycles



This work is licensed under the Apache 2 license.



Lesson 8: App architecture (UI layer)



This work is licensed under the Apache 2 license.

About this lesson

Lesson 8: App architecture (UI layer)

- Android app architecture
- ViewModel
- Data binding
- LiveData
- Transform LiveData
- Summary

This work is licensed under the Apache 2 license.

Android app architecture

This work is licensed under the Apache 2 license.

Avoid short-term hacks

- External factors, such as tight deadlines, can lead to poor decisions about app design and structure.
- Decisions have consequences for future work (app can be harder to maintain long-term).
- Need to balance on-time delivery and future maintenance burden.

This work is licensed under the Apache 2 license.

Examples of short-term hacks

- Tailoring your app to a specific device
- Blindly copying and pasting code into your files
- Placing all business logic in activity file
- Hardcoding user-facing strings in your code

This work is licensed under the Apache 2 license.

Why you need good app architecture

- Clearly defines where specific business logic belongs
- Makes it easier for developers to collaborate
- Makes your code easier to test
- Lets you benefit from already-solved problems
- Saves time and reduces technical debt as you extend your app

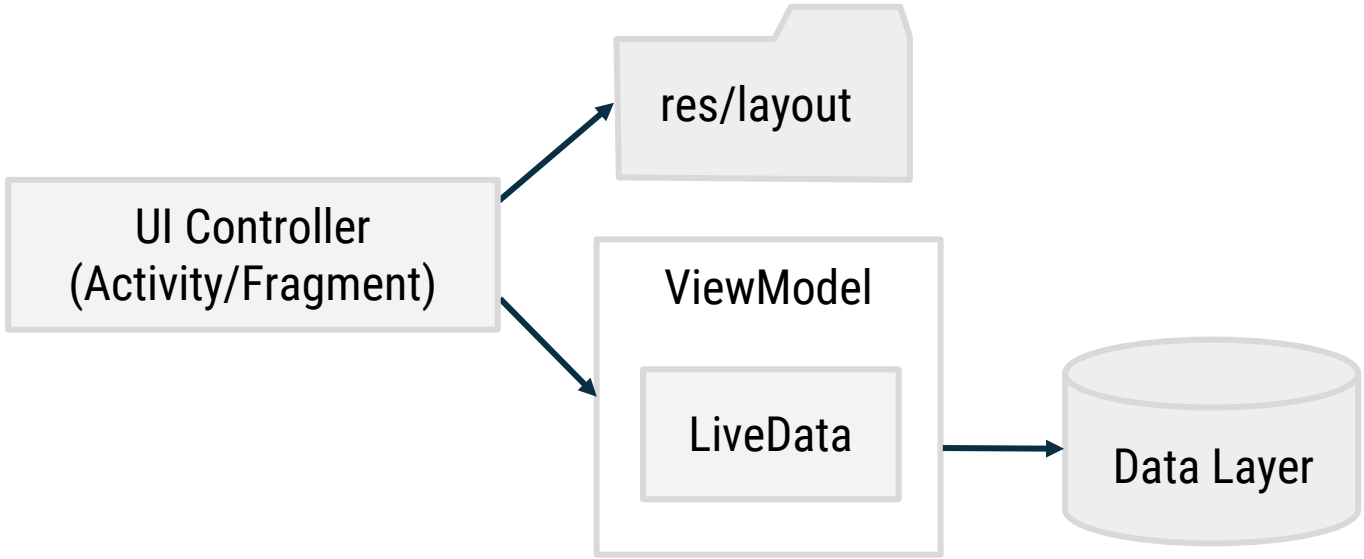
This work is licensed under the Apache 2 license.

Android Jetpack

- Android libraries that incorporate best practices and provide backward compatibility in your apps
- Jetpack comprises the `androidx.*` package libraries

This work is licensed under the Apache 2 license.

Separation of concerns



This work is licensed under the Apache 2 license.

Architecture components

- Architecture design patterns, like MVVM and MVI, describe a loose template for what the structure of your app should be.
- Jetpack architecture components help you design robust, testable, and maintainable apps.

This work is licensed under the Apache 2 license.

ViewModel

This work is licensed under the Apache 2 license.

Gradle: lifecycle extensions

In `app/build.gradle` file:

```
dependencies {
    implementation "androidx.lifecycle:lifecycle-viewmodel-ktx:$lifecycle_version"
    implementation "androidx.activity:activity-ktx:$activity_version"
}
```

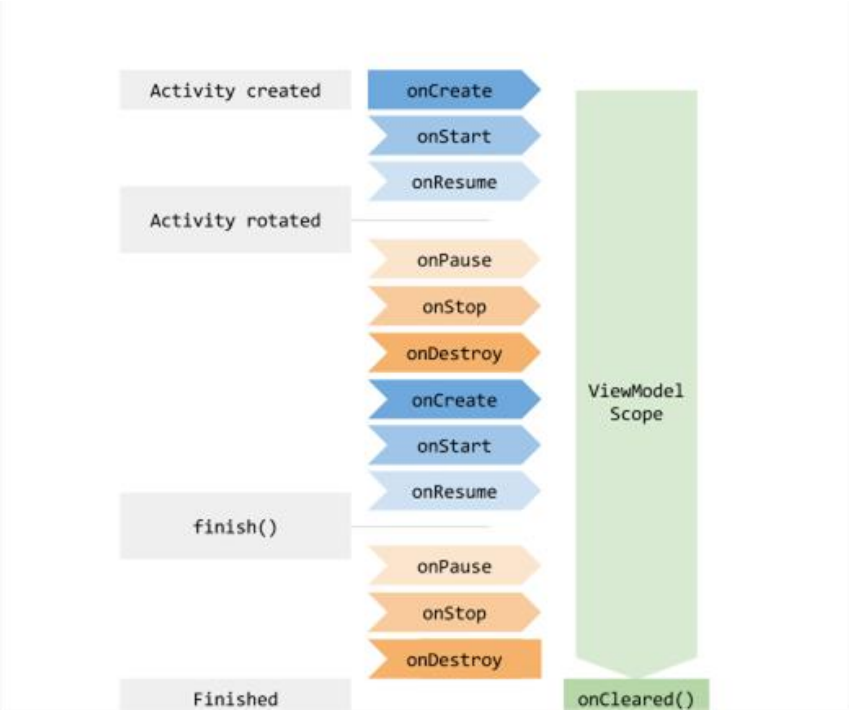

This work is licensed under the Apache 2 license.

ViewModel

- Prepares data for the UI
- Must not reference activity, fragment, or views in view hierarchy
- Scoped to a lifecycle (which activity and fragment have)
- Enables data to survive configuration changes
- Survives as long as the scope is alive

This work is licensed under the Apache 2 license.

Lifetime of a ViewModel



This work is licensed under the Apache 2 license.

Kabaddi Kounter



This work is licensed under the Apache 2 license.

ViewModel class

`abstract class ViewModel`

Summary

Public constructors	
`<init>()`	
ViewModel is a class that is responsible for preparing and managing the data for an `Activity` or a `Fragment`.	

Protected methods	
open Unit	`onCleared()` This method will be called when this ViewModel is no longer used and will be destroyed.

Extension properties	
From androidx.lifecycle	
CoroutineScope	`viewModelScope` CoroutineScope tied to this ViewModel.

This work is licensed under the Apache 2 license.

Implement a ViewModel

```
class ScoreViewModel : ViewModel() {
    var scoreA : Int = 0
    var scoreB : Int = 0
    fun incrementScore(isTeamA: Boolean) {
        if (isTeamA) {
            scoreA++
        }
        else {
            scoreB++
        }
    }
}
```


This work is licensed under the Apache 2 license.

Load and use a ViewModel

```
class MainActivity : AppCompatActivity() {
    // Delegate provided by androidx.activity.viewModels
    val viewModel: ScoreViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        ...
        val scoreViewA: TextView = findViewById(R.id.scoreA)
        scoreViewA.text = viewModel.scoreA.toString()
    }
```


This work is licensed under the Apache 2 license.

Using a ViewModel

Within `MainActivity onCreate()`:

```
val scoreViewA: TextView = findViewById(R.id.scoreA)
val plusOneButtonA: Button = findViewById(R.id.plusOne_teamA)

plusOneButtonA.setOnClickListener {
    viewModel.incrementScore(true)
    scoreViewA.text = viewModel.scoreA.toString()
}
```

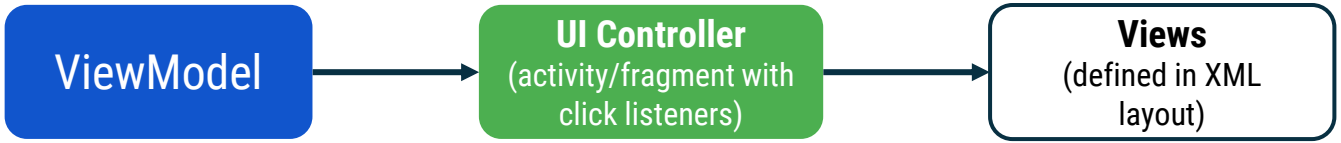

This work is licensed under the Apache 2 license.

Data binding

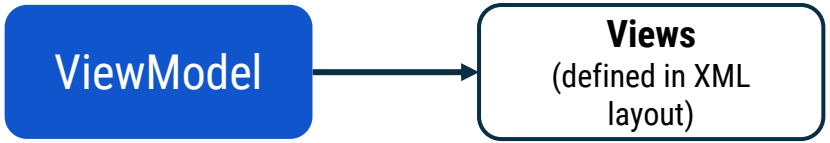
This work is licensed under the Apache 2 license.

ViewModels and data binding

- App architecture without data binding



- ViewModels can work in concert with data binding



This work is licensed under the Apache 2 license.

Data binding in XML revisited

Specify ViewModels in the `data` tag of a binding.

```
<layout>
    <data>
        <variable>
            name="viewModel"
            type="com.example.kabaddikounter.ScoreViewModel" />
    </data>
    <ConstraintLayout ../>
</layout>
```


This work is licensed under the Apache 2 license.

Attaching a ViewModel to a data binding

```
class MainActivity : AppCompatActivity() {

    val viewModel: ScoreViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val binding: ActivityMainBinding = DataBindingUtil.setContentView(this,
            R.layout.activity_main)

        binding.viewModel = viewModel

        ...
```


This work is licensed under the Apache 2 license.

Using a ViewModel from a data binding

In `activity_main.xml`:

```
<TextView
    android:id="@+id/scoreViewA"
    android:text="@{viewModel.scoreA.toString()}" />
        ...
```


This work is licensed under the Apache 2 license.

ViewModels and data binding

```
override fun onCreate(savedInstanceState: Bundle?) {
    ...
    val binding: ActivityMainBinding = DataBindingUtil.setContentView(this,
        R.layout.activity_main)

    binding.plusOneButtonA.setOnClickListener {
        viewModel.incrementScore(true)
        binding.scoreViewA.text = viewModel.scoreA.toString()
    }
}
```


This work is licensed under the Apache 2 license.

LiveData

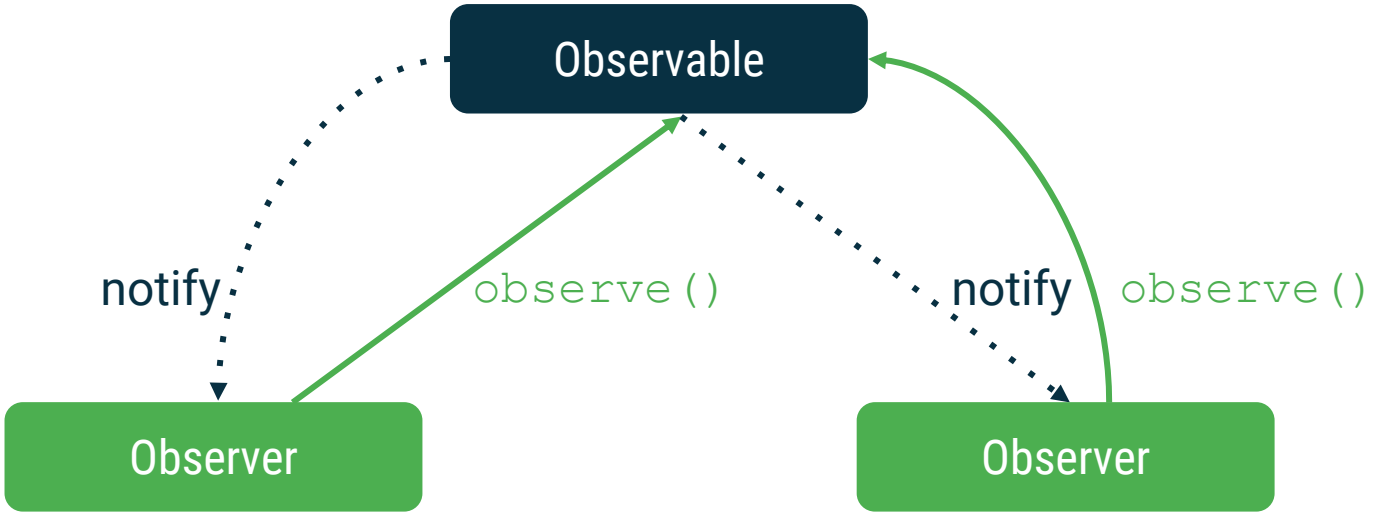
This work is licensed under the Apache 2 license.

Observer design pattern

- Subject maintains list of observers to notify when state changes.
- Observers receive state changes from subject and execute appropriate code.
- Observers can be added or removed at any time.

This work is licensed under the Apache 2 license.

Observer design pattern diagram



This work is licensed under the Apache 2 license.

LiveData

- A lifecycle-aware data holder that can be observed
- Wrapper that can be used with any data including lists (for example, `LiveData<Int>` holds an `Int`)
- Often used by ViewModels to hold individual data fields
- Observers (activity or fragment) can be added or removed
  - `observe(owner: LifecycleOwner, observer: Observer)`
    `removeObserver(observer: Observer)`

This work is licensed under the Apache 2 license.

LiveData versus MutableLiveData

`LiveData<T>`	`MutableLiveData<T>`
• `getValue()`	• `getValue()` • `postValue(value: T)` • `setValue(value: T)`

`T` is the type of data that's stored in `LiveData` or `MutableLiveData`.

This work is licensed under the Apache 2 license.

Use LiveData in ViewModel

```
class ScoreViewModel : ViewModel() {

    private val _scoreA = MutableLiveData<Int>(0)
    val scoreA: LiveData<Int>
        get() = _scoreA

    fun incrementScore(isTeamA: Boolean) {
        if (isTeamA) {
            _scoreA.value = _scoreA.value!! + 1
        }
        ...
```


This work is licensed under the Apache 2 license.

Add an observer on LiveData

Set up click listener to increment `ViewModel` score:

```
binding.plusOneButtonA.setOnClickListener {
    viewModel.incrementScore(true)
}
```

Create observer to update team A score on screen:

```
val scoreA_Observer = Observer<Int> { newValue ->
    binding.scoreViewA.text = newValue.toString()
}
```

Add the observer onto `scoreA LiveData` in `ViewModel`:

```
viewModel.scoreA.observe(this, scoreA_Observer)
```


This work is licensed under the Apache 2 license.

Two-way data binding

- We already have two-way binding with `ViewModel` and `LiveData`.
- Binding to `LiveData` in XML eliminates need for an observer in code.

This work is licensed under the Apache 2 license.

Example layout XML

```
<layout>
    <data>
        <variable>
            name="viewModel"
            type="com.example.kabaddikounter.ScoreViewModel" />
    </data>

    <ConstraintLayout ..>
        <TextView ...
            android:id="@+id/scoreViewA"
            android:text="@{viewModel.scoreA.toString()}" />
        ...
    </ConstraintLayout>
</layout>
```


This work is licensed under the Apache 2 license.

Example Activity

```
class MainActivity : AppCompatActivity() {
    val viewModel: ScoreViewModel by viewModels()
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val binding: ActivityMainBinding = DataBindingUtil
             .setContentView(this, R.layout.activity_main)

        binding.viewModel = viewModel
        binding.lifecycleOwner = this

        binding.plusOneButtonA.setOnClickListener {
            viewModel.incrementScore(true)
        }
        ...
```


This work is licensed under the Apache 2 license.

Example ViewModel

```
class ScoreViewModel : ViewModel() {
    private val _scoreA = MutableLiveData<Int>(0)
    val scoreA : LiveData<Int>
        get() = _scoreA
    private val _scoreB = MutableLiveData<Int>(0)
    val scoreB : LiveData<Int>
        get() = _scoreB
    fun incrementScore(isTeamA: Boolean) {
        if (isTeamA) {
            _scoreA.value = _scoreA.value!! + 1
        } else {
            _scoreB.value = _scoreB.value!! + 1
        }
    }
}
```


This work is licensed under the Apache 2 license.

Transform LiveData

This work is licensed under the Apache 2 license.

Manipulating LiveData with transformations

`LiveData` can be transformed into a new `LiveData` object.

- `map()`
- `switchMap()`

This work is licensed under the Apache 2 license.

Example LiveData with transformations

```
val result: LiveData<String> = Transformations.map(viewModel.scoreA) {
    x -> if (x > 10) "A Wins" else ""
}
```


This work is licensed under the Apache 2 license.

Summary

This work is licensed under the Apache 2 license.

Summary

In Lesson 8, you learned how to:

- Follow good app architecture design, and the separation-of-concerns principle to make apps more maintainable and reduce technical debt
- Create a `ViewModel` to hold data separately from a UI controller
- Use `ViewModel` with data binding to make a responsive UI with less code
- Use observers to automatically get updates from `LiveData`

This work is licensed under the Apache 2 license.

Learn More

- Guide to app architecture
- Android Jetpack
- ViewModel Overview
- Android architecture sample app
- ViewModelProvider
- Lifecycle Aware Data Loading with Architecture Components
- ViewModels and LiveData: Patterns + AntiPatterns

This work is licensed under the Apache 2 license.

Pathway

Practice what you've learned by completing the pathway:

Lesson 8: App architecture (UI layer)



This work is licensed under the Apache 2 license.